machine learning is one of the top technologies of the 21st century it has brought in a revolution in the business sectors hello everyone welcome to this full course video on machine learning this video will help you understand all the concepts and give you ample hands-on experience building machine learning models we have our experience instructors who will take you through this course first you will learn the basics of machine learning and jump into understanding the various machine learning algorithms you will learn about linear regression logistic regression decision trees support vector machines and many more then you will look at reinforcement learning and understand mathematics for machine learning you will learn the concepts such as principal component analysis regularization and feature selection finally you will perform your selection prediction analysis and understand the machine learning roadmap for 2021 so let's begin with a short animated video on machine learning we know humans learn from their past experiences and machines follow instructions given by humans but what if humans can train the machines to learn from the past data and do what humans can do act much faster well that's called machine learning but it's a lot more than just learning it's also about understanding and reasoning so today we will learn about the basics of machine learning so that's paul he loves listening to new songs he either likes them or dislikes them paul decides this on the basis of the song's tempo genre intensity and the gender of voice for simplicity let's just use tempo and intensity for now so here tempo is on the x-axis ranging from relaxed to fast whereas intensity is on the y-axis ranging from light to soaring we see that paul likes the song with fast tempo and soaring intensity while he dislikes the song with relaxed tempo and light intensity so now we know paul's choices let's say paul listens to a new song let's name it as song a song a has fast tempo and a soaring intensity so it lies somewhere here looking at the data can you guess whether paul will like the song or not correct so paul likes this song by looking at paul's past choices we were able to classify the unknown song very easily right let's say now paul listens to a new song let's label it as song b so song b lies somewhere here with medium tempo and medium intensity neither relaxed nor fast neither light nor soaring now can you guess whether ball likes it or not not able to guess whether this ball will like it or dislike it are the choices unclear correct we could easily classify song a but when the choice became complicated as in the case of song b yes and that's where machine learning comes in let's see how in the same example for song b if we draw a circle around the song b we see that there are four votes for like whereas one would for dislike if we go for the majority votes we can say that paul will definitely like the song that's all this was a basic machine learning algorithm also it's called k nearest neighbors so this is just a small example in one of the many machine learning algorithms quite easy right believe me it is but what happens when the choices become complicated as in the case of song b that's when machine learning comes in it learns the data builds the prediction model and when the new data point comes in it can easily predict for it more the data better the model higher will be the accuracy there are many ways in which the machine learns it could be either supervised learning unsupervised learning or reinforcement learning let's first quickly understand supervised learning suppose your friend gives you one million coins of three different currencies say one rupee one euro and one dirham each coin has different weights for example a coin of one rupee weighs three grams one euro weighs seven grams and one dirham weighs four grams your model will predict the currency of the coin here your weight becomes the feature of coins while currency becomes their label when you feed this data to the machine learning model it learns which feature is associated with which label for example it will learn that if a coin is of 3 grams it will be a 1 rupee coin let's give a new coin to the machine on the basis of the weight of the new coin your model will predict the currency hence supervised learning uses labeled data to train the model here the machine knew the features of the object and also the labels associated with those features on this note let's move

to unsupervised learning and see the difference suppose you have cricket data set of various players with their respective scores and wickets taken when you feed this data set to the machine the machine identifies the pattern of player performance so it plots this data with the respective wickets on the x-axis while runs on the y-axis while looking at the data you will clearly see that there are two clusters the one cluster are the players who scored high runs and took less wickets while the other cluster is of the players who scored less runs but took many wickets so here we interpret these two clusters as batsmen and bowlers the important point to note here is that there were no labels of batsmen and bowlers hence the learning with unlabeled data is unsupervised learning so we saw supervised learning where the data was labeled and the unsupervised learning where the data was unlabeled and then there is reinforcement learning which is a reward based learning or we can say that it works on the principle of feedback here let's say you provide the system with an image of a dog and ask it to identify it the system identifies it as a cat so you give a negative feedback to the machine saying that it's a dog's image the machine will learn from the feedback and finally if it comes across any other image of a dog it will be able to classify it correctly that is reinforcement learning to generalize machine learning model let's see a flowchart input is given to a machine learning model which then gives the output according to the algorithm applied if it's right we take the output as a final result else we provide feedback to the training model and ask it to predict until it learns i hope you've understood supervised and unsupervised learning so let's have a quick quiz you have to determine whether the given scenarios uses supervised or unsupervised learning simple right scenario one facebook recognizes your friend in a picture from an album of tagged photographs scenario 2 netflix recommends new movies based on someone's past movie choices scenario 3 analyzing bank data for suspicious transactions and flagging the fraud transactions think wisely and comment below your answers moving on don't you sometimes wonder how is machine learning possible in today's era well that's because today we have humongous data available everybody's online either making a transaction or just surfing the internet and that's generating a huge amount of data every minute and that data my friend is the key to analysis also the memory handling capabilities of computers have largely increased which helps them to process such huge amount of data at hand without any delay and yes computers now have great computational powers so there are a lot of applications of machine learning out there to name a few machine learning is used in healthcare where diagnostics are predicted for doctor's review the sentiment analysis that the tech giants are doing on social media is another interesting application of machine learning fraud detection in the finance sector and also to predict customer churn in the e-commerce sector while booking a gap you must have encountered surge pricing often where it says the fear of your trip has been updated continue booking yes please i'm getting late for office well that's an interesting machine learning model which is used by global taxi giant uber and others where they have differential pricing in real time based on demand the number of cars available bad weather rush r etc so they use the surge pricing model to ensure that those who need a cab can get one also it uses predictive modeling to predict where the demand will be high with the goal that drivers can take care of the demand and search pricing can be minimized great hey siri can you remind me to book a cab at 6 00 pm today okay i'll remind you thanks no problem comment below some interesting everyday examples around you where machines are learning and doing amazing jobs so that's all for machine learning basics today from my site what is machine learning hello my name is richard kirschner i'm with the simply learn team that's www.simplylearn.com get certified get ahead today we're covering what is machine learning what's in it for you we're going to cover the basics of machine learning what is machine learning artificial intelligence versus machine learning versus deep learning how does machine learning work types of machine learning

machine learning pre-requisites applications of machine learning here we have our it looks a little bit like frankenstein our frankenstein looking robot today let me tell you what is machine learning machine learning works on the development of computer programs that can access data and use it to automatically learn and improve from experience watch a robot builder construct a house in two days this was back in july 29 2016 so that's pretty impressive this amount of time to continue to grow in his development and it's smart enough to leave spaces in the brickwork for wiring and plumbing and can even cut and shape bricks to size amazon echo relies on machine learning and with more data it becomes more accurate play your favorite music order pizza from domino's voice control your home request rides from uber have you ever wondered the difference between ai machine learning and deep learning artificial intelligence a technique which enables machines to mimic human behavior this is really important because this is how we are able to gauge how well our computations or what we're working on works is the fact that we're mimicking human behavior we're using this to replace human work and make it more efficient and make it more streamlined and more accurate and so the center of artificial intelligence is the big picture of all this put together ibm deep blue chess electronic game characters those are just a couple examples of artificial intelligence machine learning a technique which uses statistical methods enabling machines to learn from their past data so this means if you have your input from last time and you have your answer you use that to help prove the next guess it makes for the correct answer ibm watson google search algorithm email spam filters these are all part of machine learning and then deep learning which is a subset of machine learning composing algorithms that allow a model to train itself and perform tasks alphago natural speech recognition these are a couple examples deep learning is associated with tools like neural networks where it's kind of a black box as it learns it changes all these things that are as a human we would have a very hard time tracking and it's able to come up with an answer from that now let's see how machine learning works first we start with training the data once we've trained the data the train we go into the machine learning algorithm which then puts the data into a processing which then goes down to machine another machine learning algorithm and then we take new data because you have to test whatever you did to make sure it works correctly and we put that into the same algorithm once we do that we check our prediction we check our results and from the prediction if we've set aside some training date and we find out it didn't do a good job predicting it and it gets a thumbs down as you see then we go back to the beginning and we retrain the algorithm and a lot of times it's not just about getting the wrong answer it's about continually trying to get a better answer so you'll see the first time you might be like oh this is not the answer i want depending on what domain you're working in whether it's medical economical business stocks whatever you try out your model and if it's not giving you a good answer you retrain it if you think you can get a better answer you retrain it and you keep doing that until you get the best answer you can let's see the types of machine learning supervised learning unsupervised learning there's a number of ways to divide up machine learning and how it works these are two main categories you can divide it into supervised learning we have a known amount of data so in this case we have a bunch of apples we have a machine learning algorithm it goes through the process it goes through and trains a model based on that known data and then once you've trained your model on the known stuff you can then put an unknown data in there and you get a new response and of course in this particular one it's an apple so it's trying to figure out whether it's an apple or another fruit there are many different algorithms you can use for computing this information for doing this supervised training just to list the some of the top ones that are currently being used and by no means not there's more than just this so by no means this isn't the complete list there's polynomial regression there's a random forest there's linear regression there's logistic

regression there's decision trees there's k-nearest neighbors and there's naive bayes like i said this is just a short list of some of the many tools that are out there nowadays and if you have supervised learning then we should also look at unsupervised learning unsupervised learning so we have unknown data in this case you can see we have a bunch of fruit and we might not have labeled it we don't know we've never had anybody look at it and say this is what this is and we take that data and we put it through the machine learning algorithm and then that goes through the processing and then the trained model and with the trained model says hey can i see a pattern here and from that pattern it divides it up into a response in this case apples and pears you can see some of these things look just like the other and it tries to put them all together so that you get similar things in similar groups and again we have a nice list of algorithms here and this is not the only algorithms used for this so don't limit yourself to this just these these are just some of the primary ones used today and of course we have the k-means clustering singular value decomposition fuzzy means partial least squares a priori hierarchical clustering principle component analysis machine learning pre-requisites computer science fundamentals and programming so any of the machine learning out today you have to know some basic scripting or programming intermediate statistical knowledge you have to understand a little bit about probabilities if a is current how likely is b going to happen if there's clouds overhead how likely is it going to rain linear algebra and intermediate calculus the linear algebra is very important because you have to understand basically drawing a line through the data points and what that means that's the most fundamental linear regression models you draw a line through all your data and you use that line to compute new values intermediate calculus means you need to have a little bit of understanding of what a differential equation is you really don't need to be an expert because the computer does all the heavy lifting for you but it's important to know the terms when they come up unless you're doing some advanced programming on the actual models themselves and data wrangling and cleaning i would say this might be the biggest one in here is you have to start getting a grip on how to clean up your data there's a saying is bad data in bad data out good data in you're more likely to have good data out some applications of machine learning instance segmentation object detection instant segmentation you can see here where they use machine learning to go in there and find where the different cats are and the different objects are in the picture and then in segmentation it actually cuts them out kind of a fun one especially if you have a google pixel phone and you can do little animation objects on top of your ongoing pictures you're taking or movies number plate detection you can see here where we have a card it comes in there and it finds a number plate on the car once it's done that it can then do automatic translation automatic translation is we pick up some symbols in this case on a machine and it does machine translation so that you can know what it's saying even if you don't speak that language let's look at another example machine learning based on the amount of rainfall how much would be the crop yield so here we have our crops we have our rainfall and we want to know how much we're going to get from our crops this year so we're going to introduce two variables independent and dependent the independent variable is a variable whose value does not change by the effect of other variables and is used to manipulate the dependent variable it is often denoted as x in our example rainfall is the independent variable this is a wonderful example because you can easily see that we can't control the rain but the rain does control the crop so we talk about the independent variable controlling the dependent variable let's define dependent variable as a variable whose value change when there is any manipulation the values of the independent variables it is often denoted as y and you can see here our crop yield is dependent variable and it is dependent on the amount of rainfall received now that we've taken a look at a real life example let's go a little bit into the theory and some definitions on machine learning and see how that fits together with linear

regression numerical and categorical values let's take our data coming in and this is kind of random data from any kind of project we want to divide it up into numerical and categorical so numerical is numbers age salary height where categorical would be a description the color a dog's breed gender categorical is limited to very specific items where numerical is a range of information now that you've seen the difference between numerical and categorical data let's take a look at some different machine learning definitions we look at our different machine learning algorithms we can divide them into three areas supervised unsupervised reinforcement we're only going to look at supervised today unsupervised means we don't have the answers we're just grouping things reinforcement is where we give positive and negative feedback to our algorithm to program it and it doesn't have the information until after the fact but today we're just looking at supervised because that's where linear regression fits in in supervised data we have our data already there and our answers for a group and then we use that to program our model and come up with an answer the two most common uses for that is through the regression and classification now we're doing linear regression so we're just going to focus on the regression side and in the regression we have simple linear regression we have multiple linear regression and we have polynomial linear regression now on these three simple linear regression is the examples we've looked at so far where we have a lot of data and we draw a straight line through it multiple linear regression means we have multiple variables remember where we had the rainfall and the crops we might add additional variables in there like how much food do we give our crops when do we harvest them those would be additional information add in to our model and that's why we multiple linear regression and finally we have polynomial linear regression that is instead of drawing a line we can draw a curved line through it now that you see where regression model fits into the machine learning algorithms and we're specifically looking at linear regression let's go ahead and take a look at applications for linear regression let's look at a few applications of linear regression economic growth used to determine the economic growth of a country or a state in the coming quarter can also be used to predict the gdp of a country product price can be used to predict what would be the price of a product in the future we can guess whether it's going to go up or down or should i buy today housing sales to estimate the number of houses a builder would sell and what price in the coming months score predictions cricket fever to predict the number of runs a player would score in the coming matches based on the previous performance i'm sure you can figure out other applications you could use linear regression for so let's jump in and let's understand linear regression and dig into the theory understanding linear regression linear regression is the statistical model used to predict the relationship between independent and dependent variables by examining two factors the first important one is which variables in particular are significant predictors of the outcome variable and the second one that we need to look at closely is how significant is the regression line to make predictions with the highest possible accuracy if it's inaccurate we can't use it so it's very important we find out the most accurate line we can get since linear regression is based on drawing a line through data we're going to jump back and take a look at some euclidean geometry the simplest form of a simple linear regression equation with one dependent and one independent variable is represented by y equals m times x plus c and if you look at our model here we plotted two points on here x1 and y1 x2 and y2 y being the dependent variable remember that from before and x being the independent variable so y depends on whatever x is m in this case is the slope of the line where m equals the difference in the y 2 minus y 1 and x2 minus x1 and finally we have c which is the coefficient of the line or where it happens to cross the zero axes let's go back and look at an example we used earlier of linear regression we're going to go back to plotting the amount of crop yield based on the amount of rainfall and here we have our rainfall remember we cannot change rainfall and we have our crop yield which is

dependent on the rainfall so we have our independent and our dependent variables we're going to take this and draw a line through it as best we can through the middle of the data and then we look at that we put the red point on the y axis is the amount of crop yield you can expect for the amount of rainfall represented by the green dot so if we have an idea what the rainfall is for this year and what's going on then we can guess how good our crops are going to be and we've created a nice line right through the middle to give us a nice mathematical formula let's take a look and see what the math looks like behind this let's look at the intuition behind the regression line now before we dive into the math and the formulas that go behind this and what's going on behind the scenes i want you to note that when we get into the case study and we actually apply some python script that this math you're going to see here is already done automatically for you you don't have to have it memorized it is however good to have an idea what's going on so if people reference the different terms you'll know what they're talking about let's consider a sample data set with five rows and find out how to draw the regression line we're only going to do five rows because if we did like the rainfall with hundreds of points of data that would be very hard to see what's going on with the mathematics so we'll go ahead and create our own two sets of data and we have our independent variable x and our dependent variable y and when x was 1 we got y equals 2 when x was 2 y was 4 and so on and so on if we go ahead and plot this data on a graph we can see how it forms a nice line through the middle you can see where it's kind of grouped going upwards to the right the next thing we want to know is what the means is of each of the data coming in the x and the y the means doesn't mean anything other than the average so we add up all the numbers and divide by the total so 1 plus 2 plus 3 plus 4 plus 5 over 5 equals 3 and the same for y we get 4. if we go ahead and plot the means on the graph we'll see we get 3 comma 4 which draws a nice line down the middle a good estimate here we're going to dig deeper into the math behind the regression line now remember before i said you don't have to have all these formulas memorized or fully understand them even though we're going to go into a little more detail of how it works and if you're not a math whis and you don't know if you've never seen the sigma character before which looks a little bit like an e that's opened up that just means summation that's all that is so when you see the sigma character it just means we're adding everything in that row and for computers this is great because as a programmer you can easily iterate through each of the x y points and create all the information you need so in the top half you can see where we've broken that down into pieces and as it goes through the first two points it computes the squared value of x the squared value of y and x times y and then it takes all of x and adds them up all of y adds them up all of x squared adds them up and so on and so on and you can see we have the sum of equal to 15 the sum is equal to 20 all the way up to x times y where the sum equals 66. this all comes from our formula for calculating a straight line where y equals the slope times x plus the coefficient c so we go down below and we're going to compute more like the averages of these and we'll explain exactly what that is in just a minute and where that information comes from is called the square means error but we'll go into that in detail in a few minutes all you need to do is look at the formula and see how we've gone about computing it line by line instead of trying to have a huge set of numbers pushed into it and down here you'll see where the slope m equals and then the top part if you read through the brackets you have the number of data points times the sum of x times y which we computed one line at a time there and that's just the 66 and take all that and you subtract it from the sum of x times the sum of y and those have both been computed so you have 15 times 20. and on the bottom we have the number of lines times the sum of x squared easily computed as 86 for the sum minus i'll take all that and subtract the sum of x squared and we end up as we come across with our formula you can plug in all those numbers which is very easy to do on the computer you don't have to do the math on a piece of paper or calculator and

you'll get a slope of 0.6 and you'll get your c coefficient if you continue to follow through that formula you'll see it comes out as equal to 2.2 continuing deeper into what's going behind the scenes let's find out the predicted values of y for corresponding values of x using the linear equation where m equals 0.6 and c equals 2.2 we're going to take these values and we're going to go ahead and plot them we're going to predict them so y equals 0.6 times where x equals 1 plus 2.2 equals 2.8 so on and so on and here the blue points represent the actual y values and the brown points represent the predicted y values based on the model we created the distance between the actual and predicted values is known as residuals or errors the best fit lines should have the least sum of squares of these errors also known as e-square if we put these into a nice chart where you can see x and you can see why what we actual values were and you can see why i predict it you can easily see where we take y minus y predicted and we get an answer what is the difference between those two and if we square that y minus y prediction squared we can then sum those squared values that's where we get the 0.64 plus the 0.36 plus 1 all the way down until we have a summation equals 2.4 so the sum of squared errors for this regression line is 2.4 we check this error for each line and conclude the best fit line having the least e square value in a nice graphical representation we can see here where we keep moving this line through the data points to make sure the best fit line has the least square distance between the data points and the regression line now we only looked at the most commonly used formula for minimizing the distance there are lots of ways to minimize the distance between the line and the data points like sum of squared errors sum of absolute errors root mean square error etc what you want to take away from this is whatever formula is being used you can easily using a computer programming and iterating through the data calculate the different parts of it that way these complicated formulas you see with the different summations and absolute values are easily computed one piece at a time up until this point we've only been looking at two values x and y well in the real world it's very rare that you only have two values when you're figuring out a solution so let's move on to the next topic multiple linear regression let's take a brief look at what happens when you have multiple inputs so in multiple linear regression we have well we'll start with the simple linear regression where we had y equals m plus x plus c and we're trying to find the value of y now with multiple linear regression we have multiple variables coming in so instead of having just x we have x1 x2 x3 and instead of having just one slope each variable has its own slope attached to it as you can see here we have m1 m2 m3 and we still just have the single coefficient so when you're dealing with multiple linear regression you basically take your single linear regression and you spread it out so you have y equals m1 times x1 plus m2 times x2 so on all the way to m to the nth x to the nth and then you add your coefficient on there implementation of linear regression now we get into my favorite part let's understand how multiple linear regression works by implementing it in python if you remember before we were looking at a company and just based on its rnd trying to figure out its profit we're going to start looking at the expenditure of the company we're going to go back to that we're going to predict his profit but instead of predicting it just on the r d we're going to look at other factors like administration costs marketing costs and so on and from there we're going to see if we can figure out what the profit of that company is going to be to start our coding we're going to begin by importing some basic libraries and we're going to be looking through the data before we do any kind of linear regression we're going to take a look at the data to see what we're playing with then we'll go ahead and format the data to the format we need to be able to run it in the linear regression model and then from there we'll go ahead and solve it and just see how valid our solution is so let's start with importing the basic libraries now i'm going to be doing this in anaconda jupiter notebook a very popular ide i enjoy it it's such a visual to look at and so easy to use just any id for python will work just fine for this so break out your favorite python ide so

here we are in our jupyter notebook let me go ahead and paste our first piece of code in there and let's walk through what libraries we're importing first we're going to import numpy as np and then i want you to skip one line and look at import pandas as pd these are very common tools that you need with most of your linear regression the numpy which stands for number python is usually denoted as np and you have to almost have that for your sk learn toolbox you always import that right off the beginning pandas although you don't have to have it for your sklearn libraries it does such a wonderful job of importing data setting it up into a data frame so we can manipulate it rather easily and it has a lot of tools also in addition to that so we usually like to use the pandas when we can and i'll show you what that looks like the other three lines are for us to get a visual of this data and take a look at it so we're going to import matplotlibrary.pipelot as plt and then seaborn as sns seaborn works with the matplot library so you have to always import matplot library and then seaborn sits on top of it and we'll take a look at what that looks like you could use any of your own plotting libraries you want there's all kinds of ways to look at the data these are just very common ones and the seaborne is so easy to use it just looks beautiful it's a nice representation that you can actually take and show somebody and the final line is the amber scion map plot library inline that is only because i'm doing an inline ide my interface in the anaconda jupiter notebook requires i put that in there or you're not going to see the graph when it comes up let's go ahead and run this it's not going to be that interesting so we're just setting up variables in fact it's not going to do anything that we can see but it is importing these different libraries and setup the next step is load the data set and extract independent and dependent variables now here in the slide you'll see companies equals pd.read csv and it has a long line there with the file at the end 1000 companies dot csv you're going to have to change this to fit whatever setup you have and the file itself you can request just go down to the commentary below this video and put a note in there and simply learn we'll try to get in contact with you and supply you with that file so you can try this coding yourself so we're going to add this code in here and we're going to see that i have companies equals pd.reader underscore csv and i've changed this path to match my computer c colon slash simply learn slash 1000 underscore companies.csv and then below there we're going to set the x equals to companies under the i location and because this is companies as a pd data set i can use this nice notation that says take every row that's what the colon the first colon is comma except for the last column that's what the second part is or we have a colon minus one and we want the values set into there so x is no longer a data set a pandas data set but we can easily extract the data from our pandas data set with this notation and then y we're going to set equal to the last row well the question is going to be what are we actually looking at so let's go ahead and take a look at that and we're going to look at the companies dot head which lists the first five rows of data and i'll open up the file in just a second so you can see where that's coming from but let's look at the data in here as far as the way the panda sees it when i hit run you'll see it breaks it out into a nice setup this is what panda is one of the things pandas is really good about is it looks just like an excel spreadsheet you have your rows and remember when we're programming we always start with zero we don't start with one so it shows the first five rows zero one two three four and then it shows your different columns r d spend administration marketing spend state profit it even notes that the top are column names it was never told that but pandas is able to recognize a lot of things that they're not the same as the data rows why don't we go ahead and open this file up in a csv so you can actually see the raw data so here i've opened it up as a text editor and you can see at the top we have rnd spend comma administration comma marketing spin comma state comma profit carrier's return i don't know about you but i go crazy trying to read files like this that's why we use the pandas you could also open this up in an excel and it would separate it since it is a comma separated variable file but

we don't want to look at this one we want to look at something we can read rather easily so let's flip back and take a look at that top part the first five row now as nice as this format is where i can see the data to me it doesn't mean a whole lot maybe you're an expert in business and investments and you understand what uh 165 349.20 cents compared to the administration cost of 136 897.80 so on so on helps to create the profit of 192 261 and 83 cents that makes no sense to me whatsoever no pun intended so let's flip back here and take a look at our next set of code where we're going to graph it so we can get a better understanding of our data and what it means so at this point we're going to use a single line of code to get a lot of information so we can see where we're going with this let's go ahead and paste that into our notebook and see what we got going and so we have the visualization and again we're using sns which is pandas as you can see we imported the map plot library dot pi plot as plt which then the seaborn uses and we imported the seaborn as sns and then that final line of code helps us show this in our inline coding without this it wouldn't display and you could display it to a file and other means and that's the matplot library in line with the amber sign at the beginning so here we come down to the single line of code seaborn is great because it actually recognizes the panda data frame so i can just take the companies dot core for coordinates and i can put that right into the seaborn and when we run this we get this beautiful plot and let's just take a look at what this plot means if you look at this plot on mine the colors are probably a little bit more purplish and blue than the original one uh we have the columns and the rows we have r and d spending we have administration we have marketing spending and profit and if you cross index any two of these since we're interested in profit if you cross index profit with profit it's going to show up if you look at the scale on the right way up in the dark why because those are the same data they have an exact correspondence so r d spending is going to be the same as r d spending and the same thing with administration costs right down the middle you get this dark row or dark um diagonal row that shows that this is the highest corresponding data that's exactly the same and as it becomes lighter there's less connections between the data so we can see with profit obviously profit is the same as profit and next it has a very high correlation with r d spending which we looked at earlier and it has a slightly less connection to marketing spending and even less to how much money we put into the administration so now that we have a nice look at the data let's go ahead and dig in and create some actual useful linear regression models so that we can predict values and have a better profit now that we've taken a look at the visualization of this data we're going to move on to the next step instead of just having a pretty picture we need to generate some hard data some hard values so let's see what that looks like we're going to set up our linear regression model in two steps the first one is we need to prepare some of our data so it fits correctly and let's go ahead and paste this code into our jupiter notebook and what we're bringing in is we're going to bring in the sklearn pre-processing where we're going to import the label encoder and the one hot encoder to use the label encoder we're going to create a variable called label encoder and set it equal to capital l label capital e encoder this creates a class that we can reuse for transferring the labels back and forth now about now you should ask what labels are we talking about let's go take a look at the data we processed before and see what i'm talking about here if you remember when we did the companies dot head and we printed the top five rows of data we have our columns going across we have column zero which is r and d spending column one which is administration column two which is marketing spending and column three is state and you'll see under state we have new york california florida now to do a linear regression model it doesn't know how to process new york it knows how to process a number so the first thing we're going to do is we're going to change that new york california and florida and we're going to change those to numbers that's what this line of code does here x equals and then it has the colon comma 3 in brackets the first part the colon comma means that

we're going to look at all the different rows so we're going to keep them all together but the only row we're going to edit is the third row and in there we're going to take the label coder and we're going to fit and transform the x also the third row so we're going to take that third row we're going to set it equal to a transformation and that transformation basically tells it that instead of having a new york it has a zero or a one or a two and then finally we need to do a one hot encoder which equals one hot inc or categorical features equals three and then we take the x and we go ahead and do that equal to one hot encoder fit transform x to array this final transformation preps our data force so it's completely set the way we need it as just a row of numbers even though it's not in here let's go ahead and print x and just take a look what this data is doing you'll see you have an array of arrays and then each array is a row of numbers and if i go ahead and just do row 0 you'll see i have a nice organized row of numbers that the computer now understands we'll go ahead and take this out there because it doesn't mean a whole lot to us it's just a row of numbers next on setting up our data we have avoiding dummy variable trap this is very important why because the computer has automatically transformed our header into the setup and it's automatically transferred all these different variables so when we did the encoder the encoder created two columns and what we need to do is just have the one because it has both the variable and the name that's what this piece of code does here let's go ahead and paste this in here and we have x equals x colon comma one colon all this is doing is removing that one extra column we put in there when we did our one hot encoder and our label encoding let's go ahead and run that and now we get to create our linear regression model and let's see what that looks like here and we're going to do that in two steps the first step is going to be in splitting the data now whenever we create a predictive model of data we always want to split it up so we have a training set and we have a testing set that's very important otherwise we'd be very unethical without testing it to see how good our fit is and then we'll go ahead and create our multiple linear regression model and train it and set it up let's go ahead and paste this next piece of code in here and i'll go ahead and shrink it down a size or two so it all fits on one line so from the sklearn module selection we're going to import train test split and you'll see that we've created four completely different variables we have capital x train capital x test smaller case y train smaller case y test that is the standard way that they usually reference these when we're doing different models you usually see that a capital x and you see the train and the test and the lower case y what this is is x is our data going in that's our r d span our administration our marketing and then why which we're training is the answer that's the profit because we want to know the profit of an unknown entity that's what we're going to shoot for in this tutorial the next part train test split we take x and we take y we've already created those x has the columns with the data in it and y has a column with profit in it and then we're going to set the test size equals 0.2 that basically means twenty percent so twenty percent of the rows are going to be tested we're going to put them off to the side so since we're using a thousand lines of data that means that 200 of those lines we're going to hold off to the side to test for later and then the random state equals zero we're going to randomize which ones it picks to hold off to the side we'll go ahead and run this it's not overly exciting cause it's setting up our variables but the next step is the next step we actually create our linear regression model now that we got to the linear regression model we get that next piece of the puzzle let's go ahead and put that code in there and walk through it so here we go we're going to paste it in there and let's go ahead and since this is a shorter line of code let's zoom up there so we can get a good look and we have from the sklearn dot linear underscore model we're going to import linear regression now i don't know if you recall from earlier when we were doing all the math let's go ahead and flip back there and take a look at that do you remember this where we had this long formula on the bottom and we were doing all this summarization and then we also looked at setting it up with the different lines

and then we also looked all the way down to multiple linear regression where we're adding all those formulas together all of that is wrapped up in this one section so what's going on here is i'm going to create a variable called regressor and the regressor equals the linear regression that's a linear regression model that has all that math built in so we don't have to have it all memorized or have to compute it individually and then we do the regressor dot fit in this case we do x train and y train because we're using the training data x being the data in and y being profit what we're looking at and this does all that math for us so within one click and one line we've created the whole linear regression model and we fit the data to the linear regression model and you can see that when i run the regressor it gives an output linear regression it says copy x equals true fit intercept equals true in jobs equal one normalize equals false it's just giving you some general information on what's going on with that regressor model now that we've created our linear regression model let's go ahead and use it and if you remember we kept a bunch of data aside so we're going to do a y predict variable and we're going to put in the x test and let's see what that looks like scroll up a little bit paste that in here predicting the test set results so here we have y predict equals regressor dot predict x test going in and this gives us y predict now because i'm in jupiter inline i can just put the variable up there and when i hit the run button it'll print that array out i could have just as easily done print y predict so if you're in a different ide that's not an inline setup like the jupiter notebook you can do it this way print y predict and you'll see that for the 200 different test variables we kept off to the side it's going to produce 200 answers this is what it says the profit are for those 200 predictions but let's don't stop there let's keep going and take a couple look we're going to take just a short detail here and calculating the coefficients and the intercepts this gives us a quick flash at what's going on behind the line we're going to take a short detour here and we're going to be calculating the coefficient and intercepts so you can see what those look like what's really nice about our regressor we created is it already has a coefficients for us we can simply just print regressor dot coefficient underscore when i run this you'll see our coefficients here and if we can do the regressor coefficient we can also do the regressor intercept and let's run that and take a look at that this all came from the multiple regression model and we'll flip over so you can remember where this is going into where it's coming from you can see the formula down here where y equals m1 times x1 plus m2 times x2 and so on and so on plus c the coefficient so these variables fit right into this formula y equals slope 1 times column 1 variable plus slope 2 times column 2 variable all the way to the m to the n and x to the n plus c the coefficient or in this case you have minus 8.89 to the power of 2 etc etc times the first column and the second column and the third column and then our intercept is the minus 1 0 3 0 0 9 point boy it gets kind of complicated when you look at it this is why we don't do this by hand anymore this is why we have the computer to make these calculations easy to understand and calculate now i told you that was a short detour and we're coming towards the end of our script as you remember from the beginning i said if we're going to divide this information we have to make sure it's a valid model that this model works and understand how good it works so calculating the r squared value that's what we're going to use to predict how good our prediction is and let's take a look what that looks like in code and so we're going to use this from sklearn.metrics we're going to import r2 score that's the r squared value we're looking at the error so in the r2 score we take our y test versus our y predict y test is the actual values we're testing that was the one that was given to us that we know are true the y predict of those 200 values is what we think it was true and when we go ahead and run this we see we get a 0.9352 that's the r2 score now it's not exactly a straight percentage so it's not saying it's 93 correct but you do want that in the upper 90s oh and higher shows that this is a very valid prediction based on the r2 score and if r squared value of 0.91 or 9 2 as we got on our model remember it does have a random

generation involved this proves the model is a good model which means success yay we successfully trained our model with certain predictors and estimated the profit of the companies using linear regression what is logistic regression as i mentioned earlier logistic regression is an algorithm for performing binary classification so let's take an example and see how this works let's say your car has not been serviced for quite a few years and now you want to find out if it is going to break down in the near future so this is like a classification problem find out whether your car will break down or not so how are we going to perform this classification so here's how it looks if we plot the information along the x and y axis x is the number of years since the last service was performed and why is the probability of your car breaking down and let's say this information was this data rather was collected from several car users it's not just your car but several car users so that is our labeled data so the data has been collected and for for the number of years and when the car broke down and what was the probability and that has been plotted along x and y axis so this provides an idea or from this graph we can find out whether your car will break down or not we'll see how so first of all the probability can go from zero to one as you all aware probability can be between zero and one and as we can imagine it is intuitive as well as the number of years are on the lower side maybe one year two years or three years till after the service the chances of your car breaking down are very limited right so for example chances of your car breaking down the probability of your car breaking down within two years of your last service are 0.1 probability similarly 3 years is maybe 0.3 and so on but as the number of years increases let's say if it was 6 or 7 years there is almost a certainty that your car is going to break down that is what this graph shows so this is an example of a application of the classification algorithm and we will see in little details how exactly logistic regression is applied here one more thing needs to be added here is that the dependent variables outcome is discrete so if we are talking about whether the car is going to break down or not so that is a discrete value the y that we are talking about the dependent variable that we are talking about what we are looking at is whether the car is going to break down or not yes or no that is what we are talking about so here the outcome is discrete and not a continuous value so this is how the logistic regression curve looks let me explain a little bit what exactly how exactly we are going to determine the class at the outcome rather so for a logistic regression curve a threshold has to be set saying that because this is a probability calculation remember this is a probability calculation and the probability itself will not be 0 or 1 but based on the probability we need to decide what the outcome should be so there has to be a threshold like for example 0.5 can be the threshold let's say in this case so any value of the probability below 0.5 is considered to be 0 and any value above 0.5 is considered to be 1. so an output of let's say 0.8 will mean that the car will break down so that is considered as an output of 1 and let's say an output of 0.29 is considered as zero which means that the car will not break down so that's the way logistic regression works now let's do a quick comparison between logistic regression and linear regression because they both have the term regression in them so it can cause confusion so let's try to remove that confusion so what is linear regression linear regression is a process is once again an algorithm for supervised learning however here you're going to find a continuous value you're going to determine a continuous value it could be the price of a real estate property it could be your hike how much hike you're going to get or it could be a stock price these are all continuous values these are not discrete compared to a yes or no kind of a response that we are looking for in logistic regression so this is one example of a linear regression let's say the hr team of a company tries to find out what should be the salary hike of an employee so they collect all the details of their existing employees their ratings and their salary hikes what has been given and that is the labeled information that is available and the system learns from us it is trained and it learns from this labeled information so that when a new employees information

is fed based on the rating it will determine what should be the high so this is a linear regression problem and a linear regression example now salary is a continuous value you can get five thousand five thousand five hundred five thousand six hundred it is not discrete like a cat or a dog or an apple or a banana these are discrete or a yes or no these are discrete values right so this way you are trying to find continuous values is where we use linear regression so let's say just to extend on the scenario we now want to find out whether this employee is going to get a promotion or not so we want to find out that is a discrete problem right a yes or no kind of a problem in this case we actually cannot use linear regression even though we may have labeled data so this is the label data so based on the employee rating these are the ratings and then some people got the promotion and this is the ratings for which people did not get promotion that is a no and this is the rating for which people got promotion we just plotted the data about whether a person has got an employer's got promotion or not yes no right so there is nothing in between and what is the employees rating okay and ratings can be continuous that is not an issue but the output is discrete in this case whether employee got promotion yes no okay so if we try to plot that and we try to find a straight line this is how it would look and as you can see it doesn't look very right because looks like there will be lot of errors thus root mean square error if you remember for linear regression would be very very high and also the the values cannot go beyond zero or beyond one so the graph should probably look somewhat like this clipped at zero and one but still the straight line doesn't look right therefore instead of using a linear equation we need to come up with something different and therefore the logistic regression model looks somewhat like this so we calculate the probability and if we plot that probability not in the form of a straight line but we need to use some other equation we will see very soon what that equation is then it is a gradual process right so you see here people with some of these ratings are not getting any promotions and then slowly at certain rating they get promotion so that is a gradual process and this is how the math behind logistic regression looks so we are trying to find the odds for a particular event happening and this is the formula for finding the odds so the probability of an event happening divided by the probability of the event not happening so p if it is the probability of the event happening probability of the person getting a promotion and divided by the probability of the person not getting a promotion that is 1 minus p so this is how you measure the odds now the values of the odds range from 0 to infinity so when this probability is 0 then the odds will the value of the odds is equal to 0 and when the probability becomes 1 then the value of the odds is 1 by 0 that will be infinity but the probability itself remains between 0 and 1. now this is how an equation of a straight line looks so y is equal to beta0 plus beta1 x where beta0 is the y-intercept and beta1 is the slope of the line if we take the odds equation and take a log of both sides then this would look somewhat like this and the term logistic is actually derived from the fact that we are doing this we take a log of dx by 1 minus px this is an extension of the calculation of odds that we have seen right and that is equal to beta0 plus beta1 x which is the equation of the straight line and now from here if you want to find out the value of p x we will see we can take the exponential on both sides and then if we solve that equation we will get the equation of p x like this p x is equal to 1 by 1 plus e to the power of minus beta 0 plus beta 1 x and recall this is nothing but the equation of the line which is equal to y y is equal to beta 0 plus beta 1 x so that this is the equation also known as the sigmoid function and this is the equation of the logistic regression all right and if this is plotted this is how the sigmoid curve is obtained so let's compare linear and logistic regression how they are different from each other let's go back so linear regression is solved or used to solve regression problems and logistic regression is used to solve classification problems so both are called regression but linear regression is used for solving regression problems where we predict continuous values whereas logistic regression is used for solving classification problems where

we have had to predict discrete values the response variables in case of linear regression are continuous in nature whereas here they are categorical or discrete in nature and linear regression helps to estimate the dependent variable when there is a change in the independent variable whereas here in case of logistic regression it helps to calculate the probability or the possibility of a particular event happening and linear regression as the name suggests is a straight line that's why it's called linear regression whereas logistic regression is a sigmoid function and the curve is the shape of the curve is s it's an s shaped curve this is another example of application of logistic regression in weather prediction whether it's going to rain or not rain now keep in mind both are used in weather prediction if we want to find the discrete values like whether it's going to rain or not rain that is a classification problem we use logistic regression but if you want to determine what is going to be the temperature tomorrow then we use linear regression so just keep in mind that in weather prediction we actually use both but these are some examples of logistic regression so we want to find out whether it's going to be rain or not is going to be sunny or not there is going to snow or not these are all logistic regression examples a few more examples classification of objects this is a again another example of logistic regression now here of course one distinction is that these are multi-class classification so logistic regression is not used in its original form but it is used in a slightly different form so we say whether it is a dog or not a dog i hope you understand so instead of saying is it a dog or a cat or an elephant we convert this into saying so because we need to keep it to binary classification so we say is it a dog or not a dog is it a cat or not a cat so that's the way logistic regression can be used for classifying objects otherwise there are other techniques which can be used for performing multi-class classification in healthcare logistic regression is used to find the survival rate of a patient so they take multiple parameters like trauma score and age and so on and so forth and they try to predict the rate of survival all right now finally let's take an example and see how we can apply logistic regression to predict the number that is shown in the image so this is actually a live demo i will take you into jupiter notebook and show the code but before that let me take you through a couple of slides to explain what we are trying to do so let's say you have an eight by eight image and there the image has a number one two three four and you need to train your model to predict what this number is so how do we do this so the first thing is obviously in any machine learning process you train your model so in this case we are using logistic regression so and then we provide a training set to train the model and then we test how accurate our model is with the test data which means that like any machine learning process we split our initial data into two parts training set and test set with the training set we train our model and then with the test set we test the model then we get good accuracy and then we use it for for inference right so that is typical methodology of uh training testing and then deploying of machine learning models so let's uh take a look at the code and see what we are doing so i will not go line by line but just take you through some of the blocks so first thing we do is import all the libraries and then we basically take a look at the images and see what is the total number of images we can display using matplotlib some of the images are a sample of these images and then we split the data into training and test as i mentioned earlier and we can do some exploratory analysis and then we build our model we train our model with the training set and then we test it with our test set and find out how accurate our model is using the confusion matrix the heat map and use heat map for visualizing this and i will show you in the code what exactly is the confusion matrix and how it can be used for finding the accuracy in our example we got we get an accuracy of about 0.94 which is pretty good or 94 which is pretty good all right so what is the confusion matrix this is an example of a confusion matrix and this is used for identifying the accuracy of a classification model or like a logistic regression model so the most important part in a confusion

matrix is that first of all this as you can see this is a matrix and the size of the matrix depends on how many outputs we are expecting right so the most important part here is that the model will be most accurate when we have the maximum numbers in its diagonal like in this case that's why it has almost 93 94 percent because the diagonals should have the maximum numbers and the others other than diagonals the cells other than the diagonal should have very few numbers so here that's what is happening so there is a 2 here there are there's a 1 here but most of them are along the diagonal this what does this mean this means that the number that has been fed is zero and the number that has been detected is also zero so the predicted value and the actual value are the same so along the diagonals that is true which means that let's let's take this diagonal right if the maximum number is here that means that like here in this case it is 34 which means that 34 of the images that have been fed or rather actually there are two misclassifications in there so 36 images have been fed which have number four and out of which 34 have been predicted correctly as number four and one has been predicted as number eight and another one has been predicted as number nine so these are two misclassification okay so that is the meaning of saying that the maximum number should be in the diagonal so if you have all of them so for an ideal model which has let's say 100 accuracy everything will be only in the diagonal there will be no numbers other than 0 in all other cells so that is like 100 percent accurate model okay so that's uh just of how to use this matrix how to use this uh confusion matrix i know the name is a little funny sounding confusion matrix but actually it is not very confusing it's very straightforward so you are just plotting what has been predicted and what is the labeled information or what is the actual data that's also known as the ground truth sometimes okay these are some fancy terms that are used so predicted label and the actual name that's all it is okay yeah so we are showing a little bit more information here so 38 have been predicted and here you will see that all of them have been predicted correctly there have been 38 zeros and the predicted value and the actual value is is exactly the same whereas in this case right it has there are i think 37 plus five yeah 42 have been fed the images 42 images are of digit 3 and the accuracy is only 37 of them have been accurately predicted three of them have been predicted as number seven and two of them have been predicted as number eight and so on and so forth okay all right so with that let's go into jupiter notebook and see how the code looks so this is the code in in jupiter notebook for logistic regression in this particular demo what we are going to do is train our model to recognize digits which are the images which have digits from let's say 0 to 5 or 0 to 9 and and then we will see how well it is trained and whether it is able to predict these numbers correctly or not so let's get started so the first part is as usual we are importing some libraries that are required and then the last line in this block is to load the digits so let's go ahead and run this code then here we will visualize the shape of these digits so we can see here if we take a look this is how the shape is 1797 by 64. these are like 8x8 images so that's that's what is reflected in this shape now from here onwards we are basically once again importing some of the libraries that are required like numpy and matplot and we will take a look at uh some of the sample images that we have uh loaded so this one for example creates a figure and then we go ahead and take a few sample images to see how they look so let me run this code and so that it becomes easy to understand so these are about five images sample images that we are looking at zero one two three four so this is how the image is this is how the data is okay and uh based on this we will actually train our logistic regression model and then we will test it and see how well it is able to recognize so the way it works is the pixel information so as you can see here this is an 8 by 8 pixel kind of a image and the each pixel whether it is activated or not activated that is the information available for each pixel now based on the pattern of this activation and non-activation of the various pixels this will be identified as a zero for example right similarly as you can see so overall each of these numbers actually has a

different pattern of the pixel activation and that's pretty much that our model needs to learn for which a number what is the pattern of the activation of the pixels right so that is what we are going to train our model okay so the first thing we need to do is to split our data into training and test data set right so whenever we perform any training we split the data into training and test so that the training data set is used to train the system so we pass this probably multiple times and then we test it with the test data set and the split is usually in the form of there and there are various ways in which you can split this data it is up to the individual preferences in our case here we are splitting in the form of 23 and 77 so when we say test size as 20 0.23 that means 23 percent of that entire data is used for testing and the remaining 77 percent is used for training so there is a readily available function which is uh called train test split so we don't have to write any special code for the splitting it will automatically split the data based on the proportion that we give here which is test size so we just give the test size automatically training size will be determined and we pass the data that we want to split and the the results will be stored in x underscore train and y underscore train for the training data set and what is x underscore train these are these are the features right which is like the independent variable and why underscore train is the label right so in this case what happens is we have the input value which is or the features value which is in x underscore train and since this is the labeled data for each of them each of the observations we already have the label information saying whether this digit is a zero or a one or a two so that this this is what will be used for comparison to find out whether the the system is able to recognize it correctly or there is an error for each observation it will compare with this right so this is the label so the same way x underscore train y underscore train is for the training data set x underscore test y underscore test is for the test data set okay so let me go ahead and execute this code as well and then we can go and check quickly what is the how many entries are there and in each of these so x underscore train the shape is 13 83 by 64. and y underscore train has 1383 because there is nothing like the second part is not required here and then x underscore test shape we see is 414 so actually there are 414 observations in test and 1383 observations in train so that's basically what these four lines of code are are saying okay then we import the logistic regression library and which is a part of scikit-learn so we we don't have to implement the logistic regression process itself we just call these the function and let me go ahead and execute that so that we have the logistic regression library imported now we create an instance of logistic regression right so logistic regr is a is an instance of logistic regression and then we use that for training our model so let me first execute this code so these two lines so the first line basically creates an instance of logistic regression model and then the second line is where we are passing our data the training data set and this is our the the predictors and uh this is our target we are passing this data set to train our model all right so once we do this in this case the data is not large but by and large the training is what takes usually a lot of time so we spend in machine learning activities in machine learning projects we spend a lot of time for the training part of it okay so here the data set is relatively small so it was pretty quick so all right so now our model has been trained using the training data set and we want to see how accurate this is so what we'll do is we will test it out in probably faces so let me first try out how well this is working for one image okay i will just try it out with one image my the first entry in my test data set and see whether it is uh correctly predicting or not so and in order to test it so for training purpose we use the fit method there is a method called fit which is for training the model and once the training is done if you want to test for a particular value new input you use the predict method okay so let's run the predict method and we pass this particular image and we see that the shape is or the prediction is four so let's try a few more let me see for the next 10 seems to be fine so let me just go ahead and test the entire data set okay that's basically what we will do so now we want to find out how accurately this has

performed so we use the score method to find what is the percentages of accuracy and we see here that it has performed up to 94 percent accurate okay so that's on this part now what we can also do is we can also see this accuracy using what is known as a confusion matrix so let us go ahead and try that as well so that we can also visualize how well this model has done so let me execute this piece of code which will basically import some of the libraries that are required and we we basically create a confusion matrix and instance of confusion matrix by running confusion matrix and passing these values so we have so this confusion underscore matrix method takes two parameters one is the y underscore test and the other is the prediction so what is the y underscore test these are the labeled values which we already know for the test data set and predictions are what the system has predicted for the test data set okay so this is known to us and this is what the system has the model has generated so we kind of create the confusion matrix and we will print it and this is how the confusion matrix looks as the name suggests it is a matrix and the key point out here is that the accuracy of the model is determined by how many numbers are there in the diagonal the more the numbers in the diagonal the better the accuracy is okay and first of all the total sum of all the numbers in this whole matrix is equal to the number of observations in the test data set that is the first thing right so if you add up all these numbers that will be equal to the number of observations in the test data set and then out of that the maximum number of them should be in the diagonal that means the accuracy is pretty good if the the numbers in the diagonal are less and in all other places there are a lot of numbers which means the accuracy is very low the diagonal indicates a correct prediction that this means that the actual value is same as the predicted value here again actual values same as the predictive value and so on right so the moment you see a number here that means the actual value is something and the predicted value is something else right similarly here the actual value is something and the predicted value is something else so that is basically how we read the confusion matrix now how do we find the accuracy you can actually add up the total values in the diagonal so it's like 38 plus 44 plus 43 and so on and divide that by the total number of test observations that will give you the percentage accuracy using a confusion matrix now let us visualize this confusion matrix in a slightly more sophisticated way using a heat map so we will create a heat map with some we'll add some colors as well it's uh it's like more visually visually more appealing so that's the whole idea so if we let me run this piece of code and this is how the heat map looks and as you can see here the diagonals again are all the values are here most of the values so which means reasonably this seems to be reasonably accurate and yeah basically the accuracy score is 94 percent this is calculated as i mentioned by adding all these numbers divided by the total test values or the total number of observations in test data set okay so this is the confusion matrix for logistic regression all right so now that we have seen the confusion matrix let's take a quick sample and see how well the system has classified and we will take a few examples of the data so if we see here we we picked up randomly a few of them so this is uh number four which is the actual value and also the predicted value both are four this is an image of zero so the predicted value is also zero actual value is of course zero then this is the image of nine so this has also been predicted correctly nine and actual value is nine and this is the image of 1 and again this has been predicted correctly as like the actual value okay so this was a quick demo of logistic regression how to use logistic regression to identify images what is a decision tree let's go through a very simple example before we dig in deep decision tree is a tree shaped diagram used to determine a course of action each branch of the tree represents a possible decision or current or reaction let's start with a simple question how to identify a random vegetable from a shopping bag so we have this group of vegetables in here and we can start off by asking a simple question is it red and if it's not then it's going to be the purple fruit to the left probably an eggplant if it's true it's

going to be one of the red fruits is the diameter greater than 2 if false is going to be a what looks to be a red chili and if it's true it's going to be a bell pepper from the capsicum family so it's a capsicum problems that decision tree can solve so let's look at the two different categories the decision tree can be used on it can be used on the classification the true false yes no and it can be used on regression where we figure out what the next value is in a series of numbers or a group of data in classification the classification tree will determine a set of logical if-then conditions to classify problems for example discriminating between three types of flowers based on certain features in regression a regression tree is used when the target variable is numerical or continuous in nature we fit the regression model to the target variable using each of the independent variables each split is made based on the sum of squared error before we dig deeper into the mechanics of the decision tree let's take a look at the advantages of using a decision tree and we'll also take a glimpse at the disadvantages the first thing you'll notice is that it's simple to understand interpret and visualize it really shines here because you can see exactly what's going on in a decision tree little effort is required for data preparation so you don't have to do special scaling there's a lot of things you don't have to worry about when using a decision tree it can handle both numerical and categorical data as we discovered earlier and non-linear parameters don't affect its performance so even if the data doesn't fit an easy curved graph you can still use it to create an effective decision or prediction if we're going to look at the advantages of a decision tree we also need to understand the disadvantages of a decision tree the first disadvantage is overfitting overfitting occurs when the algorithm captures noise in the data that means you're solving for one specific instance instead of a general solution for all the data high variance the model can get unstable due to small variation in data low bias tree a highly complicated decision tree tends to have a low bias which makes it difficult for the model to work with new data decision tree important terms before we dive in further we need to look at some basic terms we need to have some definitions to go with our decision tree in the different parts we're going to be using we'll start with entropy entropy is a measure of randomness or unpredictability in the data set for example we have a group of animals in this picture there's four different kinds of animals and this data set is considered to have a high entropy you really can't pick out what kind of animal it is based on looking at just the four animals as a big clump of uh entities so as we start splitting it into subgroups we come up with our second definition which is information gain information gain it is a measure of decrease in entropy after the data set is split so in this case based on the color yellow we've split one group of animals on one side as true and those who aren't yellow as false as we continue down the yellow side we split based on the height true or false equals ten and on the other side height is less than ten true or false and as you see as we split it the entropy continues to be less and less and less and so our information gain is simply the entropy e1 from the top and how it's changed to e2 in the bottom and we'll look at the deeper math although you really don't need to know a huge amount of math when you actually do the programming in python because they'll do it for you but we'll look on the actual math of how they compute entropy finally we went into the different parts of our tree and they call the leaf node leaf node carries the classification or the decision so it's the final end at the bottom the decision node has two or more branches this is where we're breaking the group up into different parts and finally you have the root node the topmost decision node is known as the root node how does a decision tree work wonder what kind of animals all get in the jungle today maybe you're the hunter with the gun or if you're more into photography you're a photographer with a camera so let's look at this group of animals and let's try to classify different types of animals based on their features using a decision tree so the problem statement is to classify the different types of animals based on their features using a decision tree the data set is looking quite messy and the entropy is high in this case so let's look at a

training set or a training data set and we're looking at color we're looking at height and then we have our different animals we have our elephants our giraffes our monkeys and our tigers and they're of different colors and shapes let's see what that looks like and how do we split the data we have to frame the conditions that split the data in such a way that the information gain is the highest note gain is the measure of decrease in entropy after splitting so the formula for entropy is the sum that's what this symbol looks like that looks like kind of like a e funky e of k where i equals 1 to k k would represent the number of animals the different animals in there where value or p value of i would be the percentage of that animal times the log base two of the same the percentage of that animal let's try to calculate the entropy for the current data set and take a look at what that looks like and don't be afraid of the math you don't really have to memorize this math just be aware that it's there and this is what's going on in the background and so we have three giraffes two tigers one monkey two elephants a total of eight animals gathered and if we plug that into the formula we get an entropy that equals three over eight so we have three giraffes a total of eight times the log usually they use base two on the log so log base two of three over eight plus in this case it's yellow fence two over eight two elephants over a total of eight times log base two two over eight plus one monkey over total of eight log base two one over eight and plus two over eight of the tigers log base two over eight and if we plug that into our computer our calculator i obviously can't do logs in my head we get an entropy equal to 0.571 the program will actually calculate the entropy of the data set similarly after every split to calculate the gain now we're not going to go through each set one at a time to see what those numbers are we just want you to be aware that this is a formula or the mathematics behind it gain can be calculated by finding the difference of the subsequent entropy values after a split now we'll try to choose a condition that gives us the highest gain we will do that by splitting the data using each condition and checking that the gain we get out of them the condition that gives us the highest gain will be used to make the first split can you guess what that first split will be just by looking at this image as a human is probably pretty easy to split it let's see if you're right if you guessed the color yellow you're correct let's say the condition that gives us the maximum gain is yellow so we will split the data based on the color yellow if it's true that group of animals goes to the left if it's false it goes to the right the entropy after the splitting has to decreased considerably however we still need some splitting of both the branches to attain an entropy value equal to zero so we decide to split both the nodes using height as the condition since every branch now contains single label type we can say that entropy in this case has reached the least value and here you see we have the giraffes the tigers the monkey and the elephants all separated into their own groups this tree can now predict all the classes of animals present in the dataset with a hundred percent accuracy that was easy use case loan repayment prediction let's get into my favorite part and open up some python and see what the programming code in the scripting looks like in here we're going to want to do a prediction and we start with this individual here who's requesting to find out how good his customers are going to be whether they're going to repay their loan or not for his bank and from that we want to generate a problem statement to predict if a customer will repay loan amount or not and then we're going to be using the decision tree algorithm in python let's see what that looks like and let's dive into the code in our first few steps of implementation we're going to start by importing the necessary packages that we need from python and we're going to load up our data and take a look at what the data looks like so the first thing i need is i need something to edit my python and run it in so let's flip on over and here i'm using the anaconda jupiter notebook now you can use any python ide you like to run it in but i find the jupiter notebook's really nice for doing things on the fly and let's go ahead and just paste that code in the beginning and before we start let's talk a little bit about what we're bringing in and then we're going to do a couple things in here we have to make a

couple changes as we go through this first part of the import the first thing we bring in is numpy as np that's very standard when we're dealing with mathematics especially with very complicated machine learning tools you almost always see the numpy come in for your num your number it's called number python it has your mathematics in there in this case we actually could take it out but generally you'll need it for most of your different things you work with and then we're going to use pandas as pd that's also a standard the pandas is a data frame setup and you can liken this to taking your basic data and storing it in a way that looks like an excel spreadsheet so as we come back to this when you see np or pd those are very standard uses you'll know that that's the pandas and i'll show you a little bit more we explore the data in just a minute then we're going to need to split the data so i'm going to bring in our train test and split and this is coming from the sk learn package cross validation in just a minute we're going to change that and we'll go over that too and then there's also the sk.tree import decision tree classifier that's the actual tool we're using remember i told you don't be afraid of the mathematics it's going to be done for you well the decision tree classifier has all that mathematics in there for you so you don't have to figure it back out again and then we have sklearn.metrics for accuracy score we need to score our our setup that's the whole reason we're splitting it between the training and testing data and finally we still need the sklearn import tree and that's just the basic tree function is needed for the decision tree classifier and finally we're going to load our data down here and i'm going to run this and we're going to get two things on here one we're going to get an error and two we're going to get a warning let's see what that looks like so the first thing we had is we have an error why is this error here well it's looking at this it says i need to read a file and when this was written the person who wrote it this is their path where they stored the file so let's go ahead and fix that and i'm going to put in here my file path i'm just going to call it full file name and you'll see it's on my c drive and this is very lengthy setup on here where i stored the data2.csv file don't worry too much about the full path because on your computer it'll be different the data.2 csv file was generated by simplylearn if you want a copy of that you can comment down below and request it here in the youtube and then if i'm going to give it a name full file name i'm going to go ahead and change it here to full file name so let's go ahead and run it now and see what happens and we get a warning when you're coding understanding these different warnings and these different errors that come up is probably the hardest lesson to learn so let's just go ahead and take a look at this and use this as a opportunity to understand what's going on here if you read the warning it says the cross validation is depreciated so it's a warning on it's being removed and it's going to be moved in favor of the model selection so if we go up here we have sklearn.cross validation and if you research this and go to the sklearn site you'll find out that you can actually just swap it right in there with model selection and so when i come in here and i run it again that removes a warning what they've done is they've had two different developers develop it in two different branches and then they decided to keep one of those and eventually get rid of the other one that's all that is and very easy and quick to fix before we go any further i went ahead and opened up the data from this file remember the the data file we just loaded on here the data underscore2.csv let's talk a little bit more about that and see what that looks like both as a text file because it's a comma separated variable file and in a spreadsheet this is what it looks like as a basic text file you can see at the top they've created a header and it's got one two three four five columns and each column has data in it and let me flip this over because we're also going to look at this in an actual spreadsheet so you can see what that looks like and here i've opened it up in the open office calc which is pretty much the same as excel and zoomed in and you can see we've got our columns and our rows of data a little easier to read in here we have a result yes yes no we have initial payment last payment credit score house number if we scroll way down we'll see

that this occupies a thousand and one lines of code or lines of data with the first one being a column and then 1 000 lines of data now as a programmer if you're looking at a small amount of data i usually start by pulling it up in different sources so i can see what i'm working with but in larger data you won't have that option it'll just be too too large so you need to either bring in a small amount that you can look at it like we're doing right now or we can start looking at it through the python code so let's go ahead and move on and take the next couple steps to explore the data using python let's go ahead and see what it looks like in python to print the length and the shape of the data so let's start by printing the length of the database we can use a simple lin function from python and when i run this you'll see that it's a thousand long and that's what we expected there's a thousand lines of data in there if you subtract the column head this is one of the nice things when we did the balance data from the panda read csv you'll see that the header is row zero so it automatically removes a row and then shows the data separate it does a good job sorting that data out for us and then we can use a different function and let's take a look at that and again we're going to utilize the tools in panda and since the balance underscored data was loaded as a panda data frame we can do a shape on it and let's go ahead and run the shape and see what that looks like what's nice about this shape is not only does it give me the length of the data we have a thousand lines it also tells me there's five columns so we were looking at the data we had five columns of data and then let's take one more step to explore the data using python and now that we've taken a look at the length and the shape let's go ahead and use the pandas module for head another beautiful thing in the data set that we can utilize so let's put that on our sheet here and we have print data set and balance data dot head and this is a pandas print statement of its own so it has its own print feature in there and then we went ahead and gave a label for our print job here of dataset just a simple print statement and we run that let's just take a closer look at that let me zoom in here there we go pandas does such a wonderful job of making this a very clean readable data set so you can look at the data you can look at the column headers you can have it when you put it as the head it prints the first five lines of the data and we always start with zero so we have five lines we have zero one two three four instead of one two three four five that's a standard scripting and programming set as you wanna start with the zero position and that is what the data head does it pulls the first five rows of data puts in a nice format that you can look at and view very powerful tool to view the data so instead of having to flip and open up an excel spreadsheet or open office cal or trying to look at a word doc where it's all scrunched together and hard to read you can now get a nice open view of what you're working with we're working with a shape of a thousand long five wide so we have five columns and we do the full data head you can actually see what this data looks like the initial payment last payment credit scores house number so let's take this now that we've explored the data and let's start digging into the decision tree so in our next step we're going to train and build our data tree and to do that we need to first separate the data out we're going to separate into two groups so that we have something to actually train the data with and then we have some data on the side to test it to see how good our model is remember with any of the machine learning you always want to have some kind of test set to weigh it against so you know how good your model is when you distribute it let's go ahead and break this code down and look at it in pieces so first we have our x and y where do x and y come from well x is going to be our data and y is going to be the answer or the target you can look at source and target in this case we're using x and y to denote the data in and the data that we're actually trying to guess what the answer is going to be and so to separate it we can simply put in x equals the balance of the data.values the first brackets means that we're going to select all the lines in the database so it's all the data and the second one says we're only going to look at columns one through five remember we always start with zero zero is a yes or no and that's

whether the loan went default or not so we want to start with one if we go back up here that's the initial payment and it goes all the way through the house number well if we want to look at one through five we can do the same thing for y which is the answers and we're going to set that just equal to the zero row so it's just the zero row and then it's all rows going in there so now we've divided this into two different data sets one of them with the data going in and one with the answers next we need to split the data and here you'll see that we have it split into four different parts the first one is your x training your x test your y train your y test simply put we have x going in where we're going to train it and we have to know the answer to train it with and then we have x test where we're going to test that data and we have to know in the end what the y was supposed to be and that's where this train test split comes in that we loaded earlier in the modules this does it all for us and you can see they set the test size equal to 0.3 so it's roughly 30 percent will be used in the test and then we use a random state so it's completely random which rows it takes out of there and then finally we get to actually build our decision tree and they've called it here clf underscore entropy that's the actual decision tree or decision tree classifier and in here they've added a couple variables which we'll explore in just a minute and then finally we need to fit the data to that so we take our clf entropy that we created and we fit the x train and since we know the answers for x x-trade or the y-train we go ahead and put those in and let's go ahead and run this and what most of these sklearn modules do is when you set up the variable in this case we set the clf entropy called decision tree classifier it automatically prints out what's in that decision tree there's a lot of variables you can play with in here and it's quite beyond the scope of this tutorial to go through all of these and how they work but we're working on entropy that's one of the options we've added that it's completely a random state of 100 so 100 percent and we have a max depth of three now the max depth if you remember above when we were doing the different graphs of animals means it's only going to go down three layers before it stops and then we have minimal samples of leaves as five so it's going to have at least five leaves at the end so i'll have at least three splits we'll have no more than three layers and at least five end leaves with the final result at the bottom now that we've created our decision tree classifier not only created it but trained it let's go ahead and apply it and see what that looks like so let's go ahead and make a prediction and see what that looks like we're going to paste our predict code in here and before we run it let's just take a quick look at what's it's doing here we have a variable y predict that we're going to do and we're going to use our variable clf entropy that we created and then you'll see dot predict and it's very common in the sk learn modules that their different tools have the predict when you're actually running a prediction in this case we're going to put our x test data in here now if you delivered this for use and actual commercial use and distributed it this would be the new loans you're putting in here to guess whether the person is going to be pay them back or not in this case though we need to test out the data and just see how good our sample is how good of our tree does at predicting the loan payments and finally since anaconda jupiter notebook is it works as a command line for python we can simply put the y predict e n to print it i could just as easily put the print and put brackets around y predict en to print it out we'll go ahead and do that it doesn't matter which way you do it and you'll see right here that runs a prediction this is roughly 300 in here remember it's 30 percent of a thousand so you should have about 300 answers in here and this tells you which each one of those lines of our test went in there and this is what our y predict came out so let's move on to the next step we're going to take this data and try to figure out just how good a model we have so here we go since sklearn does all the heavy lifting for you and all the math we have a simple line of code to let us know what the accuracy is and let's go ahead and go through that and see what that means and what that looks like let's go ahead and paste this in and let me zoom in a little bit there we go so you have a nice full picture and we'll see here we're

just going to do a print accuracy is and then we do the accuracy score and this was something we imported earlier if you remember at the very beginning let me just scroll up there real quick so you can see where that's coming from that's coming from here down here from sklearn.metrics import accuracy score and you could probably run a script make your own script to do this very easily how accurate is it how many out of 300 do we get right and so we put in our y test that's the one we ran the predict on and then we put in our y predict e n that's the answers we got and we're just going to multiply that by a hundred because this is just going to give us an answer as a decimal and we want to see it as a percentage and let's run that and see what it looks like and if you see here we got an accuracy of 93.6667 so when we look at the number of loans and we look at how good our model fit we can tell people it has about a 93.6 fitting to it so just a quick recap on that we now have accuracy set up on here and so we have created a model that uses the decision tree algorithm to predict whether a customer will repay the loan or not the accuracy of the model is about 94.6 percent the bank can now use this model to decide whether it should approve the loan request from a particular customer or not and so this information is really powerful we may not be able to as individuals understand all these numbers because they have thousands of numbers that come in but you can see that this is a smart decision for the bank to use a tool like this to help them to predict how good their profit's going to be off of the loan balances and how many are going to default or not so why random forest it's always important to understand why we use this tool over the other ones what are the benefits here and so with the random forest the first one is there's no overfitting if you use of multiple trees reduce the risk of overfitting training time is less overfitting means that we have fit the data so close to what we have as our sample that we pick up on all the weird parts and instead of predicting the overall data you're predicting the weird stuff which you don't want high accuracy runs efficiently in large database for large data it produces highly accurate predictions in today's world of big data this is really important and this is probably where it really shines this is where why random forest really comes in it estimates missing data data in today's world is very messy so when you have a random forest it can maintain the accuracy when a large proportion of the data is missing what that means is if you have data that comes in from five or six different areas and maybe they took one set of statistics in one area and they took a slightly different set of statistics in the other so they have some of the same same shared data but one is missing like the number of children in the house if you're doing something over demographics and the other one is missing the size of the house it will look at both of those separately and build two different trees and then it can do a very good job of guessing which one fits better even though it's missing that data let us dig deep into the theory of exactly how it works and let's look at what is a random forest random forest or random decision forest is a method that operates by constructing multiple decision trees the decision of the majority of the trees is chosen by the random forest as a final decision and this uh we have some nice graphics here we have a decision tree and they actually use a real tree to denote the decision tree which i love and given a random some kind of picture of a fruit this decision tree decides that the output is it's an apple and we have a decision tree two where we have that picture of the fruit goes in and this one decides that it's a lemon and the decision three tree gets another image and it decides it's an apple and then this all comes together in what they call the random forest and this random forest then looks at it and says okay i got two votes for apple one vote for lemon the majority is apples so the final decision is apples to understand how the random forest works we first need to dig a little deeper and take a look at the random forest and the actual decision tree and how it builds that decision tree in looking closer at how the individual decision trees work we'll go ahead and continue to use the fruit example since we're talking about trees and forests a decision tree is a tree shaped diagram used to determine a course of action each

branch of the tree represents a possible decision occurrence or reaction so in here we have a bowl of fruit and if you look at that it looks like they switch from lemons to oranges we have oranges cherries and apples and the first decision of the decision tree might be is a diameter greater than or equal to three and if it says false it knows that they're cherries because everything else is bigger than that so all the cherries fall into that decision so we have all that data we're training we can look at that we know that that's what's going to come up is the color orange well goes hmm orange or red well if it's true then it comes out as the orange and if it's false that leaves apples so in this example it sorts out the fruit in the bowl or the images of the fruit a decision tree these are very important terms to know because these are very central to understanding the decision tree when working with them the first is entropy everything on the decision tree and how it makes this decision is based on entropy entropy is a measure of randomness or unpredictability in the data set then they also have information gain the leaf node the decision node and the root node we'll cover these other four terms as we go down the tree but let's start with entropy so starting with entropy we have here a high amount of randomness what that means is that whatever is coming out of this decision if it was going to guess based on this data it wouldn't be able to tell you whether it's a lemon or an apple it would just say it's a fruit so the first thing we want to do is we want to split this apart and we take the initial data set we're going to create a data set one and a data set two we just split it in two and if you look at these new data sets after splitting them the entropy of each of those sets is much less so for the first one whatever comes in there it's going to sort that data and it's going to say okay if this data goes this direction it's probably an apple and if it goes into the other direction it's probably a lemon so that brings us up to information gain it is the measure of decrease in the entropy after the data set is split what that means in here is that we've gone from one set which has a very high entropy to two lower sets of entropy and we've added in the values of e1 for the first one and e2 for the second two which are much lower and so that information gain is increased greatly in this example and so you can find that the information grain simply equals decision e1 minus e2 as we're going down our list of definitions we'll look at the leaf node and the leaf node carries the classification or the decision so we look down here to the leaf node we finally get to our set one or our set two when it comes down there and it says okay this object's gone into set one if it's gone into set one it's going to be split by some means and we'll either end up with apples on the leaf node or a lemon on the leaf node and on the right it'll either be an apple or lemons those leaf nodes are those final decisions or classifications that's the definition of leaf node in here if we're going to have a final leaf where we make the decision we should have a name for the nodes above it and they call those decision nodes a decision node decision node has two or more branches and you can see here where we have the five apples and one lemon and in the other case the five lemons in one apple they have to make a choice of which tree it goes down based on some kind of measurement or information given to the tree and that brings us to our last definition the root node the top most decision node is known as the root node and this is where you have all of your data and you have your first decision it has to make or the first split in information so far we've looked at a very general image with the fruit being split let's look and see exactly what that means to split the data and how do we make those decisions on there let's go in there and find out how does a decision tree work so let's try to understand this and let's use a simple example and we'll stay with the fruit we have a bowl of fruit and so let's create a problem statement and the problem is we want to classify the different types of fruits in the bowl based on different features the data set in the bowl is looking quite messy and the entropy is high in this case so if this bowl was our decision maker it wouldn't know what choice to make it has so many choices which one do you pick apple grapes or lemons and so we look in here we're going to start with the dra a training set so this is our data

that we're training our data with and we have a number of options here we have the color and under the color we have red yellow purple we have a diameter 331331 and we have a label apple lemon grapes apple lemon grapes and how do we split the data we have to frame the conditions to split the data in such a way that the information gain is the highest it's very key to note that we're looking for the best gain we don't want to just start sorting out the smallest piece in there we want to split it the biggest way we can and so we measure this decrease in entropy that's what they call it entropy there's our entropy after splitting and now we'll try to choose a condition that gives us the highest gain we will do that by splitting the data using each condition and checking the gain that we get out of them the conditions that give us the highest gain will be used to make the first split so let's take a look at these different conditions we have color we have diameter and if we look underneath that we have a couple different values we have diameter equals three color equals yellow red diameter equals one and when we look at that you'll see over here we have one two three four threes that's a pretty hardy selection so let's say the condition gives us a maximum gain of three so we have the most pieces fall into that range so our first split from our decision node is we split the data based on the diameter is it greater than or equal to three if it's not that's false it goes into the grape bowl and if it's true it goes into a bowl fold of lemon and apples the entropy after splitting has decreased considerably so now we can make two decisions if you look at there very much less chaos going on there this note has already attained an entropy value of zero as you can see there's only one kind of label left for this branch so no further splitting is required for this node however this node on the right is still requires a split to decrease the entropy further so we split the right node further based on color if you look at this if i split it on color that pretty much cuts it right down the middle it's the only thing we have left in our choices of color and diameter too and if the color is yellow it's going to go to the right bowl and if it's false it's going to go to the left bowl so the entropy in this case is now zero so now we have three bowls with zero entropy there's only one type of data in each one of those bowls so we can predict a lemon with a hundred percent accuracy and we can predict the apple also with 100 accuracy along with our grapes up there in this example last week my son and i visited a fruit shop dad is that an apple or a strawberry so the question comes up what fruit do they just pick up from the fruit stand after a couple of seconds you could figure out that it was a strawberry so let's take this model a step further and let's uh why not build a model which can predict an unknown data and in this we're going to be looking at some sweet strawberries or crispy apples we want to be able to label those two and decide what the fruit is and we do that by having data already put in so we already have a bunch of strawberries we know our strawberries and already labeled as such we already have a bunch of apples we know our apples and are labeled as such then once we train our model that model then can be given the new data and the new data is this image in this case you can see a question mark on it and it comes through and goes it's a strawberry in this case we're using the support vector machine model svm is a supervised learning method that looks at data and sorts it into one of two categories and in this case we're sorting the strawberry into the strawberry site at this point you should be asking the question how does the prediction work before we dig into an example with numbers let's apply this to our fruit scenario we have our support vector machine we've taken it and we've taken labeled sample of data strawberries and apples and we draw on a line down the middle between the two groups this split now allows us to take new data in this case an apple and a strawberry and place them in the appropriate group based on which side of the line they fall in and that way we can predict the unknown as colorful and tasty as the fruit example is let's take a look at another example with some numbers involved and we can take a closer look at how the math works in this example we're going to be classifying men and women and we're going to start with a set of people with a different height and a different weight and to

make this work we'll have to have a sample data set a female where you have their height and weight 174 65 174 88 and so on and we'll need a sample data set of the male they have a height 179 90 180 to 80 and so on let's go ahead and put this on a graph so you have a nice visual so you can see here we have two groups based on the height versus the weight and on the left side we're going to have the women on the right side we're going to have the men now if we're going to create a classifier let's add a new data point and figure out if it's male or female so before we can do that we need to split our data first we can split our data by choosing any of these lines in this case we draw on two lines through the data in the middle that separates them in from the women but to predict the gender of a new data point we should split the data in the best possible way and we say the best possible way because this line has a maximum space that separates the two classes here you can see there's a clear split between the two different classes and in this one there's not so much a clear split this doesn't have the maximum space that separates the two that is why this line best splits the data we don't want to just do this by eyeballing it and before we go further we need to add some technical terms to this we can also say that the distance between the points in the line should be as far as possible in technical terms we can say the distance between the support vector and the hyperplane should be as far as possible and this is where the support vectors are the extreme points in the data set and if you look at this data set they have circled two points which seem to be right on the outskirts of the woman and one on the outskirts of the men and hyperplane has a maximum distance to the support vectors of any class now you'll see the line down the middle and we call this the hyperplane because when you're dealing with multiple dimensions it's really not just a line but a plane of intersections and you can see here where the support vectors have been drawn in dashed lines the math behind this is very simple we take d plus the shortest distance to the closest positive point which would be on the min side and d minus is the shortest distance to the closest negative point which is on the women's side the sum of d plus and d minus is called the distance margin or the distance between the two support vectors that are shown in the dashed lines and then by finding the largest distance margin we can get the optimal hyperplane once we've created an optimal hyperplane we can easily see which side the new data fits in and based on the hyperplane we can say the new data point belongs to the male gender hopefully that's clear how that works on a visual level as a data scientist you should also be asking what happens if the hyperplane is not optimal if we select a hyperplane having low margin then there is a high chance of misclassification this particular svm model the one we discussed so far is also called referred to as the ls vm so far so clear but a question should be coming up we have our sample data set but instead of looking like this what if it looked like this where we have two sets of data but one of them occurs in the middle of another set you can see here where we have the blue and the yellow and then blue again on the other side of our data line in this data set we can't use a hyperplane so when you see data like this it's necessary to move away from a 1d view of the data to a two-dimensional view of the data and for the transformation we use what's called a kernel function the kernel function will take the 1d input and transfer it to a two-dimensional output as you can see in this picture here the 1d when transferred to a two-dimensional makes it very easy to draw a line between the two data sets what if we make it even more complicated how do we perform an svm for this type of data set here you can see we have a two-dimensional data set where the data is in the middle surrounded by the green data on the outside in this case we're going to segregate the two classes we have our sample data set and if you draw a line through it's obviously not an optimal hyperplane in there so to do that we need to transfer the 2d to a 3d array and when you translate it into a three-dimensional array using the kernel you can see where you can place a hyperplane right through it and easily split the data before we start looking at a programming example and dive into the script let's look at

the advantage of the support vector machine we'll start with high dimensional input space or sometimes referred to as the curse of dimensionality we looked at earlier one dimension two dimension three dimension when you get to a thousand dimensions a lot of problems start occurring with most algorithms that have to be adjusted for the svm automatically does that in high dimensional space one of the high dimensional space one high dimensional space that we work on is sparse document vectors this is where we tokenize the words and documents so we can run our machine learning algorithms over them i've seen ones get as high as 2.4 million different tokens that's a lot of vectors to look at and finally we have regularization parameter the realization parameter or lambda is a parameter that helps figure out whether we're going to have a bias or overfitting of the data whether it's going to be overfitted to a very specific instance or it's going to be biased to a high or low value with the svm it naturally avoids the overfitting and bias problems that we see in many other algorithms these three advantages of the support vector machine make it a very powerful tool to add to your repertoire of machine learning tools now we did promise you a used case study we're actually going to dive into some python programming and so we're going to go into a problem statement and start off with the zoo so in the zoo example we have family members going to the zoo we have the young child going dead is that a group of crocodiles or alligators well that's hard to differentiate and zoos are a great place to start looking at science and understanding how things work especially as a young child and so we can see the parents sitting here thinking well what is the difference between a crocodile and an alligator well one crocodiles are larger in size alligators are smaller in size snout widths the crocodiles have a narrow snout and alligators have a wider snout and of course in the modern day and age the father is sitting here thinking how can i turn this into a lesson for my son and he goes let a support vector machine segregate the two groups i don't know if my dad ever told me that but that would be funny now in this example we're not going to use actual measurements and data we're just using that for imagery and that's very common in a lot of machine learning algorithms and setting them up but let's roll up our sleeves and we'll talk about that more in just a moment as we break into our python script so here we arrive in our actual coding and i'm going to move this into a python editor in just a moment but let's talk a little bit about what we're going to cover first we're going to cover in the code the setup how to actually create our svm and you're going to find that there's only two lines of code that actually create it and the rest of it is done so quick and fast that it's all here in the first page and we'll show you what that looks like as far as our data because we're going to create some data i talked about creating data just a minute ago and so we'll get into the creating data here and you'll see this nice correction of our two blops and we'll go through that in just a second and then the second part is we're going to take this and we're going to bump it up a notch we're going to show you what it looks like behind the scenes but let's start with actually creating our setup i like to use the anaconda jupiter notebook because it's very easy to use but you can use any of your favorite python editors or setups and go in there but let's go ahead and switch over there and see what that looks like so here we are in the anaconda python notebook or anaconda jupiter notebook with python we're using python 3. i believe this is 3.5 but it should be work in any of your 3x versions and you'd have to look at the sklearn and make sure if you're using a 2x version an earlier version let's go and put our code in there and one of the things i like about the jupiter notebook is i go up to view and i'm going to go ahead and toggle the line numbers on to make it a little bit easier to talk about and we can even increase the size because this is edited in this case i'm using google chrome explorer and that's how it opens up for the editor although anyone any like i said any editor will work now the first step is going to be our imports and we're going to import four different parts the first two i want you to look at are line one and line two are numpy as np and matplot library dot pi plot as plt now these are very

standardized imports when you're doing work the first one is the numbers python we need that because part of the platform we're using uses that for the numpy array and i'll talk about that in a minute so you can understand why we want to use a numpy array versus the standard python array and normally it's pretty standard setup to use np for numpy the map plot library is how we're going to view our data so this has you do need the np for the sk learn module but the map plot library is purely for our use for visualization and so you really don't need that for the svm but we're going to put it there so you have a nice visual aid and we can show you what it looks like that's really important at the end when you finish everything so you have a nice display for everybody to look at and then finally we're going to i'm going to jump one ahead to line number four that's the sklearn.datasets.samplesgenerator import make blobs and i told you that we were going to make up data and this is a tool that's in the sk learning to makeup data i personally don't want to go to the zoo get in trouble for jumping over the fence and probably get eaten by the crocodiles or alligators as i work on measuring their snouts and width and length instead we're just going to make up some data and that's what that make blobs is it's a wonderful tool if you're ready to test your your setup and you're not sure about what data you're going to put in there you can create this blob and it makes it really easy to use and finally we have our actual svm the sk learn import svm on line three so that covers all our imports we're going to create remember i use the make blobs to create data and we're going to create a capital x and a lowercase y equals make blobs in samples equals 40. so we're going to make 40 lines of data it's going to have two centers with a random state equals twenties which each each group is going to have 20 different pieces of data in it and the way that looks is that we'll have under x um an x y plane so i have two numbers under x and y will be zero one that's the two different centers so we have yes or no in this case alligator or crocodile that's what that represents and then i told you that the actual sk learner the svm is in two lines of code and we see it right here with clf equals svm.svc kernel equals linear and i set sequel to 1 although in this example since we are not regularizing the data because we want to be very clear and easy to see i went ahead you can set it to a thousand a lot of times when you're not doing that but for this thing linear because it's a very simple linear example we only have the two dimensions and it'll be a nice linear hyperplane it'll be a nice linear line instead of a full plane so we're not dealing with a huge amount of data and then all we have to do is do clf dot fit x comma y and that's it clf has been created and then we're going to go ahead and display it and i'm going to talk about this display here in just a second but let me go ahead and run this code and this is what we've done is we've created two blobs you'll see the blue on the side and then kind of an orangish on the other side that's our two sets of data they represent one represents crocodiles and one represents alligators and then we have our measurements in this case we have like the width and length of the snout and i did say i was going to come up here and talk just a little bit about our plot and you'll see plt that's what we imported we're going to do a scatter plot that means we're just putting dots on there and then look at this notation i have the capital x and then in brackets i have a colon comma 0. that's from numpy if you did that in a regular array you'll get an error in a python array you have to have that in a numpy array it turns out that our make blobs returns a numpy array and this notation is great because what it means is the first part is the colon means we're going to do all the rows that's all the data in our blob we created under capital x and then the second part has a comma zero we're only going to take the first value and then if you notice we do the same thing but we're going to take the second value remember we always start with zero and then one so we have column zero and column one and you can look at this as our x y plots the first one is the x plot and the second one is the y plot so the first one is on the bottom 0 2 4 6 8 and 10 and then the second one x of the one is the 4 5 6 7 8 9 10 going up the left hand side s equals 30 is just the size of the dots we can see them and said little tiny dots and then the

c map equals plt.cm.paired and you'll also see the c equals y that's the color we're using two colors zero one and that's why we get the nice blue and the two different colors for the alligator and the crocodile now you can see here that we did this the actual fit was done in two lines of code a lot of times will be a third line where we regularize the data we set it between like minus one and one and we reshape it but for this it's not necessary and it's also kind of nice because you can actually see what's going on and then if we wanted to we wanted to actually run a prediction let's take a look and see what that looks like and to predict some new data and we'll show this again as we get towards the end of digging in deep you can simply assign your new data in this case i am giving it a width and length 3 4 and a width and length 5 6. and note that i put the data as a set of brackets and then i have the brackets inside and the reason i do that is because when we're looking at data it's designed to process a large amount of data coming in we don't want to just process one line at a time and so in this case i'm processing two lines and then i'm just going to print and you'll see clf dot predict new data so the clf and the dot predict part is going to give us an answer and let's see what that looks like and you'll see 0 1. so predicted the first one the 3 4 is going to be on the one side and the 5 6 is going to be on the other side so one came out as an alligator and one came out as a crocodile now that's pretty short explanation for the setup but really we want to dug in and see what it's going on behind the scenes and let's see what that looks like so the next step is to dig in deep and find out what's going on behind the scenes and also put that in a nice pretty graph we're going to spend more work on this than we did actually generating the original model and you'll see here that we go through a few steps and i'll move this over to our editor in just a second we come in we create our original data it's exactly identical to the first part and i'll explain why we redid that and show you how not to redo that and then we're going to go in there and add in those lines we're going to see what those lines look like and how to set those up and finally we're going to plot all that on here and show it and you'll get a nice graph with the what we saw earlier when we were going through the theory behind this where it shows the support vectors and the hyperplane and those are done where you can see the support vectors as the dashed lines and the solid line which is the hyperplane let's get that into our jupiter notebook before i scroll down to a new line i want you to notice line 13 it has plot show and we're going to talk about that here in just a second but let's scroll down to a new line down here and i'm going to paste that code in and you'll see that the plot show has moved down below let's scroll up a little bit and if you look at the top here of our new section one two three and four is the same code we had before and let's go back up here and take a look at that we're gonna fit the values on our svm and then we're gonna plot scatter it and then we're gonna do a plot show so you should be asking why are we redoing the same code well when you do the plot show that blanks out what's in the plot so once i've done this plot show i have to reload that data now we could do this simply by removing it up here re-running it and then coming down here and then we wouldn't have to rerun these first four lines of code now in this it doesn't matter too much and you'll see the plot show was down here and then removed right there on line five i'll go ahead and just delete that out of there because we don't want to blank out our screen we want to move on to the next setup so we can go ahead and just skip the first four lines because we did that before and let's take a look at the ax equals plt.gca now right now we're actually spending a lot of time just graphing that's all we're doing here okay so this is how we display a nice graph with our results in our data ax is very standard note used variable when you talk about plt and it's just setting it to that axis the last axis in the plt it can get very confusing if you're working with many different layers of data on the same graph and this makes it very easy to reference the ax so this reference is looking at the plt that we created and we already mapped out our two blobs on and then we want to know the limits so we want to know how big the graph is we can find out the x limit and the y limit

simply with the get x limit and get y limit commands which is part of our metplot library and then we're going to create a grid and you'll see down here we have we've set the variable xx equal to np.line space x limit 0 x limit 1 comma 30 and we've done the same thing for the y space and then we're going to go in here and we create a mesh grid and this is a numpy command so we're back to our numbers python let's go through what these numpy commands mean with the line space and the mesh grid we've taken x x small s x x equals np line space and we have our x limit 0 and our x limit 1 and we're going to create 30 points on it and we're going to do the same thing for the y axis now this has nothing to do with our evaluation it's all we're doing is we're creating a grid of data and so we're creating a set of points between 0 and the x limit we're creating 30 points and the same thing with the y and then the mesh grid loops those all together so it forms a nice grid so if we were going to do this say between the limit 0 and 10 and do 10 points we would have a 0 0 1 1 0 1 0 2 0 3 0 4 to 10 and so on you can just imagine a point at each corner one of those boxes and the mesh grid combines them all so we take the yy and the xx we created and creates the full grid and we've set that grid into the yy coordinates and the xx coordinates now remember we're working with numbi and python we like to separate those we like to have instead of it being x comma 1 you know x comma y and then x 2 comma y 2 and this in the next set of data it would be a column of x's and a column of y's and that's what we have here is we have a column of y's and we put it as a capital yy and a column of x's capital xx with all those different points being listed and finally we get down to the numpy v stack just as we created those in the mesh grid we're now going to put them all into one array x y array now that we've created the stack of data points we're going to do something interesting here we're going to create a value z and the z equals the clf that's our that's our support vector machine we created and we've already trained and we have a dot decision function we're going to put the x y in there so here we have all this data we're going to put that x y in there that data and we're going to reshape it and you'll see that we have the x x dot shape in here this literally takes the xx resets it up connected to the y and the z value lets us know whether it is the left hand side is going to generate three different values the z value does and it'll tell us whether that data is a support vector to the left the hyperplane in the middle or the support vector to the right so it generates three different values for each of those points and those points have been reshaped so they're right on a line on those three different lines so we've set all of our data up we've labeled it to three different areas and we've reshaped it and we've just taken 30 points in each direction if you do the math you have 30 times 30 so it's 900 points of data and we separate it between the three lines and reshaped it to fit those three lines we can then go back to our map plot library we've created the ax and we're going to create a contour and you'll see here we have contour capital xx capital yy these have been reshaped to fit those lines z is the labels so now we have the three different points with the labels in there and we can set the colors equals k and i told you we had three different labels but we have three levels of data the alphas just makes it kind of see-through so it's only 0.5 of the value in there so when we graph it the data will show up from behind it wherever the lines go and finally the line styles this is where we set the two support vectors to be dash dash lines and then a single one is just a straight line that's what all that setup does and then finally we take our ax dot scatter we're going to go ahead and plot the support vectors but we've programmed it in there so that they look nice like the dash dash line and the dashed line on that grid and you can see here when we do the clf dot support vectors we are looking at column 0 and column 1 and then again we have the s equals 100 so we're going to make them larger and the line width equals 1 face colors equals none let's take a look and see what that looks like when we show it and you can see we get down to our end result it creates a really nice graph we have our two support vectors and dashed lines and they have the near data so you can see those two points or in this case the four points where those

lines nicely cleave the data and then you have your hyper plane down the middle which is as far from the two different points as possible creating the maximum distance so you can see that we have our nice output for the size of the body and the width of the snout and we've easily separated the two groups of crocodile and alligator congratulations you've done it we've made it of course these are pretend data for our crocodiles and alligators but this hands-on example will help you to encounter any support vector machine projects in the future and you can see how easy they are to set up and look at in depth we're going to cover the k nearest neighbors like referred to as k n n and k n is really a fundamental place to start in the machine learning it's the basis of a lot of other things and just the logic behind it is easy to understand and incorporated in other forms of machine learning so today what's in it for you why do we need k n n what is k n how do we choose the factor k when do we use k n n how does k n algorithm work and then we'll dive in to my favorite part the use case predict whether a person will have diabetes or not that is a very common and popular used data set as far as testing out models and learning how to use the different models in machine learning by now we all know machine learning models make predictions by learning from the past data available so we have our input values our machine learning model builds on those inputs of what we already know and then we use that to create a predicted output is that a dog little kid looking over there watching the black cat cross their path no dear you can differentiate between a cat and a dog based on their characteristics cats cats have sharp claws uses to climb smaller length of ears meows and purrs doesn't love to play around dogs they have dull claws bigger length of ears barks loves to run around you usually don't see a cat running around people although i do have a cat that does that where dogs do and we can look at these we can say we can evaluate the sharpness of the claws how sharp brother claws and we can evaluate the length of the ears and we can usually sort out cats from dogs based on even those two characteristics now tell me if it is a cat or a dog not question usually little kids know cats and dogs by now unless they live a place where there's not many cats or dogs so if we look at the sharpness of the claws the length of the ears and we can see that the cat has smaller ears and sharper claws than the other animals its features are more like cats it must be a cat sharp claws length of ears and goes in the cat group because knn is based on feature similarity we can do classification using knn classifier so we have our input value the picture of the black cat it goes into our trained model and it predicts that this is a cat coming out so what is knn what is the k n algorithm k nearest neighbors is what that stands for it's one of the simplest supervised machine learning algorithms mostly used for classification so we want to know is this a dog or is not a dog is it a cat or not a cat it classifies a data point based on how his neighbors are classified knn stores all available cases and classifies new cases based on a similarity measure and here we gone from cats and dogs right into wine another favorite of mine k n stores all available cases and classifies new cases based on a similarity measure and here you see we have a measurement of sulfur dioxide versus the chloride level and then the different wines they've tested and where they fall on that graph based on how much sulfur dioxide and how much chloride k and k n is a perimeter that refers to the number of nearest neighbors to include in the majority of the voting process and so if we add a new glass of wine there red or white we want to know what the neighbors are in this case we're going to put k equals 5. we'll talk about k in just a minute a data point is classified by the majority of votes from its five nearest neighbors here the unknown point would be classified as red since four out of five neighbors are red so how do we choose k how do we know k equals 5 i mean that's what's the value we put in there so we're going to talk about it how do we choose a factor k k n algorithm is based on feature similarity choosing the right value of k is a process called parameter tuning and is important for better accuracy so at k equals three we can classify we have a question mark in the middle as either a as a square or not is it a square or is it in this

case a triangle and so if we set k equals to three we're going to look at the three nearest neighbors we're going to say this is a square and if we put k equals a 7 we classify as a triangle depending on what the other data is around and you can see as the k changes depending on where that point is that drastically changes your answer and we jump here we go how do we choose the factor of k you'll find this in all machine learning choosing these factors that's the face you get he's like oh my gosh did i choose the right k did i set it right my values in whatever machine learning tool you're looking at so that you don't have a huge bias in one direction or the other and in terms of k n n the number of k if you choose it too low the bias is based on it's just two noises it's it's right next to a couple things and it's going to pick those things and you might get a skewed answer and if your k is too big then it's going to take forever to process so you're going to run into processing issues and resource issues so what we do the most common use and there's other options for choosing k is to use the square root of n so it is a total number of values you have you take the square root of it in most cases you also if it's an even number so if you're using like this case squares and triangles if it's even you want to make your k value odd that helps it select better so in other words you're not going to have a balance between two different factors that are equal so usually take the square root of n and if it's even you add one to it or subtract one from it and that's where you get the k value from that is the most common use and it's pretty solid it works very well when do we use knn we can use knn when data is labeled so you need a label on it we know we have a group of pictures with dogs dogs cats cats data is noise free and so you can see here when we have a class and we have like underweight 140 23 hello kitty normal that's pretty confusing we have a high variety of data coming in so it's very noisy and that would cause an issue data set is small so we're usually working with smaller data sets where you might get into a gig of data if it's really clean doesn't have a lot of noise because k n is a lazy learner i.e it doesn't learn a discriminative function from the training set so it's very lazy so if you have very complicated data and you have a large amount of it you're not going to use the knn but it's really great to get a place to start even with large data you can sort out a small sample and get an idea of what that looks like using the knn and also just using for smaller data sets k n works really good how does a k n algorithm work consider a data set having two variables height in centimeters and weight in kilograms and each point is classified as normal or underweight so we see right here we have two variables you know true false they're either normal or they're not they're underweight on the basis of the given data we have to classify the below set as normal or underweight using knn so if we have new data coming in that says 57 kilograms and 177 centimeters is that going to be normal or underweight to find the nearest neighbors we'll calculate the euclidean distance according to the euclidean distance formula the distance between two points in the plane with the coordinates x y and a b is given by distance d equals the square root of x minus a squared plus y minus b squared and you can remember that from the two edges of a triangle we're computing the third edge since we know the x side and the y side let's calculate it to understand clearly so we have our unknown point and we placed it there in red and we have our other points where the data is scattered around the distance d1 is the square root of 170 minus 167 squared plus 57 minus 51 squared which is about 6.7 and distance 2 is about 13. and distance 3 is about 13.4 similarly we will calculate the euclidean distance of unknown data point from all the points in the data set and because we're dealing with small amount of data that's not that hard to do it's actually pretty quick for a computer and it's not a really complicated mass you can just see how close is the data based on the euclidean distance hence we have calculated the euclidean distance of unknown data point from all the points as shown where x1 and y1 equal 57 and 170 whose class we have to classify so now we're looking at they were saying well here's the euclidean distance who's going to be their closest neighbors now let's calculate the nearest neighbor at k equals three and we can

see the three closest neighbors puts them at normal and that's pretty self-evident when you look at this graph it's pretty easy to say okay what you know we're just voting normal normal normal three votes for normal this is going to be a normal weight so majority of neighbors are pointing towards normal hence as per k n algorithm the class of 57 170 should be normal so a recap of k n n positive integer k is specified along with a new sample we select the k entries in our database which are closest to the new sample we find the most common classification of these entries this is the classification we give to the new sample so as you can see it's pretty straightforward we're just looking for the closest things that match what we got so let's take a look and see what that looks like in a use case in python so let's dive into the predict diabetes use case so use case predict diabetes the objective predict whether a person will be diagnosed with diabetes or not we have a data set of 768 people who were or were not diagnosed with diabetes and let's go ahead and open that file and just take a look at that data and this is in a simple spreadsheet format the data itself is comma separated very common set of data and it's also a very common way to get the data and you can see here we have columns a through i that's what one two three four five six seven eight eight columns with a particular attribute and then the ninth column which is the outcome is whether they have diabetes as a data scientist the first thing you should be looking at is insulin well you know if someone has insulin they have diabetes because that's why they're taking it and that could cause issue in some of the machine learning packages but for a very basic setup this works fine for doing the knn and the next thing you notice is it didn't take very much to open it up i can scroll down to the bottom of the data there's 768. it's pretty much a small data set you know at 769 i can easily fit this into my ram on my computer i can look at it i can manipulate it and it's not going to really tax just a regular desktop computer you don't even need an enterprise version to run a lot of this so let's start with importing all the tools we need and before that of course we need to discuss what ide i'm using certainly can use any particular editor for python but i like to use for doing very basic visual stuff the anaconda which is great for doing demos with the jupiter notebook and just a quick view of the anaconda navigator which is the new release out there which is really nice you can see under home i can choose my application we're going to be using python36 i have a couple different uh versions on this particular machine if i go under environments i can create a unique environment for each one which is nice and there's even a little button there where i can install different packages so if i click on that button and open the terminal i can use a simple pip install to install different packages i'm working with let's go ahead and go back under home and we're going to launch our notebook and i've already you know kind of like the old cooking shows i've already prepared a lot of my stuff so we don't have to wait for it to launch because it takes a few minutes for it to open up a browser window in this case i'm going to it's going to open up chrome because that's my default that i use and since the script is pre-done you'll see i have a number of windows open up at the top the one we're working in and since we're working on the k n predict whether a person will have diabetes or not let's go and put that title in there and i'm also going to go up here and click on cell actually we want to go ahead and first insert a cell below and then i'm going to go back up to the top cell and i'm going to change the cell type to markdown that means this is not going to run as python it's a markdown language so if i run this first one it comes up in nice big letters which is kind of nice remind us what we're working on and by now you should be familiar with doing all of our imports we're going to import the pandas as pd import numpy is np pandas is the pandas data frame and numpy is a number array very powerful tools to use in here so we have our imports so we've brought in our pandas our numpy our two general python tools and then you can see over here we have our trained test split by now you should be familiar with splitting the data we want to split part of it for training our thing and then training our particular model and then we want to go ahead and test the remaining

data to see how good it is pre-processing a standard scalar preprocessor so we don't have a bias of really large numbers remember in the data we had like number of pregnancies isn't going to get very large where the amount of insulin they take can get up to 256 so 256 versus 6 that will skew results so we want to go ahead and change that so they're all uniform between minus 1 and 1. and then the actual tool this is the k neighbors classifier we're going to use and finally the last three are three tools to test all about testing our model how good is it we just put down test on there and we have our confusion matrix our f1 score and our accuracy so we have our two general python modules we're importing and then we have our six modules specific from the sk learn setup and then we do need to go ahead and run this so these are actually imported there we go and then move on to the next step and so in this set we're going to go ahead and load the database we're going to use pandas remember pandas is pd and we'll take a look at the data in python we looked at it in a simple spreadsheet but usually i like to also pull it up so that we can see what we're doing so here's our data set equals pd.read csv that's a pandas command and the diabetes folder i just put in the same folder where my ipython script is if you put in a different folder you need the full length on there we can also do a quick links of the data set that is a simple python command len for length we might even let's go ahead and print that we'll go print and if you do it on its own line link that data set the jupyter notebook it'll automatically print it but when you're in most of your different setups you want to do the print in front of there and then we want to take a look at the actual data set and since we're in pandas we can simply do data set head and again let's go ahead and add the print in there if you put a bunch of these in a row you know the data set one head data set two head it only prints out the last one so i usually always like to keep the print statement in there but because most projects only use one data frame panda's data frame doing it this way doesn't really matter the other way works just fine and you can see when we hit the run button we have the 768 lines which we knew and we have our pregnancies it's automatically given a label on the left remember the head only shows the first five lines so we have zero through four and just a quick look at the data you can see it matches what we looked at before we have pregnancy glucose blood pressure all the way to age and then the outcome on the end and we're going to do a couple things in this next step we're going to create a list of columns where we can't have zero there's no such thing as zero skin thickness or zero blood pressure zero glucose uh any of those you'd be dead so not a really good factor if they don't if they have a 0 in there because they didn't have the data and we'll take a look at that because we're going to start replacing that information with a couple of different things and let's see what that looks like so first we create a nice list as you can see we have the values talked about glucose blood pressure skin thickness and this is a nice way when you're working with columns is to list the columns you need to do some kind of transformation on a very common thing to do and then for this particular setup we certainly could use the there's some panda tools that will do a lot of this where we can replace the n a but we're going to go ahead and do it as a data set column equals data set column dot replace this is this is still pandas you can do a direct there's also one that's that you look for your nan a lot of different options in here but the n a n in num p n a n is what that stands for is none it doesn't exist so the first thing we're doing here is we're replacing the zero with a numpy none there's no data there that's what that says that's what this is saying right here so put the zero in and we're going to play zeros with no data so if it's a zero that means the person's well hopefully not dead hope they just didn't get the data the next thing we want to do is we're going to create the mean which is the integer from the data set from the column dot mean where we skip n a's we can do that that is a pandas command there the skip n a so we're going to figure out the mean of that data set and then we're going to take that data set column and we're going to replace all the npn with the means why did we do that and we could have actually just taken this

step and gone right down here and just replaced zero and skip anything where except you could actually there's a way to skip zeros and then just replace all the zeros but in this case we want to go ahead and do it this way so you could see that we're switching this to a non-existent value then we're going to create the mean well this is the average person so if we don't know what it is if they did not get the data and the data is missing one of the tricks is you replace it with the average what is the most common data for that this way you can still use the rest of those values to do your computation and it kind of just brings that particular value of those missing values out of the equation let's go ahead and take this and we'll go ahead and run it doesn't actually do anything so we're still preparing our data if you want to see what that looks like we don't have anything in the first few lines just not going to show up but we certainly could look at a row let's do that let's go into our data set with printed data set and let's pick in this case let's just do glucose and if i run this this is going to print all the different glucose levels going down and we thankfully don't see anything in here that looks like missing data at least on the ones that shows you can see you skipped a bunch in the middle because that's what it does if you have too many lines in jupiter notebook it'll skip a few and go on to the next in a data set let me go and remove this and we'll just zero out that and of course before we do any processing before proceeding any further we need to split the data set into our train and testing data that way we have something to train it with and something to test it on and you're going to notice we did a little something here with the pandas database code there we go my drawing tool we've added in this right here of the data set and what this says is that the first one in pandas this is from the pd pandas it's going to say within the data set we want to look at the eye location and it is all rows that's what that says so we're going to keep all the rows but we're only looking at 0 column 0 to 8. remember column 9 here it is right up here we printed it in here is outcome well that's not part of the training data that's part of the answer yes column nine but it's listed as eight number eight so zero to eight is nine columns so uh eight is the value and when you see it in here zero this is actually zero to seven it doesn't include the last one and then we go down here to y which is our answer and we want just the last one just column 8 and you can do it this way with this particular notation and then if you remember we imported the train test split that's part of the sk learn right there and we simply put in our x and our y we're going to do random state equals zero you don't have to necessarily seat it that's a seed number i think the default is one when you see it i have to look that up and then the test size test size is 0.2 that simply means we're going to take 20 of the data and put it aside so that we can test it later that's all that is and again we're going to run it not very exciting so far we haven't had any printout other than to look at the data but that is a lot of this is prepping this data once you prep it the actual lines of code are quick and easy and we're almost there with the actual writing of our knn we need to go ahead and do a scale the data if you remember correctly we're fitting the data in a standard scalar which means instead of the data being from you know 5 to 303 in one column and the next column is 1 to 6 we're going to set that all so that all the data is between -1 and 1. that's what that standard scalar does keeps it standardized and we only want to fit the scalar with the training set but we want to make sure the testing set is the x test going in is also transformed so it's processing it the same so here we go with our standard scalar we're going to call it sc underscore x for the scalar and we're going to import the standard scalar into this variable and then our x train equals sc underscore x dot fit transform so we're creating the scalar on the x train variable and then our x test we're also going to transform it so we've trained and transformed the x train and then the x test isn't part of that training it isn't part of that of training the transformer it just gets transformed that's all it does and again we're going to go and run this if you look at this we've now gone through these steps all three of them we've taken care of replacing our zeros for key columns it shouldn't be zero and we replace that with the means of

those columns that way that they fit right in with our data models we've come down here we split the data so now we have our test data and our training data and then we've taken and we scaled the data so all of our data going in no no we don't tr we don't train the y part the y train and y test that never has to be trained it's only the data going in that's what we want to train in there then define the model using k neighbors classifier and fit the train data in the model so we do all that data prep and you can see down here we're only going to have a couple lines of code where we're actually building our model and training it that's one of the cool things about python and how far we've come it's such an exciting time to be in machine learning because there's so many automated tools let's see before we do this let's do a quick length of and let's do y we want let's just do length of y and we get 768 and if we import math we do math dot square root let's do y train there we go it's actually supposed to be x train before we do this let's go ahead and do import math and do math square root length of y test and when i run that we get 12.409 i want to show you where this number comes from we're about to use 12 is an even number so if you know if you're ever voting on things remember the neighbors all vote don't want to have an even number of neighbors voting so we want to do something odd and let's just take one away we'll make it 11. let me delete this out of here this one the reasons i love jupiter notebook is you can flip around and do all kinds of things on the fly so we'll go ahead and put in our classifier we're creating our classifier now and it's going to be the k neighbors classifier n neighbors equal 11. remember we did 12 minus 1 for 11. we have an odd number of neighbors p equals 2 because we're looking for is it are they diabetic or not and we're using the euclidean metric there are other means of measuring the distance you could do like square square means values all kinds of measure this but the euclidean is the most common one and it works quite well it's important to evaluate the model let's use the confusion matrix to do that and we're going to use the confusion matrix wonderful tool and then we'll jump into the f1 score and finally accuracy score which is probably the most commonly used quoted number when you go into a meeting or something like that so let's go ahead and paste that in there and we'll set the cm equal to confusion matrix y test y predict so those are the two values we're going to put in there and let me go ahead and run that and print it out and the way you interpret this is you have the y predicted which would be your title up here you could do let's just do p-r-e-d predicted across the top and actual going down actual it's always hard to to write in here actual that means that this column here down the middle that's the important column and it means that our prediction said 94 and prediction in the actual agreed on 94 and 32. this number here the 13 and the 15 those are what was wrong so you could have like three different if you're looking at this across three different variables instead of just two you'd end up with the third row down here and that column going down the middle so in the first case we have the the and i believe the zero is the 94 people who don't have diabetes the prediction said that 13 of those people did have diabetes and were at high risk and the 32 that had diabetes it had correct but our prediction said another 15 out of that 15 it classified as incorrect so you can see where that classification comes in and how that works on the confusion matrix then we're going to go ahead and print the f1 score let me just run that and you see we get a 0.69 in our f1 score the f1 takes into account both sides of the balance of false positives where if we go ahead and just do the accuracy account and that's what most people think of is it looks at just how many we got right out of how many we got wrong so a lot of people when you're data scientist and you're talking to other data scientists they're going to ask you what the f1 score the f score is if you're talking to the general public or the decision makers in the business they're going to ask what the accuracy is and the accuracy is always better than the f1 score but the f1 score is more telling it lets us know that there's more false positives than we would like on here but 82 percent not too bad for a quick flash look at people's different statistics and running an sk learn and running the knn the k nearest neighbor

on it so we have created a model using knn which can predict whether a person will have diabetes or not or at the very least whether they should go get a checkup and have their glucose checked regularly or not the print accuracy score we got the 0.818 was pretty close to what we got and we can pretty much round that off and just say we have an accuracy of 80 percent it tells us it is a pretty fair fit in the model what is k-means clustering k-means clustering is an unsupervised learning algorithm in this case you don't have labeled data unlike in supervised learning so you have a set of data and you want to group them and as the name suggests you want to put them into clusters which means objects that are similar in nature similar in characteristics need to be put together so that's what k means clustering is all about the term k is basically is a number so we need to tell the system how many clusters we need to perform so if k is equal to two there will be two clusters if k is equal to three three clusters and so on and so forth that's what the k stands for and of course there is a way of finding out what is the best or optimum value of k for a given data we will look at that so that is k means clustering so let's take an example k-means clustering is used in many many scenarios but let's take an example of cricket the game of cricket let's say you received data of a lot of players from maybe all over the country or all over the world and this data has information about the runs scored by the people ordered by the player and the wickets taken by the player and based on this information we need to cluster this data into two clusters batsman and bowlers so this is an interesting example let's see how we can perform this so we have the data which consists of primarily two characteristics which is the runs and the wickets so the bowlers basically take wickets and the batsmen score runs there will be of course a few bowlers who can score some runs and similarly there will be some batsmen who will who would have taken a few wickets but with this information we want to cluster those players into batsmen and bowlers so how does this work let's say this is how the data is so there are information there is information on the y-axis about the scored and on the x-axis about the wickets taken by the players so if we do a quick plot this is how it would look and when we do the clustering we need to have the clusters like shown in the third diagram rtms we need to have a cluster which consists of people who have scored high runs which is basically the batsman and then we need a cluster with people who have taken a lot of wickets which is typically the bowlers there may be a certain amount of overlap but we will not talk about it right now so with k-means clustering we will have here that means k is equal to two and we will have two clusters which is batsman and bowlers so how does this work the way it works is the first step in k-means clustering is the allocation of two centroids randomly so two points are assigned as so-called centroids so in this case we want two clusters which means k is equal to two so two points have been randomly assigned as centroids keep in mind these points can be anywhere there are random points they are not initially they are not really the centroids centroid means it's a central point of a given data set but in this case when it starts off it's not really the centroid okay so these points though in our presentation here we have shown them one point closer to these data points and another closer to these data points they can be assigned randomly anywhere okay so that's the first step the next step is to determine the distance of each of the data points from each of the randomly assigned centroids so for example we take this point and find the distance from this centroid and the distance from this center right this point is taken and the distance is found from this centroid and the center and so on and so forth so for every point the distance is measured from both the centroids and then whichever distance is less that point is assigned to that centroid so for example in this case visually it is very obvious that all these data points are assigned to this centroid and all these data points are assigned to this centroid and that's what is represented here in blue color and in this yellow color the next step is to actually determine the central point or the actual centroid for these two clusters so we have this one initial cluster this one initial

cluster but as you can see these points are not really the centroid centroid means it should be the central position of this data set central position of this data set so that is what needs to be determined as the next step so the central point of the actual centroid is determined and the original randomly allocated centroid is repositioned to the actual centroid of these new clusters and this process is actually repeated now what might happen is some of these points may get reallocated in our example that is not happening probably but it may so happen that the distance is found between each of these data points once again with these centroids and if there is if it is required some points may be reallocated we will see that in a later example but for now we will keep it simple so this process is continued till the centroid repositioning stops and that is our final cluster so this is our so after iteration we come to this position this situation where the centroid doesn't need any more repositioning and that means our algorithm has converged convergence has occurred and we have the cluster two clusters we have the clusters with a centroid so this process is repeated the process of calculating the distance and repositioning the centroid is repeated till the repositioning stops which means that the algorithm has converged and we have the final cluster with the data points and the centroids so this is what you're going to learn from this session we will talk about the types of clustering what is k-means clustering application of k-means clustering k-means clustering is done using distance measure so we will talk about the common distance measures and then we will talk about how k-means clustering works and go into the details of k-means clustering algorithm and then we will end with a demo and a use case for k-means clustering so let's begin first of all what are the types of clustering there are primarily two categories of clustering hierarchical clustering and then partitional clustering and each of these categories are further subdivided into agglomerative and divisive clustering and k-means and fuzzy c means clustering let's take a quick look at what each of these types of clustering are in hierarchical clustering the clusters have a tree-like structure and hierarchical clustering is further divided into agglomerative and divisive agglomerative clustering is a bottom-up approach we begin with each element as a separate cluster and merge them into successively larger clusters so for example we have a b c d e f we start by combining b and c form one cluster d e and e form one more then we combine d e and f one more bigger cluster and then add bc to that and then finally a to it compared to that divisive clustering or divisive clustering is the top-down approach we begin with the whole set and proceed to divide it into successively smaller clusters so we have abcdef we first take that as a single cluster and then break it down into a b c d e f then we have partitional clustering split into two subtypes k means clustering and fuzzy c means in k means clustering the objects are divided into the number of clusters mentioned by the number k that's where the k comes from so if we say k is equal to 2 the objects are divided into two clusters c1 and c2 and the way it is done is the features or characteristics are compared and all objects having similar characteristics are clubbed together so that's how k means clustering is done we will see it in more detail as we move forward and fuzzy c means is very similar to k means in the sense that it clubs objects that have similar characteristics together but while in k means clustering two objects cannot belong to or any object a single object cannot belong to two different clusters in c means objects can belong to more than one cluster so that is the primary difference between k-means and fuzzy c means so what are some of the applications of k-means clustering k-means clustering is used in a variety of examples or variety of business cases in real life starting from academic performance diagnostic systems search engines and wireless sensor networks and many more so let us take a little deeper look at each of these examples academic performance so based on the scores of the students students are categorized into a b c and so on clustering forms a backbone of search engines when a search is performed the search results need to be grouped together the search engines very often use clustering to do this and

similarly in case of wireless sensor networks the clustering algorithm plays the role of finding the cluster heads which collects all the data in its respective cluster so clustering especially k means clustering uses distance measure so let's take a look at what is distance measure so while these are the different types of clustering in this video we will focus on k-means clustering so distance measure tells how similar some objects are so the similarity is measured using what is known as distance measure and what are the various types of distance measures there is euclidean distance there is manhattan distance then we have squared euclidean distance measure and cosine distance measure these are some of the distance measures supported by k-means clustering let's take a look at each of these what is euclidean distance measure this is nothing but the distance between two points so we have learnt in high school how to find the distance between two points this is a little sophisticated formula for that but we know a simpler one is square root of y2 minus y1 whole square plus x2 minus x1 whole square so this is an extension of this formula so that is the euclidean distance between two points what is the squared euclidean distance measure it's nothing but the square of the euclidean distance as the name suggests so instead of taking the square root we leave the square as it is and then we have manhattan distance measure in case of manhattan distance it is the sum of the distances across the x-axis and the y-axis and note that we are taking the absolute value so that the negative values don't come into play so that is the manhattan distance measure then we have cosine distance measure in this case we take the angle between the two vectors formed by joining the points from the origin so that is the cosine distance measure okay so that was a quick overview about the various distance measures that are supported by k-means now let's go and check how exactly k-means clustering works okay so this is how k-means clustering works this is like a flowchart of the whole process there is a starting point and then we specify the number of clusters that we want now there are a couple of ways of doing this we can do by trial and error so we specify a certain number maybe k is equal to 3 or 4 or 5 to start with and then as we progress we keep changing until we get the best clusters or there is a technique called elbow technique whereby we can determine the value of k what should be the best value of k how many clusters should be formed so once we have the value of k we specify that and then the system will assign that many centroids so it picks randomly that to start with randomly that many points that are considered to be the centroids of these clusters and then it measures the distance of each of the data points from these centroids and assigns those points to the corresponding centroid from which the distance is minimum so each data point will be assigned to the centroid which is closest to it and thereby we have k number of initial clusters however this is not the final clusters the next step it does is for the new groups for the clusters that have been formed it calculates the mean position thereby calculates the new centroid position the position of the centroid moves compared to the randomly allocated one so it's an iterative process once again the distance of each point is measured from this new centroid point and if required the data points are reallocated to the new centroids and the mean position or the new centroid is calculated once again if the centroid moves then the iteration continues which means the convergence has not happened the clustering has not converged so as long as there is a movement of the centroid this iteration keeps happening but once the centroid stops moving which means that the cluster has converged or the clustering process has converged that will be the end result so now we have the final position of the centroid and the data points are allocated accordingly to the closest centroid i know it's a little difficult to understand from this simple flowchart so let's do a little bit of visualization and see if we can explain it better let's take an example if we have a data set for a grocery shop so let's say we have a data set for a grocery shop and now we want to find out how many clusters this has to be spread across so how do we find the optimum number of clusters there is a technique called the elbow method so when

these clusters are formed there is a parameter called within sum of squares and the lower this value is the better the cluster is that means all these points are very close to each other so we use this within sum of squares as a measure to find the optimum number of clusters that can be formed for a given data set so we create clusters or we let the system create clusters of a variety of numbers maybe of 10 10 clusters and for each value of k the within ss is measured and the value of k which has the least amount of within ss or wss that is taken as the optimum value of k so this is the diagrammatic representation so we have on the y-axis the within sum of squares or wss and on the x-axis we have the number of clusters so as you can imagine if you have k is equal to 1 which means all the data points are in a single cluster the within s value will be very high because they are probably scattered all over the moment you split it into two there will be a drastic fall in the within ss value that's what is represented here but then as the value of k increases the decrease the rate of decrease will not be so high it will continue to decrease but probably the rate of decrease will not be high so that gives us an idea so from here we get an idea for example the optimum value of k should be either 2 or 3 or at the most 4 but beyond that increasing the number of clusters is not dramatically changing the value in wss because that pretty much gets stabilized okay now that we have got the value of k and let's assume that these are our delivery points the next step is basically to assign two centroids randomly so let's say c1 and c2 are the centroids assigned randomly now the distance of each location from the centroid is measured and each point is assigned to the centroid which is closest to it so for example these points are very obvious that these are closest to c1 whereas this point is far away from c2 so these points will be assigned which are close to c1 will be assigned to c1 and these points or locations which are close to c2 will be assigned to c2 and then so this is the how the initial grouping is done this is part of c1 and this is part of c2 then the next step is to calculate the actual centroid of this data because remember c1 and c2 are not the centroids they have been randomly assigned points and only thing that has been done was the data points which are closest to them have been assigned but now in this step the actual centroid will be calculated which may be for each of these data sets somewhere in the middle so that's like the main point that will be calculated and the centroid will actually be positioned or repositioned there same with c2 so the new centroid for this group is c2 in this new position and c1 is in this new position once again the distance of each of the data points is calculated from these centroids now remember it's not necessary that the distance still remains the or each of these data points still remain in the same group by recalculating the distance it may be possible that some points get reallocated like so you see this so this point earlier was closer to c2 because c2 was here but after recalculating repositioning it is observed that this is closer to c1 than c2 so this is the new grouping so some points will be reassigned and again the centroid will be calculated and if the centroid doesn't change so that is the repetitive process iterative process and if the centroid doesn't change once the centroid stops changing that means the algorithm has converged and this is our final cluster with this as the centroid c1 and c2 as the centroids these data points as a part of each cluster so i hope this helps in understanding the whole process iterative process of k-means clustering so let's take a look at the k-means clustering algorithm let's say we have x1 x2 x3 n number of points as our inputs and we want to split this into k clusters or we want to create k clusters so the first step is to randomly pick k points and call them centroids they are not real centroids because centroid is supposed to be a central point but they are just called centroids and we calculate the distance of each and every input point from each of the centroids so the distance of x1 from c1 from c2 c3 each of the distances we calculate and then find out which distance is the lowest and assign x1 to that particular random centroid repeat that process for x2 calculate its distance from each of the centroids c1 c2 c3 up to ck and find which is the lowest distance and assign x2 to that particular centroid

same with x3 and so on so that is the first round of assignment that is done now we have k groups because there are we have assigned the value of k so there are k centroids and so there are k groups all these inputs have been split into k groups however remember we picked the centroids randomly so they are not real centroids so now what we have to do we have to calculate the actual centroids for each of these groups which is like the mean position which means that the position of the randomly selected centroids will now change and they will be the main positions of these newly formed k groups and once that is done we once again repeat this process of calculating the distance right so this is what we are doing as a part of step 4 we repeat step two and three so we again calculate the distance of x one from the centroid c one c two c three and then c which is the lowest value and assign x x1 to that calculate the distance of x2 from c1 c2 c3 or whatever up to ck and find whichever is the lowest distance and assign x2 to that centroid and so on in this process there may be some reassignment x1 pro was probably assigned to cluster c2 and after doing this calculation maybe now x1 is assigned to c1 so that kind of reallocation may happen so we repeat the steps 2 and 3 till the position of the centroids don't change or stop changing and that's when we have convergence so let's take a detailed look at it at each of these steps so we randomly pick k cluster centers we call them centroids because they are not initially they are not really the centroids so we let us name them c1 c2 up to ck and then step two we assign each data point to the closest center so what we do we calculate the distance of each x value from each c value so the distance between x1 c1 distance between x1 c2 x1 c3 and then we find which is the lowest value right that's the minimum value we find and assign x1 to that particular centroid then we go next to x2 find the distance of x2 from c1 x2 from c2 x2 from c3 and so on up to ck and then assign it to the point or to the centroid which has the lowest value and so on so that is step number two in step number three we now find the actual centroid for each group so what has happened as a part of step number two we now have all the points all the data points grouped into k groups because we we wanted to create k clusters right so we have k groups each one may be having a certain number of input values they need not be equally distributed by the way based on the distance we will have k groups but remember the initial values of the c1 c2 were not really the centroids of these groups right we assigned them randomly so now in step 3 we actually calculate the centroid of each group which means the original point which we thought was the centroid will shift to the new position which is the actual centroid for each of these groups okay and we again calculate the distance so we go back to step two which is what we calculate again the distance of each of these points from the newly positioned centroids and if required we reassign these points to the new centroids so as i said earlier there may be a real occasion so we now have a new set or a new group we still have k groups but the number of items and the actual assignment may be different from what was in step two here okay so that might change then we perform step three once again to find the new centroid of this new group so we have again a new set of clusters new centroids and new assignments we repeat this step two again once again we find and then it is possible that after iterating through three or four or five times the centroid will stop moving in the sense that when you calculate the new value of the centroid that will be same as the original value or there will be very marginal change so that is when we say convergence has occurred and that is our final cluster that's the formation of the final cluster all right so let's see a couple of demos of k-means clustering we will actually see some live demos in python notebook using pattern notebook but before that let's find out what's the problem that we are trying to solve the problem statement is let's say walmart wants to open a chain of stores across the state of florida and it wants to find the optimal store locations now the issue here is if they open too many stores close to each other obviously the they will not make profit but if they if the stores are too far apart then they will not have enough sales so how do they optimize this now for an

organization like walmart which is an e-commerce giant they already have the addresses of their customers in their database so they can actually use this information or this data and use k-means clustering to find the optimal location now before we go into the python notebook and show you the live code i wanted to take you through very quickly a summary of the code in the slides and then we will go into the python notebook so in this block we are basically importing all the required libraries like numpy matplotlib and so on and we are loading the data that is available in the form of let's say the addresses for simplicity sake we will just take them as some data points then the next thing we do is quickly do a scatter plot to see how they are related to each other with respect to each other so in the scatter plot we see that there are a few distinct groups already being formed so you can actually get an idea about how the cluster would look and how many clusters what is the optimal number of clusters and then starts the actual k-means clustering process so we will assign each of these points to the centroids and then check whether they are the optimal distance which is the shortest distance and assign each of the points data points to the centroids and then go through this iterative process till the whole process converges and finally we get an output like this so we have four distinct clusters and which is if we can say that this is how the population is probably distributed across florida state and the centroids are like the location where the store should be the optimum location where the store should be so that's the way we determine the best locations for the store and that's how we can help walmart find the best locations for the stores in florida so now let's take this into python notebook let's see how this looks when we are learning running the code live all right so this is the code for k-means clustering in jupiter notebook we have a few examples here which we will demonstrate how k-means clustering is used and even there is a small implementation of k-means clustering as well okay so let's get started okay so this block is basically importing the various libraries that are required like matplotlib and numpy and so on and so forth which would be used as a part of the code then we are going and creating blobs which are similar to clusters now this is a very neat feature which is available in scikit scikit-learn make blobs is a nice feature which creates clusters of data sets so that's a wonderful functionality that is readily available for us to create some test data kind of thing okay so that's exactly what we are doing here we are using make blobs and we can specify how many clusters we want so centers we are mentioning here so it will go ahead and so we just mentioned four so it will go ahead and create some test data for us and this is how it looks as you can see visually also we can figure out that there are four distinct classes or clusters in this data set and that is what make blobs actually provides now from here onwards we will basically run the standard k-means functionality that is readily available so we really don't have to implement k-means itself the k-means functionality or the function is readily available you just need to feed the data and we'll create the clusters so this is the code for that we import k-means and then we create an instance of k means and we specify the value of k this n underscore clusters is the value of k remember k means in k means k is basically the number of clusters that you want to create and it is a integer value so this is where we are specifying that so we have k is equal to four and so that instance is created we take that instance and as with any other machine learning functionality fit is what we use the function or the method rather fit is what we use to train the model here there is no real training kind of thing but that's the call okay so we are calling fit and what we are doing here we are just passing the data so x has these values the data that has been created right so that is what we are passing here and this will go ahead and create the clusters and then we are using after doing uh fit we run the predict which uh basically assigns for each of these observations which cluster where it belongs to all right so it will name the clusters maybe this is cluster 1 this is 2 3 and so on or i will actually start from 0 cluster 0 1 2 and 3 maybe and then for each of the observations it will assign based on which cluster it

belongs to it will assign a value so that is stored in y underscore k means when we call predict that is what it does and we can take a quick look at these y underscore k means are with the cluster numbers that have been assigned for each observation so this is the cluster number assigned for observation 1 maybe this is for observation 2 observation 3 and so on so we have how many about i think 300 samples right so all the 300 samples there are 300 values here each of them the cluster number is given and the cluster number goes from 0 to 3 so there are four clusters so the numbers go from 0 1 2 3. so that's what is seen here okay now so this was a quick example of generating some dummy data and then clustering that okay and this can be applied if you have proper data you can just load it up into x for example here and then run the cable so this is the central part of the k-means clustering program example so you basically create an instance and you mention how many customers you want by specifying this parameter and underscore clusters and that is also the value of k and then pass the data to get the values now the next section of this code is the implementation of a k-means now this is kind of a rough implementation of the k-means algorithm so we will just walk you through i will walk you through the code at each step what it is doing and then we will see a couple of more examples of how k-means clustering can be used in maybe some real life examples real life use cases all right so in this case here what we are doing is basically implementing k-means clustering and there is a function or a library calculates for a given two pairs of points it will calculate the the distance between them and see which one is the closest and so on so this is like this is pretty much like what k means does right so it calculates the distance of each point or each data set from predefined centroid and then based on whichever is the lowest this particular data point is assigned to that centroid so that is basically available as a standard function and we will be using that here so as explained in the slides the first step that is done in case of k-means clustering is to randomly assign some centroids so as a first step we randomly allocate a couple of centroids which we call here we are calling as centers and then we put this in a loop and we take it through an iterative process for each of the data points we first find out using this function pairwise distance argument for each of the points we find out which one which center or which randomly selected centroid is the closest and accordingly we assign that data or the data point to that particular centroid or cluster and once that is done for all the data points we calculate the new centroid by finding out the mean position which is the center position right so we calculate the new centroid and then we check if the new centroid is the coordinates or the position is the same as the previous centroid the positions we will compare and if it is the same that means the process has converged so remember we do this process still the centroids or the centroid doesn't move anymore right so the centroid gets relocated each time this reallocation is done so the moment it doesn't change anymore the position of the centroid doesn't change anymore we know that convergence has occurred so till then so you see here this is like an infinite loop while true is an infinite loop it only breaks when the centers are the same the new center and the old center positions are the same and once that is done we return the centers and the labels now of course as explained this is not a very sophisticated and advanced implementation very basic implementation because one of the flaws in this is that sometimes what happens is the centroid the position will keep moving but in the change will be very minor so in that case also with that is actually convergence right so for example the change is point zero zero zero one we can consider that as convergence otherwise what will happen is this will either take forever or it will be never ending so that's a small flaw here so that is something additional checks may have to be added here but again as mentioned this is not the most sophisticated implementation uh this is like a kind of a rough implementation of the k-means clustering okay so if we execute this code this is what we get as the output so this is the definition of this particular function and then we call that find underscore clusters and we pass

our data x and the number of clusters which is four and if we run that and plot it this is the output that we get so this is of course each cluster is represented by a different color so we have a cluster in green color yellow color and so on and so forth and these big points here these are the centroids this is the final position of the centroids and as you can see visually also this appears like a kind of a center of all these points here right similarly this is like the center of all these points here and so on so this is the example or this is an example of an implementation of k-means clustering and next we will move on to see a couple of examples of how k-means clustering is used in maybe some real-life scenarios or use cases in the next example or demo we are going to see how we can use k-means clustering to perform color compression we will take a couple of images so there will be two examples and we will try to use k-means clustering to compress the colors this is a common situation in image processing when you have an image with millions of colors but then you cannot render it on some devices which may not have enough memory uh so that is the scenario where where something like this can be used so before again we go into the python notebook let's take a look at quickly the the code as usual we import the libraries and then we import the image and then we will flatten it so the reshaping is basically we have the image information is stored in the form of pixels and if the image is like for example 427 by 640 and it has three colors so that's the overall dimension of the of the initial image we just reshape it and then feed this to our algorithm and this will then create clusters of only 16 clusters so this this colors there are millions of colors and now we need to bring it down to 16 colors so we use k is equal to 16 and this is how when we visualize this is how it looks there are these are all about 16 million possible colors the input color space has 16 million possible colors and we just sub compress it to 16 colors so this is how it would look when we compress it to 16 colors and this is how the original image looks and after compression to 16 colors this is how the new image looks as you can see there is not a lot of information that has been lost though the image quality is definitely reduced a little bit so this is an example which we are going to now see in python notebook let's go into the python node and once again as always we will import some libraries and load this image called flower dot jpg okay so let we load that and this is how it looks this is the original image which has i think 16 million colors and this is the shape of this image which is basically what is the shape is nothing but the overall size right so this is 427 pixel by 640 pixel and then there are three layers which is this three basically is for rgb which is red green blue so color image will have that right so that is the shape of this now what we need to do is data let's take a look at how data is looking so let me just create a new cell and show you what is in data basically we have captured this information so data is what let me just show you here all right so let's take a look at china what are the values in china and if we see here this is how the data is stored this is nothing but the pixel values okay so this is like a matrix and each one has about for this 427 by 640 pixels all right so this is how it looks now the issue here is these values are large the numbers are a large so we need to normalize them to between 0 and 1 right so that's why we will basically create one more variable which is data which will contain the values between 0 and 1 and the way to do that is divide by 255 so we divide china by 255 and we get the new values in data so let's just run this piece of code and this is the shape so we now have also yeah what we have done is we changed using reshape we converted into the three-dimensional into a two-dimensional data set and let us also take a look at how let me just insert uh probably a cell here and take a look at how data is looking all right so this is how data is looking and now you see this is the values are between zero and one right so if you earlier noticed in case of china the values were large numbers now everything is between zero and one this is one of the things we need to do all right so after that the next thing that we need to do is to visualize this and we can take random set of maybe 10 000 points and plot it and check and see how this looks let us just plot this and so this is how the original the

color the pixel distribution is these are two plots one is red against green and another is red against blue and this is the original distribution of the color so then what we will do is we will use k means clustering to create just 16 clusters for the various colors and then apply that to the image now what will happen is since the data is large because there are millions of colors using regular k means may be a little time consuming so there is another version of k-means which is called mini batch gaming so we will use that which is which processes in the overall concept remains the same but this basically processes it in smaller batches that's the only thing okay so the results will pretty much be the same so let's go ahead and execute this piece of code and also visualize this so that we can see that there are these this is how the 16 colors would look so this is red against green and this is red against blue there is uh quite a bit of similarity between this original color schema and the new one right so it doesn't look very very completely different or anything like that now we apply this the newly created colors to the image and we can take a look uh how this is looking now we can compare both the images so this is our original image and this is our new image so as you can see there is not a lot of information that has been lost it pretty much looks like the original image yes we can see that for example here there is a little bit it appears a little dullish compared to this one right because we kind of took off some of the finer details of the color but overall the high level information has been maintained at the same time the main advantage is that now this can be this is an image which can be rendered on a device which may not be that very sophisticated now let's take one more example with a different image in the second example we will take an image of the summer palace in china and we repeat the same process this is a high definition color image with millions of colors and also three dimensional now we will reduce that to 16 colors using k means clustering and we do the same process like before we reshape it and then we cluster the colors to 16 and then we render the image once again and we will see that the color the quality of the image slightly deteriorates as you can see here this has much finer details in this which are probably missing here but then that's the compromise because there are some devices which may not be able to handle this kind of high density images so let's run this chord in python notebook all right so let's apply the same technique for another picture which is even more intricate and has probably much complicated color schema so this is the image now once again we can take a look at the shape which is 427 by 640 by 3 and this is the new data would look somewhat like this compared to the flower image so we have some new values here and we will also bring this as you can see the numbers are much big so we will much bigger so we will now have to scale them down to values between 0 and 1 and that is done by dividing by 255 so let's go ahead and do that and reshape it okay so we get a two-dimensional matrix and we will then as a next step we will go ahead and visualize this how it looks does the 16 colors and this is basically how it would look 16 million colors and now we can create the clusters out of this the 16 k means clusters we will create so this is how the distribution of the pixels would look with 16 colors and then we go ahead and apply this and visualize how it is looking for with the new just the 16 color so once again as you can see this looks much richer in color but at the same time and this probably doesn't have as we can see it doesn't look as rich as this one but nevertheless the information is not lost the shape and all that stuff and this can be also rendered on a slightly a device which is probably not that sophisticated okay so that's pretty much it so we have seen two examples of how color compression can be done uh using k-means clustering and we have also seen in the previous examples of how to implement k-means the code to roughly how to implement k-means clustering and we use some sample data using blob to just execute the k-means clustering we're going to cover mathematics for machine learning so today's agenda is going to cover data and its types then we're going to dive into linear algebra and its concepts calculus statistics for machine learning probability for machine learning hands-on demos and of

course thrown in there in the middle is going to be your matrixes and a few other things to go along with all this data and as types data denotes the individual pieces of factual information collected from various sources it is stored processed and later used for analysis and so we see here just a huge grouping of information a lot of tech stuff money dollar signs numbers and then you have your performing analytics to drive insights and hopefully you have a nice share your shareholders gather it at the meeting and you're able to explain it in something they can understand so we talk about data types of data we have in our types of data we have a qualitative categorical you think nominal or ordinal and then you have your quantitative or numerical which is discrete or continuous and let's look a little closer at those data type vocabulary always people's favorite is the vocabulary words okay not mine but let's dive into this what we mean by nominal nominal they are used to label various uh label our variables without providing any measurable value country gender race hair color etc it's something that you either mark true or false this is a label it's on or off either they have a red hat on or they do not so a lot of times when you're thinking nominal data labels think of it as a true false kind of setup and we look at ordinal this is categorical data with a set order or a scale to it and you can think of salary range as a great one movie ratings etc you see here the salary range if you have ten thousand to twenty thousand number of employees earning that rate is a hundred and fifty twenty thousand to thirty thousand a hundred and so forth some of the terms you'll hear is bucket uh this is where you have ten different buckets and you to separate it into something that makes sense into those 10 buckets and so when we start talking about ordinal a lot of times when you get down to the brass bones again we're talking true false so if you're a member of the 10 to 20 k range so forth those would each be either part of that group or you're not but now we're talking about buckets and we want to count how many people are in that bucket quantitative numerical data falls into two classes discrete or continuous and so data with a final set of values which can be categorized class strength questions answered correctly and runs hit and cricket a lot of times when you see this you can think integer and a very restricted integer i you can only have a hundred questions um on a test so you can it's very discrete i only have a hundred different values that it can attain so think usually you're talking about integers but within a very small range they don't have an open end or anything like that so discrete is very solid simple to count set number continuous on the other hand continuous data can take any numerical value within a range so water pressure weight of a person etc usually we start thinking about float values where they can get phenomenally small in their in what they're worth and there's a whole series of values that falls right between discrete and continuous you can think of the stock market you have dollar amounts it's still discrete but it starts to get complicated enough when you have like you know jump in the stock market from 525.33 cents to five hundred and eighty dollars point sixty seven cents there's a lot of point values in there it'd still be called discrete but you start looking at it as almost continuous because it does have such a variance in it now we talked about no we did we went over nominal and ordinal almost true false charts and we looked at quantitative and numerical data which were starting to get into numbers discrete you can usually a lot of times discrete will be put into it could be put into true false but usually it's not so we want to address this stuff and the first thing you want to look at is the very basic which is your algebra so we're going to take a look at linear algebra you can remember back when your euclidean geometry we have a line well let's go through this we have a linear algebra is the domain of mathematics concerning linear equations and the representations in vector spaces and through matrixes i told you we're going to talk about matrixes so a linear equation is simply 2x plus 4y minus 3z equals 10. very linear 10x plus 12.4y equals z and now you can actually solve these two equations by combining them and this we're talking about a linear equation in the vectors we have a plus b equals c now we're starting to look at a direction and these values usually

think of an x y z plot so each one is a direction and the actual distance of like a triangle a b is c and then your matrix can describe all kinds of things i find matrixes confuse a lot of people not because they're particularly difficult but because of the magnitude and the different things they're used for and a matrix is a chart or a you know think of a spreadsheet but you have your rows and your columns and you'll see here we have a times b equals c very important to know your counts uh so depending on how the math is being done what you're using it for making sure you have the same rows and number of columns or a single number there's all kinds of things that play in that that can make matrixes confusing but really it has a lot more to do with what domain you're working in are you adding in multiple polynomials where you have like uh a x squared plus b y plus you know you start to see that can be very confusing versus a very straightforward matrix and let's just go a little deeper into these because these are such primary this is what we're here to talk about is these different math uh mathematical computations that come up so we're looking at linear equations let's dig deeper into that one an equation having a maximum order of one is called a linear equation so it's linear because when you look at this we have a x plus b equals c which is a one variable we have two variable a x plus b y equals c a x plus b y plus z c z equals d and so forth but all of these are to the power of one you don't see x squared you don't see x cubed so we're talking about linear equations that's what we're talking about in their addition if you have already dived into say neural networks you should recognize this a x plus b y plus c z setup plus the intercept which is basically your your neural network each node adding up all the different inputs and we can drill down into that most common formula is your y equals mx plus c so you have your y equals the m which is your slope your x value plus c which is your y-intercept you kind of label it wrong here threw me for a loop but the the c would be your y-intercept so when you set x equal to 0 y equals c and that's that's your y-intercept right there uh and that's they just had a reverse value of y when x equals zero equals the y-intercept which is c and your slow gradient line which is your m so it's your y equals two x plus three and there's lots of easy ways to compute this this way this is why we always start with the most basic one when we're solving one of these problems and then of course the one of the most important takeaways is the slope gradient of the line so the slope is very important that m value in this case we went ahead and solved this if you have y equals 2 x plus 3 you can see how it has a nice line graph here on the right so matrixes a matrix refers to a rectangular representation of an array of numbers arranged in columns and rows so we're talking m rows by n columns here a11 is denotes the element of the first row in the first column similarly a12 and it's really pronounced a11 in this particular setup so it's a row one column one a 12 is a row one column two uh first row and second column and so on and there's a lot of ways to denote this i've seen these as like a capital letter a smaller case a for the top row or i mean you can see where they can go all kinds of different directions as far as the value you just take a moment to realize there's need to be some designation as far as what row it's in and what column it's in and we have our basic operations we have addition so when you think about addition you have two matrixes of two by 2 and you just add each individual number in that matrix and then when you get to the bottom you have in this case the solution is 12 10 plus 2 is 12 5 plus 3 is 8 and so on and the same thing with subtraction now again you're counting matrixes you want to check your dimensions of the matrix the shape you'll see shape come up a lot in programming so we're talking about dimensions we're talking about the shape if the two shapes are equal this is what happens when you add them together or subtract them and we have multiplication when you look at the multiplication you end up with a very a slightly different setup going now if we look at our last one we're uh we're like why this always gets to me when we get to matrixes because they don't really say why you multiply matrixes you know my first thought is one times two four times three but if you look at this we get one times two plus four times three one times three

plus four times five uh six times two plus three times three six times three plus three times five if you're looking at these matrixes uh think of this more as an equation and so we have if you remember we went back up here for our multiple line equations let's just go back up a couple slides where we were looking at a two variable so this is a two variable equation ax plus b y equals c and this is a way to make it very quick to solve these variables and that's why you have the matrix and that's why you do the multiplication the way they do and this is the dot product of one times two plus four times three one times three plus four times five six times two plus three times three times 3 plus 3 times 5 and it gives us a nice little 14 23 21 and 33 over here which then can be used and reduced down to a sample formula as far as solving the variables as you have enough inputs and then in matrix operations when you're dealing with a lot of matrixes uh now keep in mind multiplying matrixes is different than finding the product of two matrixes okay so we're talking about multiplication we're talking about solving uh for equations when you're finding the product you are just finding 1 times 2. keep that in mind because that does come up i've had that come up a number of times where i'm altering data and i get confused as to what i'm doing with it transpose flipping the matrix over is diagonal comes up all the time where you have you still have 12 but instead of it being uh 12 8 it's now 12 14 8 21 you're just flipping the columns and the rows and then of course you can do an inverse changing the signs of the values across this main diagonal and you can see here we have the inverse a to the minus 1 and ends up with instead of 12 8 14 12 it's now minus 22 minus 12. vectors vector just means we have a value and a direction and we have down four numbers here on our vector in mathematics a one-dimensional matrix is called a vector so if you have your x plot and you have a single value that values along the x-axis and it's a single dimension if you have two dimensions you can think about putting them on a graph you might have x and you might have y and each value denotes a direction and then of course the actual distance is going to be the hypothesis of that triangle and you can do that with three dimensionals x y and z and you can do it all the way to nth dimensions so when they talk about the k means for categorizing and how close data is together they will compute that based on the pythagorean theorem so you would take the square of each value add them all together and find the square root and that gives you a distance as far as where that point is where that vector exists or an actual point value and then you can compare that point value to another one it makes a very easy comparison versus comparing 50 or 60 different numbers and that brings us up to i gene vectors and i gene values i gene vectors the vectors that don't change their span while transformation and igene values the scalar values that are associated to the vectors conceptually you can think of the vector as your picture you have a picture it's two dimensions x and y and so when you do those two dimensions and those two values or whatever that value is that is that point but the values change when you skew it and so if we take and we have a vector a and that's a set value uh b is um your is your you have a and b which is your hygiene vector two is the i gene value so we're altering all the values by two that means we're uh maybe we're stretching it out one direction making it tall if you're doing picture editing that's one of the places this comes in but you can see when you're transforming your different information how you transform it is then your hygiene value and you can see here vector after line transit transition we have 3a a is the hygiene vector 3 is the iogene value so a doesn't change that's whatever we started with that's your original picture and three is skewing it one direction and maybe uh b is being skewed another direction and so you have a nice tilted picture because you altered it by those by the aging values so let's go ahead and pull up a demo on linear algebra and to do this i'm going to go through my trusted anaconda into my jupiter notebook and we'll create a new notebook called linear algebra since we are working in python we're going to use our numpy i always import that as np or numpy array probably the most popular module for doing matrixes and

things in given that this is part of a series i'm not going to go too much into numpy we are going to go ahead and create two different variables a for a numpy array 10 15 and b 29 we'll go ahead and run this and you can see there's our two arrays 10 15 29 and went and added a space there in between so it's easier to read and since it's the last line we don't have to put the print statement on it unless you want we can simply but we can simply do a plus b so when i run this we have 10 15 29 and we get 30 24 which is what you expect 10 plus 20 15 plus 9 you could almost look at this addition as being just adding up the columns on here coming down and if we wanted to do it a different way we could also do a dot t plus b dot t remember that t flips them and so if we do that we now get them we now have 30 24 going the other way we could also do something kind of fun there's a lot of different ways to do this as far as a plus b i can also do a plus b dot t and you're going to see that that will come out the same the 30 24 whether i transpose a and b or transpose them both at the end and likewise we can very easily subtract two vectors i can go a minus b and we run that and we get minus 10 6. now remember this is the last line in this particular section that's why i don't have to put the print around it and just like we did before we can transpose either the individual or we can transpose the main set up and then we get a minus 10 6 going the other way now we didn't mention this in our notes but you can also do a scalar multiplication and just put down scalar so you can remember that uh what we're talking about here is i have this array here u and if i go a times u we'll take the value 2 we'll multiply it by every value in here so 2 times 30 is 60 2 times 15. and just like we did before this happens a lot because when you're doing matrixes you do need to flip them you get 60 30 coming this way so in numpy uh we have what they call dot product and uh with this this is in a two dimensional vectors it is equivalent of two matrix multiplication remember we were talking about matrix multiplication uh where it is the well let's walk through it we'll go ahead and start by defining two um numpy arrays we'll have uh 10 20 25 6 or our u and our v and then we're going to go ahead and do if we take the values and if you remember correctly an array like this would be 10 times 25 plus 20 times 6. we'll go ahead and print that there we go and then we'll go ahead and do the np dot dot of u comma v and we'll find when we do this we go and run this uh we're going to get 370 370. so this is a strain multiplication where they use it to solve linear algebra uh when you have multiple numbers going across and so this could be very complicated we could have a whole string of different variables going in here but for this we get a nice value for our dot multiplication and we did addition earlier which is just your basic addition and of course the matrix you can get very complicated on these or in this case we'll go ahead and do let's create two complex matrixes this one is a matrix of um you know 12 10 4 6 4 31. we'll just print out a so you can see what that looks like here's print a we print a out you can see that we have a 2 by 3 layer matrix for a and we can also put together always kind of fun when you're playing with print values we could do something like this we could go in here there we go we could print a we have it end with equals a run and this kind of gives it a nice look here's your matrix that's all this is comma n means it just tags it on the end that's all all that is doing on there and then we can simply add in what is a plus b and you should already guess because this is the same as what we did before there's no difference we do a simple vector addition we have 12 plus 2 is 14 10 plus 8 is 18 and so on and just like we did the matrix addition we can also do a minus b and do our matrix subtraction and we look at this uh we have what 12 minus 2 is 10 10 minus 8 um where are we oh there we go 8 minus confusing what i'm looking at i should have reprinted out the original numbers but we can see here 12 minus 2 is of course 10 10 minus 8 is 2 4 minus 46 is minus 42 and so forth so same as a subtraction as before we just call it matrix subtraction it's identical now if you remember up here we had a scalar addition we're adding just one number to a matrix you can also do scalar multiplication and so simply if you have a single value a and you have b which is your array we can also do a times b when we run that you can

see here we have 2 times 4 is 8. 5 times 4 is 20 and so forth you're just multiplying the 4 across each one of these values and this is an interesting one that comes up a little bit of a brain teaser is matrix and vector multiplication and so we're looking at this we are just doing regular arrays it doesn't necessarily have to be a numpy array we have a which has our array of arrays and b which is a single array and so we can from here do the dot a b and this is going to return two values and the first value is that is you could say it's like uh we're doing the this array b array first with a and then with a second one and so it splits it up so you have a matrix of vector multiplication and you can mix and match when you get into really complicated uh back end stuff this becomes more common because you're now you've got layers upon layers of data and so you'll end up with a matrix and a set of bolt vector matrices do you want to now keep in mind that if you're doing data science a lot of times you're not looking at this this is what's going on behind the scenes so if you're in the scikit looking at sk learn where you're doing linear regression models this is some of the math that's hidden behind the scenes that's going on other times you might find yourself having to do part of this and manipulate the data around so it fits right and then you go back in and you run it through the psi kit and if we can do up here where we did a matrix and vector multiplication we can also do matrix two matrix multiplication and if we run this where we have the two matrixes uh you can see a very complicated array that of course comes out on there for our dot and just to reiterate it we have our transpose matrix which is your dot t and so if we create a matrix a and we do transpose it you can see how it flips it from 5 10 15 20 25 30 to 5 15 25 10 20 30 rows and columns and certainly with the math this comes up a lot it also comes up a lot with xy plotting when you put into pi plot you have one format where they're looking at pairs and numbers and then they want all of x's and all y's so you know the transpose is an important tool both for your math and for plotting and all kinds of things another tool that we didn't discuss uh is your identity matrix and this one is more definition uh the identity matrix we have here one where we just did two so it comes down as one zero zero one one zero zero zero one zero it creates a diagonal of one and what that is is when you're doing your identities you could be comparing all your different features to the different features and how they correlate and of course when you have feature one compared to feature one to itself it is always one where usually it's between zero one depending on how well correlates so when we're talking about identity matrix that's what we're talking about right here is that you create this preset matrix and then you might adjust these numbers depending on what you're working with and what the domain is and then another thing we can do to kind of wrap this up we'll hit you with the most complicated uh piece of this puzzle here is an inverse a matrix and let's just go ahead and put the um it's a lengthy description let's go and put the description this is straight out of the uh the website for numpy so given a square matrix a here's our square matrix a which is 2 1 0 0 1 0 1 2 1. keep in mind 3 by 3 is square it's got to be equal it's going to return the matrix a inverse satisfying dot a a inverse so here's our matrix multiplication um and then of course it equals the dot yeah a inverse of a with an identity shape of a dot shaped 0. this is just reshaping the identity that's a little complicated there uh so we're going to have our here's our array we'll go ahead and run this and you can see what we end up with is we end up with an array 0.5 minus 0.5 and so forth with our 2 1 1 going down 2 1 0 0 1 0 1 2 1. getting into a little deep on the math understanding when you need this is probably really is is what's really important when you're doing data science versus uh handwriting this out and looking up the math and handwriting all the pieces out you do need to know about the linear algorithm inverse of a so if it comes up you can easily pull it up or at least remember where to look it up you took a look at the algebra side of it let's go ahead and take a look at the calculus side of what's going on here with the machine learning so calculus oh my goodness and differential equations you got to throw that in there because that's all part of the bag of tricks especially when you're doing

large neural networks but also comes up in many other areas the good news is most of it's already done for you in the back end so when it comes up you really do need to understand from the data science not data analytics data analytics means you're digging deep into actually solving these math equations and a neural network is just a giant differential equation so we talk about calculus we're going to go ahead and understand it by talking about cars versus time and speed so helps to calculate the spontaneous rate of change so suppose we plot a graph of the speed of a car with respect to time so as you can see here going down the highway probably merged into the highway from an on-ramp so i had to accelerate so my speed went way up stuck in traffic merged into the traffic traffic opens up and i accelerate again up to the speed limit and maybe peter's off up there so you can look at this as as the speed versus time i'm getting faster and faster because i'm continually accelerating and if i hit the brakes it go the other way so the rate of change of speed with respect of time is nothing but acceleration how fast are we accelerating the acceleration is the area between the star point of x and the end point of delta x so we can calculate a simple if you had x and delta x we could put a line there and that slope of the line is our acceleration now that's pretty easy when you're doing linear algebra but i don't want to know it just for that line in those two points i want to know what across the whole of what i'm working with that's where we get into calculus so we talk about the distance between x and delta x it has to be the smallest possible near to zero in order to approximate the acceleration uh so the idea is instead of i mean if you ever did took a basic calculus class they would draw bars down here and you would divide this area up let's go back up the screen you divide this area of this time period up into maybe 10 sections and you'd use that and you could calculate the acceleration between each one of those 10 sections kind of thing and then we just keep making that space smaller and smaller until delta x is almost infinitesimally small and so we get a function of a equals a limit as h goes to 0 of a function of a plus h minus a function of a over h and that is you're computing the slope of the line we're just computing that slope under smaller and smaller and smaller samples and that's what calculus is calculus is the integral you can see down here we have our nice uh integral sign looks like a giant s and that's what that means is that we've taken this down to as small as we can for that sampling uh so we're talking about calculus we're finding the area under the slope is the main process in the integration similar small intervals are made of the smallest possible length of x plus delta x where delta x approaches almost an infinitesimally small space and then it helps to find the overall acceleration by summing up all the links together uh so we're summing up all the accelerations from the beginning to the end and so here's our integral we sum of a of x times d of x equals a plus c that is our basic calculus here so when we talk about multivariate calculus multivariate calculus deals with functions that have multiple variables and you can see here we start getting into some very complicated equations um changing w over change of time equals change of w over change of z the differential of z to dx differential of x to dt it gets pretty complicated uh it really translates into the multivariate integration using double integrals and so you have the the sum of the sum of f of x y of d of a equals the sum from c to d and a to b of f of x y d x d y equals the sum of a to b sum of c to d of f x of y d y d x understanding the very specifics of everything going on in here and actually doing the math is usually calculus 1 calculus 2 and differential equations so you're talking about three full-length courses to dig into and solve these math equations what we want to take from here is we're talking about calculus uh we're talking about summing of all these different slopes and so we're still solving a linear expression we're still solving y equals m x plus b but we're doing this for infinitely small x's and then we want to sum them up that's what this integral sign means the sum of a of x d of x equals a plus c and when you see these very complicated uh multivariate differentiation using the chain rule uh when we come in here and we have the change of w to the change of t equals the change of w

dz uh and so forth that's what's going on here that's what these means we're basically looking for the area under the curve which really comes to how is the change changing speeds going up how is that changing and then you end up with a multiple layer so if i have three layers of neural networks how is the third layer changing based on the second layer changing which is based on the first layer changing and you get the picture here that now we have a very complicated uh multivariate integration with integrals the good news is we can solve this uh mathematically and that's what we do when you do neural networks and reverse propagation uh so the nice thing is that you don't have to solve this on paper unless you're a data analysis and you're working on the back end of integrating these formulas and building the script to actually build them so we talk about applications of calculus it provides us the tools to build an accurate predictive model so it's really behind the scenes we want to guess at what the change of the change or the change is that's a little goofy i i know i just threw that out there it's kind of a meta term but if you can guess how things are going to change then you can guess what the new numbers are multivariate calculus explains the change in our target variable in relation to the rate of change in the input variables so there's our multiple variables going in there if one variable is changing how does it affect the other variable and then in gradient descent calculus is used to find the local and global maxima and this is really big uh we're going to actually going to have a whole section here on gradient descent because it is really i mean i talked about neural networks and how you can see how the different layers go in there but gradient descent is one of the most key things for trying to guess the best answer to something so let's take a look at the code behind gradient descent and before we open up the code let's just do real quick gradient descent let's say we have a curve like this and most common is that this is going to represent your error oops error there we go error ah hard to read there and i want to make the error as low as possible and so what i'm looking at it is i want to find this line here which is the minimum value so we're looking for the minimum and it does that by sampling there and then based on this it guesses it might be someplace here and it goes hey this is still going down it goes here and then goes back over here and then goes a little bit closer and it's just playing a high low until it gets to that spot that bottom spot and so we want to minimize the error and on the flip note you could also want to be maximizing something you want to get the best output of it that's simply minus the value so if you're looking for where the peak is this is the same as a negative for where the valley is i'm looking for that valley uh that's all that is this is a way of finding it so the cool thing is um all the heavy lifting's done um i actually ended up putting together one of these a while back is when i didn't know about sidekick and i was just starting boys a long while back and is playing high low how do you play high low not get stuck in the valleys figure out these curves and things like that well you do that and the back end is all the calculus and differential equations to calculate this out the good news is you don't have to do those uh so instead we're going to put together the code and let's go ahead and see what we can do with that so uh guys in the back put together a nice little piece of code here which is kind of fun uh some things we're going to note and this is this is really important stuff because when you start doing your data science and digging into your machine learning models you're going to find these things are stumbling blocks the first one is current x where do we start at keep in mind your model that you're working with is very generic so whatever you use to minimize it the first question is where do we start and we started at this because the algorithm starts at x equals three so we arbitrarily picked five learning rate is uh how many bars to skip going one where the other in fact i'm going to separate that a little bit because these two are really important if we're dealing with something like this where we're talking about well here's our here's the function we're going to use our gradient of our function 2 times x plus 5 keep it simple so that's the function we're going to work with so if i'm dealing with increments of 1000.1 is going to be a very long time and if i'm dealing with

increments of 0.001 0.1 is going to skip over my answer so i won't get a very good answer and then we look at precision this tells us when to stop the algorithm so again very specific to what you're working on if you're working with money and you don't convert it into a float value you might be dealing with 0.01 which is a penny that might be your precision you're working with and then of course the previous step size max iterations we want something to cut out at a certain point usually that's built into a lot of minimization functions and then here's our actual formula we're going to be working with and then we come in we go while previous step size is greater than precision and iters is less than max and max iters let's say that 10 times fast um we're just saying if it's uh if we're if we're still greater than our precision level we still got to keep digging deeper and then we also don't want to go past a thou or whatever this is a million or ten thousand uh running that's actually pretty high we almost never do max iterations more than like a hundred or two 200 rare occasions you might go up to four or five hundred if it's depending on the problem you're working with uh so we have our previous equals our current that way we can track time wise uh the current now equals the current minus the rate times the formula of our previous x so now we've generated our new version previous step size equals the absolute current previous so we're looking for the change in x errors equals iterations plus one that's so we know to stop if we get too far and then we're just going to print the local minimum occurs at x on here and if we go ahead and run this you can see right here it gets down to this point and it says hey local minimum is minus 3.322 for this particular series we created this is created off of our formula here lambda x2 times x plus 5. now when i'm running this stuff you'll see this come up a lot in with the sk learn kit and one of the nice reasons of breaking this down the way we did is i could go over those top pieces uh those top pieces are everything when you start looking at these minimization tool kits in built-in code and so from we'll just do it's actually docs dot scipy.org and we're looking at the psi kit there we go optimize minimize you can only minimize one value you have the function that's going in this function can be very complicated so we used a very simple function up here it could be there's all kinds of things that could be on there and there's a number of methods to solve this as far as how they shrink down and your x naught there's your there's your start value so your function your start value there's all kinds of things that come in here that you can look at which we're not going to optimization automatically creates constraints bounds some of this it does automatically but you really the big thing i want to point out here is you need to have a starting point you want to start with something that you already know is mostly the answer if you don't then it's going to have a heck of a time trying to calculate it out or you can write your own little script that does this and does a high low guessing and tries to find the max value that brings us to statistics what this is kind of all about is figuring things out a lot of vocabulary and statistics ah so statistics well i guess it's all relative it's definitely not an edel class so a bunch of stuff going on statistics statistics concerns with the collection organization analysis interpretation and presentation of data that is a mouthful so we have from end to end where does it come from is it valid what does it mean how do we organize it how do we analyze it then you gotta take those analysis and interpret it into something that people can use kind of reduce it to understandable and nowadays you have to be able to present it if you can't present it then no one else is going to understand what the heck you did so we look at the terminologies there is a lot of terminologies depending on what domain you're working in so clearly if you're working in um a domain that deals with viruses and t cells and and how does you know where does that come from you're studying the different people then you can have a population if you are working with mechanical gear you know a little bit different if you're looking for the wobbling statistics to know when to replace a rotor on a machine or something like that that can be a big deal you know we have these huge fans that turn in our sewage processing systems and so those fans they start to wobble and hum and do different

things that the sensors pick up at one point do you replace them instead of waiting for it to break in which case it costs a lot of money instead of replacing a bushing you're replacing the whole fan unit an interesting project that came up for our city a while back so population all objects are measurements whose properties are being observed uh so that's your population all the objects it's easy to see it with people because we have our population in large but in the case of the sewer fans we're talking about how the fan units that's the population of fans that we're working with you have a parameter a matrix that is used to represent a population or characteristic you have your sample a subset of the population studied you don't want to do them all because then you don't have a if you come up with a conclusion for everyone you don't have a way of testing it so you take a sample sometimes you don't have a choice you can only take a sample of what's going on you can't study the whole population and a variable a metric of interest for each person or object in a population types of sampling we have a probabilistic approach selecting samples from a larger population using a method based on the theory of probability and we'll go into a little bit more deeper on these we have random systematic stratified and then you have a non-probabilistic approach selecting samples based on the subjective judgment of the researcher rather than random selection it has to do with convenience trying to reach a quota or snowball and they're very biased that's one of the reasons you'll see this big stamp on it says biased so you've got to be very careful on that so probabilistic sampling uh when we talk about a random sampling we select random size samples from each group or category so we it's as random as you can get we talk about systematic sampling we're selecting random size samples from each group or category with a fixed periodic interval uh so we kind of split it up this would be like a time set up or our different categories and you might ask your question what is a category or a group if you look at i'm going to go back a window let's say we're studying economics of different of an area we know pretty much that based on their culture where they came from they might need to be separated and so uh and when i say separated i don't mean separated from their place where they live i mean as far as the analysis we want to look at the different groups and make sure they're all represented so if we had like an eighty percent uh of a group that is uh say hispanic and or indian and also in that same area we have twenty percent twenty percent who are uh let's call our expatriates they left america and they're nice and your caucasian group we might want to sample a group that is representative of both so we're talking about stratified sampling and we're talking about groups those are the groups we're talking about and that brings us to stratified sampling selecting approximately equal size samples from each group or category this way we can actually separate the categories and give us an insight into the different cultures and how that might affect them in that area so you can see these are very very different kind of depends on what you're working with as far as your data and what you're studying and so we can see here just a little bit more we'd have selecting 25 employees from a company of 250 employees randomly don't care anything about them what groups are in which office they're in nothing and we might be selecting one employee from every 50 unique employees in a company of 250 employees and then we have selecting one employee from every branch in the company office so we have all the different branches there's our group or categories by the branch and the category could depend on what you're studying so it has a lot of variation on there you see this kind of grouping and categorizing is also used to generate a lot of misinformation so if you only study one group and you say this is what it is then everybody assumes that's what it is for everybody and so you've got to be very careful of that it's very unethical thing to kind of do so types of statistics uh we talk about statistics we're going to talk about descriptive and inferential statistics there are so many different terms and statistics to break it up so we so we're talking about a particular setup so we're talking about descriptive and inferential uh statistics the base of the word describe is pretty solid you're describing the data

what does it look like with inferential statistics we're going to take that from the small population to a large population so if you're working with a drug company you might look at the data and say these people were helped by this drug they did 80 better as far as their health or 80 percent better survival rate than the people um who did not have the drug so we can infer that that drug will work in the greater populace and will help people so that's where you get your inferential so we are predicting how it's going to affect the greater population so descriptive statistics it is used to describe the basic features of data and form the basis of quantitative analysis of data so we have a measure of central tendencies we have your mean median and mode and then we have a measure of spread like your range your interquartile range your variance and your standard deviation and we're going to look at all these a little deeper here in a second but one of them you can think of is how it the data difference differences you know what's the max min range all that stuff is your spread and anything that's just a single number is usually your central uh tendencies measure of central tendencies so we talk about the mean it is the average of the set of values considered what is the average outcome of whatever's going on and then your median separates the higher half and the lower half of data so where's the center point of all your different data points so your mean might have some a couple really big numbers that skew it so that the average is much higher than if you took those outliers out where the median would by separating the high from the low might give you a much lower number you might look at and say oh that's that's odd why is the average so much higher than the median well it's because you have some outliers or why is it so much lower and then the mode is the most frequent appearing value this is really interesting if you're studying economics and how people are doing you might find that the most common income like in the u.s was uh 1.24 000 a year where the average was closer to 80 000 and it's like wow what a difference well there's some people have a lot of money and so that skews that way up so the average person is not making that kind of money and then you look at the median income and you're like well the median income is a little bit closer to the average so it does create a very interesting way of looking at the data again these are all central tendencies single numbers you can look at for the whole spread of the data and we look at the measure of central tendencies the mean is the average marks of a students in a classroom so here we have the mean sum of the marks of the students total number of students and as we talked about the median if we have 0 through 10 and we take half the numbers and put them on one side of the line half the numbers on the other side of the line we end up with 5 in the middle and then the mode what mark was scored by most of the students in a test in a simple case where most people scored like an 82 percent and got certain problems wrong easy to figure out not so easy when you have different areas where like you have like the um oh let's go back to economy a little bit more difficult to calculate if you have a large group that scores that makes 30 000 and a slightly bigger group that makes 26 000. so what do you put down for the mode uh certainly there's a number of ways to calculate that and there's actually a different variations depending on what you're doing so now we're looking at a measure of spread uh range what's the difference between the highest and the lowest value first thing you want to look at you know it's uh we had everybody in the test score between 60 and 100 percent somebody got 100 or maybe 60 to 90 percent it was so hard that a lot of people could not get 100 percent um you have your inter-quartile range quartiles divide a rank ordered data set into four equal parts very common thing to do as part of all the basic packages whether you're working in data frames with pandas whether you're working in scala whether you're working in r you'll see this come up where they have range your min your max and then it'll have your interquartile range how does it look like in each quarter of data variance measures how far each number in the set is from the mean and therefore from every other number in the set so you have like a how much turbulence is going on in this data and then the standard deviation it is the measure of the

variance or the dispersion of a set of values from the mean and you'll usually see if i'm doing a graph i might have the value graphed and then based on the the error i might graph the standard deviation in the error on the graph as a background so you can see how far off it is so standard deviation is used a lot so measurement of spread marks of a student out of a hundred we have here from 50 to 63 or 50 to 90. so the range maximum marks minimum marks we have 90 to 45 and the spread of that is 45 90 minus 45. and then we have the interquartile range using the same marks over there you can see here where the median is and then there's the first quarter the second quarter and the third quarter based on splitting it apart by those values and to understand the variance and standard deviation we first need to find out the mean so here's our you know calculating the average there we end up approximately 66 for the average and then we look at that the variance once we know the means we can do equals the marks minus the mean squared why is it squared because one you want to make sure it's you don't have like if you if you're putting all this stuff together you end up with an error as far as one's negative one's positive one's a little higher one's a little lower uh so you always see the squared value and over the total observations and so the standard deviation equals the square root of the variance which is approximately 16. and if you were looking at a predictable model you would be looking at the deviation based on the error how much error does it have uh and that's again really important to know if your if your prediction is predicting something what's the chance of it being way off or just a little bit off now that we've looked at the tools as far as some of the basics for doing your statistics and what we're talking about let's go ahead and pull up a little demo and show you what that looks like in python code so you can get some little hands-on here for that let's go back into our jupiter notebook in python now almost all of this you can do in numpy last time we worked in numpy this time we're going to go ahead and use pandas and if you remember from pandas on here this is basically a data frame rows columns let's just go ahead and do a print df.head and run that and you can see we have the name jane michael william rosie hannah sat on their salaries on here and of course instead of having to do all those hand calculations and add everything together and divide by the total we can do something very simple on this like use the command mean in pandas and so if i go and do this print df pick our column salary because we want to find the means of that calorie we want to find the means of that column and we go and print this out and you can see that the average income on here is 71 000. and let's just go ahead and do this we'll go ahead and put in means and if we're going to do that we also might want to find the median and the median is very similar except it actually is just median we're used to means in average it's kind of interesting that those are they use the two different words there can be in some computation slight differences but for the most part the means is the average and then the median oops just put a median here do you have salary that way it displays a little better we can see the median is 54 000. so the halfway mark is significantly below the average why because we have somebody in here makes 189 thousand darn you rosie for throwing off our numbers but that's something you'd want to notice this is this is the difference between these is huge and so is what is the meaning behind that when you're studying a populist and looking at the different data coming in and of course we also want to find out hey what's the most uh common income that people make in this little tiny sample and so we'll go ahead and do the mode and you can see here with the mode uh it's at 50 000. so this is this is very telling that most people are making fifty thousand the middle point is at fifty four thousand so half the people are making more than that what that tells me is that if the most common income is weight is below the median then there's a few there's a scale there's a lot of high salaries going up but there's some really low salaries in there and so this trend which is very common in statistics when you're analyzing the economy in different people's income is pretty common and the bigger difference between these is also very important when we're

studying statistics and when you hear someone just say hey the average income was you might start asking questions at that point why aren't you talking about the median income why aren't you talking about the mode the most common income what are you hiding uh and if you're doing these analysis you should be looking at these saying hey why are these discrepancies why are these so different and of course with any analysis it's important to find out the minimum and the maximum so we'll go ahead it's just simply dot min i'll pull up your minimum and then dot max pulls up the maximum pretty straightforward on as far as um translating it and knowing which you know what the your lowest value which your highest value is here which you'll use to generate like a spread later on and real quick on no mode uh note that it puts mode 0. like i said there's a couple different ways you can compute the mode um although the standard one is pretty good we can of course do the range which is your max minus your min so now we have a range of 149 000 between the upper end and the lower end and you might want to be looking at the individual values on all of these but it turns out there is a describe feature in pandas and so in pandas we can actually do df salary describe and if we do this you can see we have that there's seven setups here's our mean our standard deviation which we didn't compute yet which would just be a dot std and you got to be a little careful because when it computes it it looks for axes and things like that we have our minimum value and here's our quartiles our maximum value and then of course the name salary uh so these are the these are the basic statistics you can pull them up and like just describe this is a dictionary so i could actually do something like um in here i could actually go uh count and run and now it just prints the count so because this is a dictionary you can pull any one of these values out of here it's kind of a quick and dirty way to pull all the different information and then split it up and depending on what you need now if i just walked in and gave you this information in a meeting at some point you would just kind of fall asleep that's what i would do anyway so we want to go ahead and see about graphing it here and we'll go ahead and put it into a histogram and plot that graph on it of the salaries and let's just go ahead and put that in here so we do our map plot inline remember that's a jupiter's notebook thing a lot of the new version of the matplot library does it automatically but just in case i always put it in there import matplot library pi plot is plt that's my plotting and then we have our data frame i don't i guess i really don't need to respel the data frame maybe we could just remind ourselves what's in it so we'll go ahead and just print df that way we still have it and then we have our salary df salary salary.plot history title salary distribution color gray plot ax v line salary the mean value so we're going to take the mean value color violet line style dash this is just all making it pretty what color dashed line line width of 2 that kind of thing and the median and let's go ahead and run this just so you can see we're talking about and so up here we are taking on our plot um so here's the data here's our our data frame print it out so you can see it with the salaries we'll look at the salary distribution and just look at this the way the salary is distributed you have our in this case we did let's see we had red for the median we have violet for our average or mean and you can just see how it really i mean here's our outlier here's our person who makes a lot of money here's the um average and here's the median um and so as you look at this you can say wow based on the average it really doesn't tell you much about what people are really taking home all it does is tell you how much money is in this you know what the average salary is so some of the things you want to take away in addition to this is that it's very easy to plot um an ax v line these are these up and down lines for your markers and as you just display the data i mean you can add all kinds of things to this and get really complicated keeping it simple is pretty straightforward i look at this and i can see we have a major outlier out here we can definitely do a histogram and stuff like that but you know pictures worth a thousand words what you really want to make sure you take away is that we can do a basic describe which pulls all this information out and we can print any of the individual information from the

describe because this is a dictionary and so if we want to go ahead and look up um the mean value we can also do describe mean so if you're doing a lot of statistics being able to doesn't have the print on there so it's only going to print the last one which happens to be the mean you can very easily reference any one of these and then you can also if you're doing something a little bit more complicated and you don't need just the basics you can come through and pull any one of the individual references from the from the pandas on here so now we've had a chance to describe our data let's get into inferential statistics inferential statistics allows you to make predictions or inferences from data and you can see here we have a nice little picture movie ratings and if we took this group of people and said hey how many people like the movie dislike it can't say and then you ask just a random person who comes out of the movie who hasn't been in this study you can infer that 55 chance of saying liked 35 chance of saying disliked or a 10 or 11 chance of can't say so that's real basics of what we're talking about is you're going to infer that the next person is going to follow these statistics so let's look at point estimation it is a process of finding an approximate value for a population's parameter like mean or average from random samples of the population let's take an example of testing vaccines for covid19 vaccines and flu bugs all that it's a pretty big thing of how do you test these out and make sure they're going to work on the populace a group of people are chosen from the population medical trials are performed results are generalized for the whole population so here's a protected there's our small group up here where we've selected them we run medical trials on them and then the results work for the population nice diagram with the arrows going back and forth and the very scary coveted virus in the middle of one and let's take a look at the applications of inferential statistics very central is what they call hypotheses testing and the confidence interval which go with that and then as we get into probability we get into our binomial theorem our normal distribution in central limit theorem hypothesis testing hypothesis testing is used to measure the plausibility of a hypothesis assumption by using sample data now when we talk about theorems theory hypothesis keep in mind that if you are in a philosophy class theory is the same as hypothesis where theorem is a scientific statement that is something that has been proven although it is always up for debate because in science we always want to make sure things are up to debate so hypothesis is the same as a philosophical class calling a theory where theory in science is not the same theory in science says this has been well proven gravity is a theory so if you want to debate the theory of gravity try jumping up and down if you want to have a theory about why the economy is collapsing in your area that is a philosophical debate very important i've heard people mix those up and it is a pet peeve of mine when we talk about hypotheses testing the steps involved in hypotheses testing is first we formulate a hypothesis we figure out the right test to test our hypothesis we execute the test and we make a decision and so when you're talking about hypothesis you're usually trying to disprove it if you can't disprove it and it works for all the facts then you might call that a theorem at some point so in a use case uh let's consider an example we have four students we're given a task to clean a room every day sounds like working with my kids they decided to distribute the job of cleaning the room among themselves they did so by making four chits which has their names on it and the name that gets picked up has to do the cleaning for that day rob took the opportunity to make chits and wrote everyone's name on it so here's our four people nick rob emlia imlia and summer now rick emilia and summer are asking us to decide whether rob has done some mischief in preparing the chits i.e whether rob has written his name on one of the chit for that we will find out the probability of rob getting the cleaning job on first day second day third day and so on till 12 days the probability of rob getting the job decreases every day ie his turn never comes up then definitely he has done some mischief while making the chits so the probability of rob not doing work on day one is uh three out of four there's a point 75 chance that he didn't do work two days

three-fourths times three-fourths equals 0.56 three days you have three fourths three fourths three fourths which equals point four two uh when you get to day twelve it's 0.032 which is less than 0.05 remember this 0.05 that comes up a lot when we're talking about certain values when we're looking at statistics rob is cheating as he wasn't chosen for 12 consecutive days that's a very high probability when on day 12 he still hasn't gotten the job cleaning the room so we come up to our important important terminologies we have null hypothesis a general statement that states that there is no relationship between two measured phenomena or no association among the groups alternative hypothesis contrary to the null hypothesis it states whenever something is happening a new theory is preferred instead of an old one and so the two hypotheses go hand in hand uh so your null this is always interesting in and when talking about data science and the math behind it it's about proving that the things have no correlation null hypothesis says these two have zero relation to each other where the alternative hypothesis says hey we found a relation this is what it is we have p-value the p-value is a probability of finding the observed or more extreme results when the null hypothesis of a study question is true and the t value it is simply the calculated difference represented in units of standard error the greater the magnitude of t the greater the evidence against the null hypothesis and you can look at the t values being specific to the test you're doing where the p value is derived from your t value and you're looking for what they call the five percent or the 0.05 showing that it has a high correlation so digging in deeper let's assume that a new drug is developed with the goal of lowering the blood pressure more than the existing drug and this is a good one because the null value here isn't that you don't have any drug the null value here is that it's better than the existing drug the new drug doesn't lower the blood pressure more than the existing drug now if we get that that says our null hypothesis is correct there is no correlation and the new drug is not doing its job the alternative hypothesis the new drug does significantly lower the blood pressure more than the existing drug yay we got a new drug out there and that's our alternative hypothesis or the h1 or ha and we look at the p-value results from the evidence like medical trial showing positive results which will reject the null hypothesis and again they're looking for a 0.05 or 5 percent and the t value comparing all the positive test results and finding means of different samples in order to test hypothesis so this is specific to the test how what percentage of increase did they have and this leads us to the confidence intervals a confidence interval is a range of values we are sure our true values of observations lie in let's say you asked a dog owner around you and asked them how many cans of food do you buy for your per year for your dog through calculations you got to know that the on an average around 95 percent of the people bought around 200 to 300 cans of food hence we can say that we have a confidence interval of 2 300 where 95 percent of our values lie in that data spread and this the graph really helps a lot so you can start seeing what you're looking at here we have the 95 percent you have your peak in this case it's a normal distributions here the nice bell curve equal on both sides it's not asymmetrical and 95 of all the values lie within a very small range and then you have your outliers the 2.5 percent going each way so we touched upon hypothesis uh we're going to move into probability uh so you have your hypothesis once you've generated your hypothesis we want to know the probability of something occurring probability is a measure of the likelihood of an event to occur any event can be predicted with total certainty and can only be predicted as a likelihood of its occurrence so any event cannot be predicted with total certainty can only be predicted as a likelihood of its occurrence score prediction how good you're going to do in whatever sport you're in weather prediction stock prediction if you've studied physics and chaos theory even the location of the chair you're sitting on has a probability that it might move three feet over granted that probability is one in like uh i think we calculated as under one in trillions upon trillions so it's the better the probability the more likely it's going to happen there are some

things that have such a low probability that we don't see them so we talk about random variable a random variable is a variable whose possible values are numerical outcomes of a random phenomena so uh we have the coin tossed how many heads will occur in the series of 20 coin flips probably you know the on average they're 10 but you really can't know because it's very random how many times a red ball is picked from a bag of balls if there's equal number of red balls and blue balls and green balls in there how many times the sum of digits on two dice results are five each so you know there's how often you're gonna roll two fives on your pair of dice so in a use case uh let's consider the example of rolling two dice we have a random variable outcome equals y you can take values two three four five six seven eight nine ten eleven twelve so we have a random variable and a combination of dice and instead of looking at how many times both dice for roll five let's go ahead and look at a total sum of five and you have in as far as your random variables you can have a one four equals five four one two three three two so four of those rolls can be four if you look at all the different options you have four of those random rolls can be a five and if we look at the total number which happens to be 36 different options uh you can see that we have 4 out of 36 chance every time you roll the dice that you're gonna roll a total of five you can have an outcome of five and uh we'll look a little deeper as to what that means but you could think of that at what point if someone never rolls a five or they always roll a five can you say hey that person's probably cheating we'll look a little closer at the math behind that but let's just consider this is one of the cases is rolling two dice and gambling there's also a binomial distribution it is the probability of getting success or failure as an outcome in an experiment or trial that is repeated multiple times and the key is is by meaning two binomial so passing or failing an exam winning or losing a game and getting either head or tails so if you ever see binomial distribution it's based on a true false kind of setup you win or lose let's consider a use case and let's consider the game of football between two clubs barcelona and dortmund the teams will have to play a total of four matches and we have to find out the chances of barcelona winning the series so we look at the total games and we're looking at five different games or matches let's say that the winning chance for barcelona is 75 percent or 0.75 that means that each game they have a 75 chance that they're going to win that game and losing chances are 25 or 0.25 clearly 0.75 plus 0.25 equals 1. so that accounts for 100 of the game probability for getting k wins in in matches is calculated and we we're talking like so if you have five games and you want to know if i play um how many wins in those five games should i get what's a percentage on those and the probability for getting k wins and in matches is calculated by p x equals k equals n c k p to the k q to the n minus k here p is the probability of success and q is the probability of failure and so we can do total games of n equals five where k equals zero one two three four five p which is the chance of winning is point 0.75 q the chance of losing equals 1 minus p which equals 1 minus 0.075 which equals 0.25 the probability that barcelona will lose all of the matches can then just plug in the numbers and we end up with a point zero zero zero nine seven six five six two five so very small chance they're going to lose all their matches and we can plug in uh the value for two matches probability that barcelona will win at least two matches is 0.0878 and of course we can go on to the probability that barcelona will win three matches the 0.26 and of course four matches and so on and it's always nice to take this information and let's find the accumulated discrete probabilities for each of the outcomes where barcelona has won three or more matches x equals three x equals four x equals five and we end up with the p equals 0.264 plus 0.395 plus 237 which equals 0.89 in reality the probability of barcelona winning the series is much higher than 0.75 and it's always nice to put out a nice graph so you can actually see the number of wins to the probability and how that pans out with our binomial case continuing in our important terminology location the location of the center of the graph depends on the mean value and this is some very important

things so much of the data we look at and when you start looking at probabilities almost always has a normalized look like the graph in the middle but you do have left skewed where the data is skewed off to the left and you have more stuff happening off to the left and you have right skewed data and so when this comes up and these probabilities come up where they're skewed it's really important to take a closer look at that mostly you end up with a normalized set of data but you got to also be aware that sometimes it's a skewed data and then the height height of the slope inversely depends upon the standard deviation so you can see down here the standard deviation is really large it kind of squishes it out and if the standard deviation is small then most of your data is going to hit right there in the middle you can have a nice peak and so being aware of this that you might have a probability that fits certain data but it has a lot of outliers so you're if you have a really high standard deviation if you're doing stock market analysis this means your predictions are probably not going to make you much money where if you have a very small deviation you might be right on target and set to become a millionaire which leads us to the z-score z-score tells you how far from the mean a data point is it is measured in terms of standard deviations from the mean around 68 percent of the results are found between one standard deviation around 95 percent of the results are found between two standard deviations and you read the symbols of course they love to throw some greek letters in there we have a mu minus two sigma mu is just a quick way it's a kind of funky u it just means the mean uh and then the sigma is the standard deviation and that's the o with the little arrow off to the right or the little wagy tail going up the o with it with the line on it so mu minus two sigma is your uh 95 percent of the results are found between two standard deviations central limit theorem this goes back to the skew if you remember we were looking at the skew values on this previous slide have left skewed normalized and right skewed when we're talking about it being skewed or not skewed the distribution of the sample means will be approximately normally distributed evenly distributed not skewed if you take large random samples from the population with the mean mu and the standard deviation sigma with replacement and you can see here of course we have our mu minus two sigma and the spread down here the mean the median and the mode and so you're talking about very large populations these numbers should come together and you shouldn't have a skewed value if you do that's a flag that something's wrong that's why this is so important to be aware of what's going on with your data where your samples are coming from and the math behind it if we're going to do all this we got to jump into conditional probability the conditional probability of an event a is the probability that the event will occur given the knowledge that an event b has already occurred and you'll see this as bayes theorem b-a-y-e-s base and this is red i mean you have these funky looking little p brackets a b this is the probability of a being true while b is already true and you have the probability of b being true when a is already true so p b of a probability of a being true divided by the probability of b being true and we talk about bayes theorem which occurred back in the 1800s when he discovered this this is such an important formula and it's really it's not if you actually do the math you could just kind of do x y equals j k and then you divide them out and you're going to see the same math but it works with probabilities which makes it really nice and so if you have a set you might have eight or nine different studies going on in different areas different people have done the studies they brought them together if we look at today's covered virus the virus spread certainly the studies done in china versus the studies the way they're done in the u.s that data is different in each of those studies but if you can find a place where it overlaps where they're studying the same thing together you can then compute the changes that you need to make in one study to make them equal and this is also true if you have a study of one group and you want to find out more about it so this formula is very powerful and it really has to do with the data collection part of the math and data science and understanding where your data is coming from and how you

going to combine different studies in different groups and we're going to go into a use case let's find out the chance of a person getting lung disease due to smoking and this is kind of interesting the way they word this let's say that according to medical report provided by the hospital states that around 10 percent of all patients they treated suffered lung disease so we have kind of a generic medical report they further found out by a survey that fifteen percent of the patients that visit them smoke so we have ten percent that are lung disease and um fifteen percent of the patients smoke and finally five percent of the people continued smoke even when they had lung disease uh not the brightest choice but you know it is an addiction so it can be really difficult to kick and so we can look at the probability of a uh prior probability of 10 people having lung disease and then probability b probability that a patient smokes is 15 percent uh and the probability of b if b then a the probability of a patient smokes even though they have lung disease is five percent and probability of a is b probability that the patient will have lung disease if they smoke and then when you put the formulas together uh you get a nice solution here you get the probability of a of b probability that the patient will have lung disease if they smoke and you can just plug the numbers right in and we get a 3.33 percent chance hence there is a 3.33 chance that a person who smokes will get a lung disease so we're going to pull up a little python code i'm always my favorite roll up the sleeves keep in mind we're going to be doing this kind of like the back end way so that you can see what's going on and then later on we're going to create um we'll get into another demo which shows you some of the tools that are already pre-built for this let's start by creating a set so we're going to create a set with curly braces this means that our set has only unique values so you have a list you have your tuples which can never change and then you have in this case the the set so four seven you can't create a four seven comma four it'll delete the four out so it's only unique values and if you use dictionaries quick reminder this should look familiar because it is a dictionary where you have a value and that value is assigned to or that key is assigned to a value so you could have a key value set up as a dictionary so it's like a dictionary without the value it's just the keys and they all have to be unique and if we run this we have a set of four seven we can also take a list a regular setup and i'm going to go ahead and just throw in another number in here four and run it and you can see here if i take my list one two three four four and i convert it to a set and here it is my set from list equals set my list the result is one two three four so it just deletes that last four right out of there and with the sets you can also go in there and print here is my set my set uh three is in the set and then if you do three in my set that's going to be a logic function uh and one in my set six is not in the set and so forth if we run this we get three is in the set true one is in the set false because three five seven is another one six is in the set uh six is not in the set so not in my set you can also use this with the list we could have just used three five seven and it would have um the same response on there is three and usually you do if three is in but three in my set is still works on just a regular list and we'll go ahead and do a little iteration we're going to do kind of the dice one remember one two three four five six and so we're going to bring in the iteration tool and import product as product and i'll show you what that means in just a second so we have our two dice we have dice a and it's going to be a set of values you can only have one value for each one that's why they put it in a set and if you remember from range it is up to seven so this is going to be one two three four five six it will not include the seven and the same thing for our dice b and then we're gonna do is we're gonna create a list which is the product of a and b so what's a plus b and if we go ahead and run this it'll print that out and you'll see in this case when they say product because it's an iteration tool we're talking about creating a tuple of the two so we've now created a tuple of all possible outcomes of the dice where dice a is one two three one to six and dice b is one to six and you can see one to one one to two one to three and so forth you remember we had a slide on this earlier where we talked about the

different all the different outcomes of a dice we can play around with this a little bit we can do in dice equals two dice faces one two three four five six another way of doing we did before and then we can create an event space where we have a set which is the product of the dice faces repeat equals in dice and we'll go and just run this and you can see here it just again puts it through all the different possible variables we can have and then if we wanted to take the same set on here and print them all out like we had before we can just go through for outcome and event space outcome and equals so the event space is creating a sequence and as you can see here when we print it out it stacks them versus going through and putting them in a nice line and we'll go ahead and do something um let's go print since we have the end printing with a comma that just means it's just gonna it's not gonna hit the return going down to the next line uh and we'll go ahead and do the links of our event space that'll be an important variable we're going to want to know in a minute and of course if i get carried away with my typing of length uh we'll print it twice and it'll give me an error so we have 36 different possible variations here and we might want to calculate something like what about the multiple of 3 what if we want to have the probability of the multiple three in our setup and so we can put together the code for the outcome and event space of x y equals outcome if x plus y remainder 3. so we're going to divide by 3 and look at the remainder and it equals 0. then it's a favorable outcome we're going to pop that outcome on the in there and we'll turn it into a set so the favor outcome equals a set not necessary because we know it's not going to be repeating itself but just in case we'll go ahead and do that and if we want to print out the outcome we can go ahead and see what that looks like and you can see here these are all multiples of three one plus two is three five plus four is nine which divided by three is three and so forth and just like we looked up the length of the one before let's go ahead and print the length of our f outcome so we can see what that looks like there we go and of course i did forget to add the print in the middle because we're looping through and putting an end on the on the setup on there so we're going to put the print in there and if i run this you can see um we end up with 12. so we have 36 total options we have 12 that are multiple that add up to a multiple of three and we can easily conv compute the probability of this by simply taking the length of our favorable outcome over the length of the event space and if we print it out let me put that in there probability last line so we just type it in we end up with a 0.3333 chance that's roughly a third and we want to make this look nice so let's go ahead and put in another line there the probability of getting the sum which is a multiple of three is point three three three three we can compute the same thing for five dice and if we do this for five dice and go and run it you can see we just have a huge amount of choices so it just goes on and on down here and we can look at the length of the event space and we have over 7776 choices that's a lot of choices and if we want to ask the question like we did above uh what is the sum where the sum is a multiple of five but not a multiple of three we can go through all of these different options and then uh you can see here uh d1 d2 d3 d4 d5 equals the outcome and if you add these all together and the division by five does not have a remainder of zero but the remainder is also of a division by three is not equal to zero so the multiple of five is equal to zero but the multiple three is not we can just append that on here and then we can look at that uh favorable outcome we'll go ahead and set that and we'll just take a look at this what's our length of our favorable outcome it's always good to see what we're working with and so we have 904 out of 770 6 and then of course we can just do a simple division to get the probability on here what's the probability that we're going to roll a multiple of 5 when you add them together but not a multiple of 3. and so we're just going to divide those two numbers and you can see here we get 0.11625 or 11.62 percent and so you can really have a nice visual that this is not really complicated math right here on probabilities it's just how many options do you have and how many of those are you possibly going to be able to come up with with the solution you're

looking for and this leads us to a confusion matrix a confusion matrix is a table which is used to describe the performance of a classification model on a set of test data for which the true values are known and so you'll see in the left we have the predicted and the actual and we have a negative uh false negative positive true positive um and then we have false positive and true negative and you can think of this as your predicted model what does that mean that means if you divided your data and you used two-third of it to create the model you might then test it against an actual case for the last third to see how well it comes out how many times was it true positive versus false positive it gave a false positive response and you can imagine in medical situations this is a pretty big deal you don't want to give a false positive so you might adjust your model accordingly so you don't have a false positive say with a covavirus test it'd be better to have a false negative and they go back and get retested than to have 30 false positives where then the test is pretty much invalid so in a use case like cancer prediction let's consider an example where a cancer prediction model is put to the test for its accuracy and precision actual result of a person's medical report is compared with the prediction made by the machine learning model and so you can see here here's our actual predicted whether they have cancer or not you know cancer a big one you don't want to have a false positive i mean a false negative in other words you don't want to have it tell you that you don't have cancer when you do so that would be something you'd really be looking for in this particular domain you don't want a false negative and this is again you know you've created a model you have hundreds of people or thousands of pieces of data that come in there's a real famous case study where they have the imagery and all the measurements they take and there's about 36 different measurements they take and then if you run the a basic model you want to know just how accurate it is how many negative results do you have that are either telling people they have cancer that don't or telling people that don't have cancer that they do and then we can take these numbers and we can feed them into our accuracy our precision and our recall so accuracy precision and recall accuracy metric to measure how accurately the results are predicted and this is your total um true where you got the right results you add them together the true positive the two negative over all the results so what percentage of them were accurate versus what were wrong we talked about precision is a metric to measure how many of the correctly predicted cases are actually turned out to be positive uh so we have a precision on true positive again if you're talking about like covid testing with the viruses uh you really want this to be a high number you want this true that to be the center point where you might have the opposite if you're dealing with a cancer where you want no false negatives so this is your metric on here precision is your test positive true positive plus false positive and then your recall how many of the actual positive cases we were able to predict quickly with our model uh so test positive is the test positive plus the false negative on there and we'll want to go ahead and do a demo on the naive bayes classifier before i get too far into a naive bayes classifier because we're going to pull it from the sk learn or the scikit let's go ahead kind of an interesting page here for classifiers when you go into the sk learn kit there's a lot of ways to do classification and i'll just zoom up in here so you can see some of the titles there's everything from the nearest neighbor linear but we're going to be focusing on the naive bayes over here and this is just a sample data set that they put together and you can see how some of these have a very different output the naive bayes remember is set up as probably the most simplified uh calculator or set of predictions out there and so what we've been talking about with the true false and stuff like that where there's a and then a belief that there is a independent assumption between the features where the features are very assumed to have some kind of connection uh then we can go ahead and use that for the prediction and so that's what we're using is a naive bayes classifier versus many of the other classifiers that are out there for this we're going to use the social network ads it's a little data set

on here and let me go and just open that up the file here we go it has user id gender age estimated salary uh purchased and so we have you can see the user id mail 19 estimated salary 19 000 and purchase zero so it's either gonna make a purchase or not so look at that last one zero one we should be thinking of binomials we should be thinking of a simple naive bayes classifier kind of setup so if we close this out we're going to go ahead and import our numpy as np we're going to nice to have a good visual of our data so we'll put in our matplot library here's our pandas our data frame uh and then we're going to go ahead and import the data set and the data set's going to be we're going to read it from the social network ads.csv then we're going to print the head just so you can see it again even though i showed you it in the file and x equals the data set i location two three values and y is going to be the 4 column 4. let me just run this it's a little easier to go over that you can see right here we're going to be looking at 0 1 2 as age and estimated salary so 2 3 and that's what i location just means that we're looking at the number versus a regular location a regular location you'd actually say age and estimated salary and then column four is did they make a purchase they purchased something uh so those are the three columns we're going to be looking at when we do this and we've gone ahead and imported these and imported the data so now our data set is all set with this information in it and we'll need to go ahead and split the date up so we need our from the sk learn model selection we can import train test split this does a nice job we can set the random state so randomly picks the data and we're just going to take uh 25 of it's going to go into the test our x test and our y test and the 75 will go to x train and y train that way once we create our model we can then have data to see just how accurate or how well it has performed with our prediction the next step in pre-processing our data is to go ahead and do feature scaling now a lot of this to start to look familiar if you've done a number of the other modules and setup you should start noticing that we bring in our data we take a look at what we're working with we go ahead and split it up into training and testing in this case we're going to go ahead and scale it scale it means we're putting it between a value of minus one and one uh or someplace in the middle ground there this way if you have any huge set you don't have this huge setup if we go back up to here where salary the salary is 20 000 versus age 35. well there's a good chance with a lot of the back end math that 20 000 will skew the results and the estimated salary will have a higher impact than the age instead of balancing them out and letting the calculations weigh them properly and finally we get to actually create our naive bayes model um and then we're going to go ahead and import the gaussian naive bayes and the gaussian is is the most basic one that's what we're looking at now it turns out though if you go to the sk learn kit they have a number of different ones you can pull in there there's a bernoulli i've never used that one categorical complement and here's our gaussian so there's a number of different options you can look at gaussian when you come to the naive bayes is the most commonly used so we're talking about the naive bayes that's usually what people are talking about when they when they're pulling this in and one of the nice things about the gaussian if you go to their website to sk learn the naive bayes gaussian there's a lot of cool features one of them is you can do partial fit on here that means if you have a huge amount of data you don't have to process it all i want you once you can batch it into the gaussian nb model and there's many other different things you can do with it as far as fitting the data and how you manipulate it we're just doing the basics so we're going to go ahead and create our classifier we're going to equal the gaussian in b and then we're going to do a fit we're going to fit our training data and our training solution so x train y train and we'll go ahead and run this and it's going to tell us that it ran the code right there and now we have our trained classifier model so the next step is we need to go ahead and run a prediction we're going to do our y predict equals the classifier.predict x test so here we fit the data and now we're going to go ahead and predict and now we get to our confusion matrix so from the sk learn matrix metrics

you can import your confusion matrix just as saves you from doing all the simple math that does it all for you and then we'll go ahead and create our confusion metrics with the y test and the y predict so we have our actual and we have our predicted value and you can see from here this is the chart we looked at here's predicted so true positive false positive false negative true negative and if we go ahead and run this there we have it 65 3 7 25 and in this particular prediction we had 65 or predicted the truth as far as a purchase they're gonna make a purchase and we guessed three wrong and then we had 25 we predicted would not purchase and seven of them did so there's our our confusion matrix at this point if you were with your shareholders or a board meeting um you would start to hear some snoozing if they were looking at the numbers and you say hey here's my confusion matrix so let's go ahead and visualize the results we're going to pull from the matplot library colors import listed color map and this is actually my machine is going to throw an error because this is being because of the way the setup is i have a newer version on here than when they put together the demo and we need our x set and our y set which is our x train and y train and then we'll create our x1 x2 and we'll put that into a grid and we set our x at minimum stop and our x at max stop and if you come all the way over here we're going to step .01 this is going to give us a nice line uh is what that's doing and then we're going to plot the contour plot the x limit plot the y limit and put the scatter plot in there let's go ahead and run this uh to be honest when i'm doing these graphs there's so many different ways to do that there's so many different ways to put this code together to show you what we're doing it's a lot easier to pull up the graph and then go back up and explain it so the first thing we want to note here when we're looking at the data is this is the training set and so we have those who didn't make a purchase we've drawn a nice area for that that's defined by the naive bayes setup and then we have those who did make a purchase the green and you can see that some of the green drops fall into the red area and some of the red dots fall into the green so even our training set isn't going to be a hundred percent we couldn't do that and so we're looking at our different data coming down uh we can kind of arrange our x1 x2 so we have a nice plot going on and we're going to create the contour that's that nice line that's drawing down the middle on here with the red green that's where that's what this is doing right here with the reshape and notice that we had to do the dot t if you remember from numpy if you did the numpy module you end up with pairs you know x x1 x2 x1 x2 next row and so forth you have to flip it so it's all one row you have all your x1 and all your x2s so that's what we're kind of looking for right here on this setup and then the scatter plot is of course your scattered data across there we're just going through all the points that puts these nice little dots on to our setup on here and we have our estimated salary and our h and then of course the dots are did they make a purchase or not and just a quick note this is kind of funny you can see up here where it says x set y set equals x train y train which seems kind of a little weird to do this is because this is probably originally a definition uh so it was its own module that could be called over and over again and which is really a good way to do it because the next thing we're going to want to do is do the exact same thing but we're going to visualize the test set results that way we can see what happened with our test group our 25 percent and you can see down here we have the test set and it if you look at the two graphs next to each other this one obviously has 75 percent of the data so it's going to show a lot more this is only 25 of the data you can see that there's a number that are kind of on the edge as to whether they could guess by age and income they're going to make a purchase or not but that said it still is pretty clear it's pretty good as far as how much the estimate is and how good it does now graphs are really effective for showing people what's going on but you also need to have the numbers and so we're going to do from sklearn we're going to import metrics and then we're going to print our metrics classification port from the y test and the y predict and you can see here we have precision precision of zeros is 90 there's

our recall 0.96 we have an f1 score and a support and we have our precision the recall on getting it right and then we can do our accuracy the macro average and the weighted average so you can see it it pulls in pretty good as far as how accurate it is you could say it's going to be about 90 percent is going to guess correctly that that they're not going to purchase and we had an 89 chance that they are going to purchase and then the other numbers as you get down have a little bit different meaning but it's pretty straightforward on here here's our accuracy and here's our micro average and the weighted average and everything else you might need and if you forgot the exact definition of accuracy it is the true positive true negative over all of the different setups precision is your true positive over all positives true and false and recall is a true positive over true positive plus false negative and we can just real quick flip back there so you can see those numbers on here uh here's our precision here's our recall and here's our accuracy on this we're going to cover reinforcement learning today and what's in it for you we'll start with why reinforcement learning we'll look at what is reinforcement learning we'll see what the different kinds of learning strategies are that are being used today in computer models under supervised versus unsupervised versus reinforcement we'll cover important terms specific to reinforcement learning we'll talk about markov's decision process and we'll take a look at a reinforcement learning example well we'll teach a tic-tac-toe how to play why reinforcement learning training a machine learning model requires a lot of data which might not always be available to us further the data provided might not be reliable learning from a small subset of actions will not help expand the vast realm of solutions that may work for a particular problem you can see here we have the robot learning to walk very complicated setup when you're learning how to walk and you'll start asking questions like if i'm taking one step forward and left what happens if i pick up a 50-pound object how does that change how a robot would walk these things are very difficult to program because there's no actual information on it until it's actually tried out learning from a small subset of actions will not help expand the vast realm of solutions that may work for a particular problem and we'll see here learned how to walk this is going to slow the growth that technology is capable of machines need to learn to perform actions by themselves and not just learn off humans and you see the objective climate mountain real interesting point here is that as human beings we can go into a very unknown environment and we can adjust for it and kind of explore and play with it most of the models the non-reinforcement models in computer machine learning aren't able to do that very well there's a couple of them that can be used or integrated to see how it goes is what we're talking about with reinforcement learning so what is reinforcement learning reinforcement learning is a sub-branch of machine learning that trains the model to return an optimum solution for a problem by taking a sequence of decisions by itself consider a robot learning to go from one place to another the robot is given a scenario must arrive at a solution by itself the robot can take different paths to reach the destination it will know the best path by the time taken on each path it might even come up with a unique solution all by itself and that's really important is we're looking for unique solutions we want the best solution but you can't find it unless you try it so we're looking at our different systems our different model we have supervised versus unsupervised versus reinforcement learning and with the supervised learning that is probably the most controlled environment we have a lot of different supervised learning models whether it's linear regression neural networks there's all kinds of things in between decision trees the data provided is labeled data with output values specified and this is important because we talk about supervised learning you already know the answer for all this information you already know the picture has a motorcycle in it so you're supervised learning you already know that the outcome for tomorrow for you know going back a week you're looking at stock you can already have like the graph of what the next day looks like so you have an answer for it and you have

labeled data which is used you have an external supervision and solves problems by mapping labeled input to known output so very controlled unsupervised learning and unsupervised learning is really interesting because it's now taking part in many other models they start with and you can actually insert an unsupervised learning model in almost either supervised or reinforcement learning as part of the system which is really cool data provided is unlabeled data the outputs are not specified machine makes its own predictions used to solve association with clustering problems unlabeled data is used no supervision solves problems by understanding patterns and discovering output so you can look at this and you can think some of these things go with each other they belong together so it's looking for what connects in different ways and there's a lot of different algorithms that look at this when you start getting into those are some really cool images that come up of what unsupervised learning is how it can pick out say the area of a donut one model will see the area of the donut and the other one will divide it into three sections based on this location versus what's next to it so there's a lot of stuff that goes in with unsupervised learning and then we're looking at reinforcement learning probably the biggest industry in today's market in machine learning or growing market it's very it's very infant stage as far as how it works and what's going to be capable of the machine learns from its environment using rewards and errors used to solve reward based problems no predefined data is used no supervision follows trail and error problem solving approach so again we have a random first you start with a random i try this it works and this is my reward doesn't work very well maybe or maybe doesn't even get you where you're trying to get it to do and you get your reward back and then it looks at that and says well let's try something else and it starts to play with these different things finding the best route so let's take a look at important terms in today's reinforcement model and this has become pretty standardized over the last few years so these are really good to know we have the agent agent is the model that is being trained via reinforcement learning so this is your actual uh entity that has however you're doing it whether using a neural network or a cue table or whatever combination thereof this is the actual agent that you're using this is the model and you have your environment the training situation that the model must optimize to is called its environment and you can see here i guess we have a robot who's trying to get chest full of gems or whatever and that's the output and then you have your action this is all possible steps that can be taken by the model and it picks one action and you can see here it's picked three different routes to get to the chest of diamonds and gems we have a state the current position condition returned by the model and you could look at this uh if you're playing like a video game this is the screen you're looking at so when you go back here the environment is a whole game board so if you're playing one of those mobius games you might have the whole game board going on uh but then you have your current position where are you on that game board what's around that what's around you if you were talking about a robot the environment might be moving around the yard where it is in the yard and what it can see what input it has in that location that would be the current position condition returned by the model and then the reward uh to help the model move in the right direction it is rewarded points are given to it to appraise some kind of action so yeah you did good or didn't do as good trying to maximize the reward and have the best reward possible and then policy policy determines how an agent will behave at any time it acts as a mapping between action and present state this is part of the model what what is your action that you're you're going to take what's the policy you're using to have an output from your agent one of the reasons they separate policy as its own entity is that you usually have a prediction of a different options and then the policy well how am i going to pick the best based on those predictions i'm going to guess at different options and we'll actually weigh those options in and find the best option we think will work so it's a little tricky but the policy thing is actually pretty cool how it works let's go ahead and take a

look at a reinforcement learning example and just in looking at this we're going to take a look consider what a dog that we want to train so the dog would be like the agent so you have your your puppy or whatever and then your environment is going to be the whole house or whatever it is where you're training them and then you have an action we want to teach the dog to fetch so action equals fetching and then we have a little biscuit so we can get the dog to perform various actions by offering incentives such as a dog biscuit as a reward the dog will follow a policy to maximize this reward and hence will follow every command and might even learn new actions like begging by itself uh so you have b you know so we start off with fetching it goes oh i get a biscuit for that it tries something else you get a handshake or begging or something like that and it goes oh this is also reward based and so it kind of explores things to find out what will bring is biscuit and that's very much like how a reinforced model goes as it looks for different rewards how do i find can i try different things and find a reward that works the dog also will want to run around and play an explorer's environment this quality of model is called exploration so there's a little randomness going on in exploration and explores new parts of the house climbing on the sofa doesn't get a reward in fact it usually gets kicked off the sofa so let's talk a little bit about markov's decision process markov's decision process is a reinforcement learning policy used to map a current state to an action where the agent continuously interacts with the environment to produce new solutions and receive rewards and you'll see here's all of our different uh vocabulary we just went over we have a reward our state our agent our environment interaction and so even though the environment kind of contains everything that you really when you're actually writing the program your environment is going to put out a reward in state that goes into the agent the agent then looks at this state or it looks at the reward usually first and it says okay i got rewarded for whatever i just did or it didn't get rewarded and it looks at the state and then it comes back and if you remember from policy the policy comes in and then we have a reward the policy is that part that's connected at the bottom and so it looks at that policy and says hey what's a good action that will probably be similar to what i did or sometimes they're completely random but what's a good action that's going to bring me a different reward so taking the time to just understand these different pieces as they go is pretty important in most of the models today and so a lot of them actually have templates based on this you can pull in and start using pretty straightforward as far as once you start seeing how it works uh you can see your environment sends it says hey this is the agent did this if you're a character in a game this happened and it shoots out a reward in a state the agent looks at the reward looks at the new state and then takes a little guess and says i'm going to try this action and then that action goes back into the environment it affects the environment the environment then changes depending on what the action was and then it has a new state and a new reward that goes back to the agent so in the diagram shown we need to find the shortest path between node a and d each path has a reward associated with it and the path with a maximum reward is what we want to choose the nodes a b c d denote the nodes to travel from node a to b is an action reward is a cost of each path and policy is each path taken and you can see here a can go to b or a can go to c right off the bat or can go right to d and if explored all three of these you would find that a going to d was a zero reward a going to c and d would generate a different reward or you could go a c b d there's a lot of options here and so when we start looking at this diagram you start to realize that even though today's reinforced learning models do really good at finding an answer they end up trying almost all the different directions you see and so they take up a lot of work uh or a lot of processing time for reinforcement learning they're right now in their infant stage and they're really good at solving simple problems and we'll take a look at one of those in just a minute in the tic-tac-toe game but you can see here uh once it's gone through these and it's explored it's gonna find the a c d is the best reward it gets a full 30 points for it so let's go ahead

and take a look at a reinforcement learning demo in this demo we're going to use reinforcement learning to make a tic-tac-toe game you will be playing this game against the machine learning model and we'll go ahead and we're doing it in python so let's go ahead and go through i always not always actually have a lot of python tools let's go through anaconda which will open up a jupiter notebook it seems like a lot of steps but it's worth it to keep all my stuff separate and it's also has a nice display when you're in the jupiter notebook for doing python so here's our anaconda navigator i open up the notebook which is going to take me to a webpage and i've gone in here and created a new python folder in this case i've already done it and enabled it to change the name to tic-tac-toe and then for this example we're going to go ahead and import a couple things we're going to import numpy as np we'll go ahead and import pickle numpy of course is our number array and then pickle is just a nice way sometimes for storing different information different states that we're going to go through on here and so we're going to create a class called state we're going to start with that and there's a lot of lines of code to this class that we're going to put in here don't let that scare you too much there's not as much here it looks like there's going to be a lie here but there really is just a lot of setup going on in the in our class state and so we have up here we're going to initialize it um we have our board it's a tic tac toe board so we're only dealing with nine spots on the board uh we have player one player two uh is end we're gonna create a board hash uh we'll look at that in just a minute we're just gonna store some information in there symbol player equals one um so there's a few things going on as far as the initialization uh then something simple we're just going to get the hash of the board you get the information from the board on there which is columns and rows we want to know when a winner occurs uh so if you get three in a row that's what this whole section here is for uh let me go ahead and scroll up a little bit and you can get a copy of this code if you send a note over to simply learn we'll send you over this particular file and you can play with it yourself and see how it's put together i don't want to spend a huge amount of time on this because this is just some real general python coding but you can see here we're just going through all the rows and you add them together and if it equals three three in a row same thing with columns diagonal so you gotta check the diagonal that's what all this stuff does here is it just goes through the different areas actually let me go ahead and put there we go um and then it comes down here and we do our sum and it says true uh minus three just says did somebody win or is it a tie so you gotta add up all the numbers on there anyway just in case they're all filled up and next we also need to know available positions um these are ones that don't no one's ever used before this way when you try something or the computer tries something uh it's not going to give it an illegal move that's what the available positions is doing then we want to update our state and so you have your position going in we're just sending in the position that you just chose and you'll see there's a little user interface we put in there we can pick the row and column in there and again i mean this is a lot of code uh so really it's kind of a thing you'd want to go through and play with a little bit and just read through it get a copy of it a great way to understand how this works and here is a given reward um so we're going to give a reward result equals self winner this is one of the hearts of what's going on here is we have a result a self.winner so if there's a winner then we have a result that the result equals one here's our feedback uh if it doesn't equal one then it gets a zero so it only gets a reward in this particular case if it wins and that's important to know because different systems of reinforced learning do rewarding a lot differently depending on what you're trying to do this is a very simple example with a three by three board imagine if you're playing a video game certainly you only have so many actions but your environment is huge you have a lot going on in the environment and suddenly a reward system like this is going to be just it's going to have to change a little bit it's going to have to have different rewards and different setup and there's all kinds of advanced ways to do that as far as weighing you add

weights to it and so they can add the weights up depending on where the reward comes in so it might be that you actually get a reward in this case you get the reward at the end of the game and i'm spending just a little bit of time on this this is an important thing to note but there's different ways to add up those rewards it might have like if you take a certain path the first reward is going to be weighed a little bit less than the last reward because the last reward is actually winning the game or scoring or whatever it is so this reward system gets really complicated in some of the more advanced setups in this case though you can see right here that they give a 0.1 and a 0.5 reward just for getting picking the right value and something that's actually valid instead of picking an invalid value so rewards again that's like key it's huge how do you feed the rewards back in then we have a board reset that's pretty straightforward it just goes back and resets the board to the beginning because it's going to try out all these different things while it's learning it's going to do it by trial and error so you have to keep resetting it and then of course there's the play we want to go ahead and play rounds equals 100 depends on what you want to do on here you can set this different you obviously set that to higher level but this is just going to go through and you'll see in here that we have player 1 and player 2. this is this is the computer playing itself uh one of the more powerful ways to learn to play a game or even learn something that isn't a game is to have two of these models that are basically trying to beat each other and so they always they keep finding explore new things this one works for this one so this one tries new things it beats this we've seen this in um chess i think was a big one where they had the two players in chess with reinforcement learning uh it was one of the ways they trained one of the top computer chess playing algorithms uh so this is just what this is it's going to choose an action it's going to try something and the more it try stuff the more we're going to record the hash we actually have a board hash where they self get the hash set up on here where it stores all the information and then once you get to a win one of them wins it gets the reward uh then we go back and reset and try again and then kind of the fun part we actually get down here is we're going to play with a human so we'll get a chance to come in here and see what that looks like when you put your own information in and then it just comes in here and does the same thing it did above it gives it a reward for its things or sees if it wins or ties looks at available positions all that fun of fun stuff and then finally we want to show the board so it's going to print the board out each time really as an integration is not that exciting what's exciting in here is one looking at this reward system whoops play one more up the reward system is really the heart of this how do you reward the different setup and the other one is when it's playing it's got to take an action and so what it chooses for an action is also the heart of reinforcement learning how do we choose that action and those are really key to right now where reinforcement learning is in today's technology is figuring this out how do we reward it and how do we guess the next best action so we have our environment and you can see the environment is we're going to be or the state which is kind of like what's going on we're going to return the state depending on what happens and we want to go ahead and create our agent on this place our player so each one is let me go and grab that until we look at a class player this is where a lot of the magic is really going on is what how is this player figuring out how to maneuver around the board and then the board of course returns a state that it can look at and reward so we want to take a look at this we have name self state this is class player and when you say class player we're not talking about a human player we're talking about just the computer players and this is kind of interesting so remember i told you depending on what you're doing there's going to be a decay gamma explorate these are what i'm talking about is how do we train it um as you try different moves it gets to the end the first move is important but it's not as important as the last one and so you could say that the last one has the heaviest weight and then as you as you get there the first one to see the first move gives you a five

reward the second gives you a two reward and the third one gives you a 10 reward because that's the final ending you got it the 10's gonna count more than the first step uh and here's our we're gonna get the board information coming in and then choose an action this was the second part that i was talking about that was so important so once you have your training going on we have to do a little randomness and you can see right here is our np random uniform so it's picking out a random number take a random action this is going to just pick which row and which column it is um and so choosing the action this one you can see we're just doing random states um choice length of positions action position and then it skips in there and takes a look at the board uh for p in positions it's actually storing the different boards each time you go through so it has a record of what it did so it can properly weigh the values and this simply just depends a hash state what's the last date append it to the to our states on here here's our feedback rewards the reward comes in and it's going to take a look at this and say is it none what is the reward and here is that formula remember i was telling you about up here that was important because it has decay gamma times the reward this is where as it goes through each step and this is really important this is this is kind of the heart of this of what i was talking about earlier you have step one and this might have a reward of two you have step two i should probably should have done abc this has a step three step four and so on until you get to step in and this might have a reward of ten so reward a ten we're going to add that but we're not adding let's say this one right here let's say this reward here right before 10 was um let's say it's also 10 it just makes the math easy so we had 10 and 10. we had 10 this is 10 and 10 n whatever it is but it's time it's 0.9 uh so instead of putting a full 10 here we only do 9. that's a 0.9 times 10. and so this formula as far as the decay times the reward minus the cell state value it basically adds in it says here's one or here's two i'm sorry i should have done this abc would have been easier so the first move goes in here and it puts two in here then we have our self set up on here you can see how this gets pretty complicated in the math but this is really the key is how do we train our states and we want the the final state the win to get the most points if you win you get most points and the first step gets the least amount of points so you're really training this almost in reverse you're training you're training it from the last place where you have like it says okay this is now where i was need to sum up my rewards and i want to sum them up going in reverse and i want to find the answer in reverse kind of an interesting play on the mind when you're trying to figure this stuff out and of course we want to go ahead and reset the board down here save the policy load policy these are the different things that are going in between the agent and the state to figure out what's going on let's go ahead and load that up and then finally we want to go ahead and create a human player and the human player is going to be a little different in that you choose an action row and column here's your action if action is if action in positions meaning positions that are available uh you return the action if not it just keeps asking you until you get the action that actually works and then we're going to go ahead and append to the hash state which we don't need to worry about because it returns the action up here and feed forward again this is because it's a human at the end of the game bat propagate and update state values this part isn't being done because it's not programming the model the model is getting its own rewards so we've gone ahead and loaded this in here so here's all our pieces and the first thing we want to do is set up p1 player 1 uh p2 player 2 and then we're going to send our players to our state so now it has p1 p2 and it's going to play and it's going to play 50 000 rounds now we can probably do a lot less than this and it's not going to get the full results in fact you know what uh let's go ahead and just do five just to play with it because i want to show you something here oops somewhere in there i forgot to load something there we go i must have forgot to run this run oops forgot a reference there for the board rows and columns three by three there's actually in the state it references that we just tack it on on the end it was supposed

to be at the beginning uh so now i've only set this up with um see where we go in here i've only set this up to train five times and the reason i did that is we're going to come in and actually play it and then i'm going to change that and we can see how it differs on there there we go and then you make it through a run and we're going to go ahead and save the policy so now we have our player 1 and our player 2 policy the way we set it up it has two separate policies loaded up in there and then we're gonna come in here and we're gonna do uh player one is gonna be the computer experience rate zero load policy one human player human and we're gonna go ahead and play this i remember i only went through it just one round of training in fact minimal training and so it puts an x there and i'm going to go ahead and do row 0 column 1. you can see this is very basic on here and so i put in my zero and then i'm going to go zero block it zero zero and you can see right here it let me win uh just like that i was able to win zero 2 and human winds so i only trained it 5 times we're going to run this again and this time instead of 5 let's do 5 000 or 50 000 i think that's what the guys in the back had and this takes a while to train it this is where reinforcement learning really falls apart look how simple this game is we're talking about a three by three set of columns and so for me to train it on this i could do a q table which would take which would go much quicker you could build a quick queue table with almost all the different options on there and you would probably get a the same result much quicker we're just using this as an example so when we look at reinforcement learning you need to be very careful what you apply it to it sounds like a good deal until you do like a large neural network where you're doing you set the neural network to a learning increment of one so every time it goes through it learns and then you do your actions you pick from the learning uh setup and you actually try actions on the learning setup until you get the what you think is going to be the best action so you actually feed what you think is right back through the neural network there's a whole layer there which is really fun to play with and then it has an output well think of all those processes i mean that is just a huge amount of work it's going to do let's go ahead and skip ahead here give it a moment it's going to take a minute or two to go ahead and run now to train it we went ahead and let it run and it took a while this just took i got a pretty powerful processor and it took about five minutes plus to run it and we'll go ahead and run our player set up on here oops it brought in the last whoops i brought in the last round so give me just a moment to re do the policy save there we go i forgot to save the policy back in there and then go ahead and run our player again so we've saved the policy then we want to go ahead and load the policy for p1 as the computer and we can see the computer's gone in the bottom right corner i'm going to go ahead and go one one which is the center and it's gone right up the top and if you ever played tic-tac-toe you know the computer has me but we'll go ahead and play it out row zero column two there it is and then it's gone here and so i'm going to go ahead and go row zero one two no zero one there we go and column zero that's where i want it oh and it says okay you your action there we go boom uh so you can see here we've got a didn't catch the win on this it said tie kind of funny that didn't catch the win on there but if we play this a bunch of times you'll find it's going to win more and more the more we train it the more the reinforcement happens this lengthy training process is really the stopper on reinforcement learning as this changes reinforcement learning will be one of the more powerful packages evolving over the next decade or two in fact i would even go as far as to say it is the most important machine learning tool and artificial intelligence tool out there as it learns not only a simple tic tac toe board but we start learning environments and the environment would be like in language if you're translating a language or something from one language to the other so much of it is lost if you don't know the context it's in what's the environments it's in and so being able to attach environment and context and all those things together is going to require reinforcement learning to do so again if you want to get a copy of the tic tac toe board it's kind of fun to play with run it you can test it out you can do you know test it

for different values you can switch from p1 computer where we loaded the policy one to load the policy two and just see how it varies there's all kinds of things you can do on there a quick uh recap so we're going to go back and do a reinforcement learning recap and you'll remember we have our agent our action or environment and our reward and the state and so reinforcement learning is a branch of machine learning that trains a model to come to an optimum solution for a problem by taking decisions by itself so we look at it we analyze the data make decisions solve problems it consists of an environment which an agent will interact with to learn to reach a goal or perform an action and you see here we have the dog and the action equals fetching and the first thing you're going to notice here and i'll talk a little bit more about this is that our action is kind of a singularity uh the action isn't a regression type model where we're looking at float numbers or we're trying to predict something uh or doing a reinforcement learning where we're trying to guess the value of the stock market where you have five point three three five dollars and thirty three cents this is very a little bit different and uh you'll see why q learning is so powerful but also where it falls short uh the agent is also given reward if the action performed by it is bringing is closer to the goal is leading to the goal this is done to train the model in the right direction we have a reward of a hopefully a doggy cookie and action equals fetching so it gets a reward it gets to go fetch the ball reinforcement learning can be divided based on whether the machine uses a model to learn or learns by itself so reinforcement learning it's model based learn the model given model and you can think of neural networks i think are the most common right now used for reinforcement learning there certainly are a lot of other options out there but you have this black box and you go back and you send the error back up and it trains it based on how much of an error the error could be computed based on maybe a maximum reward subtraction or something like that so the closer you get to the maximum reward the better and so we talk about a model free you have your policy optimization and your q learning and this is kind of the cool thing about q learning is it can do it's very powerful on certain things making it very quick and easy to do a reinforcement learning so what is q learning q learning is reinforcement learning policy which will fill the next best action given a current state it chooses this action at random and aims to maximize the reward and so you can see here's our standard reinforcement learning graph by now if you're doing any reinforcement learning you should be familiar with this where you have your agent your agent takes an action the action affects the environment and then the environment sends back the reward or the feedback and the state it's the new state the agent's in where is it at on the chess board where is it at in the video game um if your robots out there picking trash up off the side of the road where is it at on the road consider an ad recommendation system usually when you look up a product online you get ads which will suggest the same product over and over again using q learning we can make an ad recommendation system which will suggest related products to our previous purchase the reward will be if user clicks on the suggested product and again you can see you might have a lot of products on your web advertisement or your pages but it's still not a float number it's still a set number and that's something to be aware of when you're using q learning and you can see here that if you have a hundred people clicking on ads and you click on one of the ads it might go in there and say okay this person clicked on this ad what is the best set of ads based on clicking on this ad or these two ads afterwards based on where they are browsing so let's go ahead and look at some important terms we talk about q learning we have states the state s represents the current position of an agent in an environment the action the action a is the step taken by the agent when it is particular state rewards for every action the agent will get a positive or negative reward and again when we talk about states we're usually not with when you're using a q table you're not usually talking about float variables you're talking about true false and we'll take a closer look at that in a second and episodes when an agent ends up in a

terminating state and can't take a new action this might be if you're playing a video game your character stepped in and is now dead or whatever q values used to determine how good an action a taken at a particular state s is q a of s and temporal difference a formula used to find the q value by using the value of the current state and action and previous state in action and various i mean there's bellman's equation which basically is the equation that kind of covers what we just looked at in all those different terms the bellman equation is used to determine the values of a particular state and deduce how good it is to be in take that state the optimus the optimal state will give us the highest optimal value factor influencing q values the current state and action that's your sa so your current state and your action then you have your previous date in action which is your s i guess prime i'm not sure how how they reference that s prime a prime so this is what happened before uh then you have a reward for action so you have your r reward and you have your maximum expected future reward and you can see there's also a learning rate put in there and a discount rate so we're looking at these just like any other model we don't want to have an absolute final value on here we don't want it to if you do absolute values instead of taking smaller steps you don't really have that approach to the solution you just have it jump and then pretty soon if you jump one solution out that's what's going to be the new solution whichever one jumps up really high first kind of ruining the whole idea of doing a random selection i'll go into the random selection in just a second steps in q learning step one create an initial q table with all values initialized to zero again we're looking at zero one so are you you know here's our action we start we're an idol we took a wrong action we took a correct action and end and then we have our actions fetching sitting and running of course we're just using the dog example and choose an action and perform it update values in the table and of course when we're choosing an action we're going to kind of do something random and just randomly pick one so you start out and you sit and you have then uh then depending on that action you took you can now update the value for sitting after you start from start to sitting get the value of the reward and calculate the value the value q value using the bellman equation and so now we attach a reward to sitting and we attach all those rewards we continue the same until the table is filled with or an episode ends any and my image is going to come back to the random side of this and there's a few different formulas i use for the random setup to pick it i usually look whatever q model i'm using do their standard one because someone's usually gone in and done the math for the optimal spread but you can look at this if i have running has a reward of 10 sitting as a reward is seven fetching has a reward of five um just kind of without doing like a a means to you know using the bell curve for the means value and like i said there's some math you can put in there to pick so that you're more like so that running has even a higher chance but even if you were just going to do an average on this you could do an average a random number by adding them all together so you get 10 plus 7 plus 5 is 22 you could do 0 to 22 or 0 to 21 but 1 to 22 1 to 5 would be fetching and so forth you know the last 10. so you can just look at this as what percentage are you going to go for that particular option and then that gets your random setup in there and then as you slowly increment these up uh you see that if your idle where's one here we go sitting at the end if you're at the end of wherever you're at sitting gets a reward of one where's a good one on here oh wrong action running for a wrong action gets almost no reward so that becomes very very less likely to happen but it still might happen it still might have a percentage of coming up and that's where the random programming and q learning comes in the below table gives us an idea of how many times an action has been taken and how positively correct action or negatively wrong action it is going to affect the next state so let's go ahead and dive in and pull up a little piece of code and see what this looks like [Music] in python in this demo we'll use q learning to find the shortest path between two given points if getting your learning started is half the battle what if you could do that for free visit skill

up by simply learn click on the link in the description to know more if you've seen my videos before i like to do it in the anaconda jupiter notebook setup just because it's really easy to see and it's a nice demo and so here's my anaconda this one i'm actually using uh python36 environment that i set up in here and we'll go ahead and launch the jupiter notebook on this and once we're in our jupiter notebook which has the kernel loaded with python3 we'll go ahead and create a new python3 folder in here and we'll call this q learning and to start this demo let's go ahead and import our numpy array we'll just run that so it's imported and like a lot of these uh model programs when you're building them you spend a lot of time putting it all together and then you end up with this really short answer at the end uh and we'll we'll take a look at that as we come into it so we go ahead and start with our location to state uh so we have l1 l2 these are our nine locations one to nine and then of course the state is going to be 0 1 2 3 4. it's just a mapping of our location to a integer on there and then we have our actions our actions are simply moving from one location to another so i can go to i can go to location zero i can go to location one two three four five six seven eight uh so these are my actions i can choose these are the locations of our state and if you remember earlier i mentioned uh um that the limitation is that you you don't want to put in um a continually growing table because you can actually create a dynamic cue table where you continually add in new values as they arise because if you have float values this just becomes infinite and then your memory in your computer is gone or you know does it's not going to work at the same time you might think well that kind of really limits the the q uh t learning setup but there are ways to use it in conjunction with other systems and so you might look at uh well i do um i've been doing some work in stock um and one of the questions that comes out is to buy or sell the stock and the stake coming in might be um you might take a concrete way called buckets um where anything that you predict is going to return more than a certain amount of money um the error for that stock that you've had in the past you put those in buckets and suddenly you start putting creating these buckets you realize you do have a limited amount of information coming in you no longer have a float number you now have bucket one two three and four and then you can take those buckets put them through a q learning table and come up with the best action which stock should i buy it's like gambling stock is pretty much gambling if you're doing day trading you're not doing long term um investments and so you can start looking at it like that a lot of the current feeds say that the best algorithms used for day traders redoing it on your own is really to ask the question do i want to trade the stock yes or no and now you have it in a q learning table and now you can take it to that next level and you can see where that can be a really powerful tool at the end of doing a basic linear regression model or something what is the best investment and you start getting the best reward on there and the so if we're gonna have rewards these rewards we just create um it says uh if basically if you're uh this should match our cue table because it's gonna be uh you have your state and you have your action across the top if you remember from the dog and so we have whatever state we're in going down and then the next action and what the reward is for it and of course if you were actually doing a something more connected your reward would be based on the actual environment it's in and then we want to go ahead and create a state to location so we can map the indexes so just like we defined our rewards we're going to go and do state to location and you can see here it's a dictionary setup for location state and location to state with items and we also need to define what we want for learning rates you remember we had our two different rates as far as like learning from the past and learning from the current so we'll go ahead and set those to 0.75 and the alpha set to 0.9 and we'll see that when we do the formula and of course any of this code send a note to our simplylearn team they'll get you a copy of this code on here let's go ahead and pull there we go on the next two sections um since we're going to keep it short and sweet here we go so let's go ahead and create our agent um so

our agent is going to have our initialization where we send it all the information uh we'll define ourself gamma equals gamma we could have just set the gamma rate down here instead of submitting it it's kind of nice to keep them separate because you can play with these numbers on our self alpha then we have our location state we'll set that in here we have our choice of actions we're going to go ahead and just embed the rewards right into the agent so obviously this would be coming from somewhere else instead of from self-generated and then a self state to location equals our state to location dictionary and we go ahead and create a q learning table and i went ahead and just set the q learning table up to um zero to zero what what what the setup is uh location to state how many of them are there and this just creates an array of zero to zero setup on there and then the big part is the training we have our rewards new equals a copy of self.rewards ending state equals the self-location state in location so this is whatever we end up at rewards new equals ending state plus ending state equals 999 just kind of goes to a dead end and we start going through iterations and we'll go ahead um let's do this uh so this we're going to come back and we're going to call call it on here let me just erase that switch it to an arrow there we go so what we're doing is we're going to send in here to train it we're going to say hey i want to iterate through this a thousand times and see what happens now this part would actually be instead of iterating you might have your external environment and they're going back and forth and you iterate through outside of here but just for ease of use our agent is going to come in here and iterate through this sometimes i'll put this iteration in here and i'll have it call the environment and say hey this is what i did what's the next state and the environment does this thing right in here as i iterate through it uh and then we want to go ahead and pick a random state to start with that's what's going on here you have to start somewhere um and then you have your playable actions we're going to start with just an empty thing for playable actions and we'll fill that up so that's what choices i have and so we're going to iterate through the rewards matrix to get the states directly reachable from the randomly chosen current state assign those states to a list named playable actions and so you can see here we have a range nine i usually use length of whatever i'm looking at which is our locations or states as they are we have a reward so we want to look at the current the rewards uh the new reward is our is in our chart here of rewards underscore new uh current state um plus j uh j being what is the next state we want to try and so we go ahead and do our playable actions and we append j and so we're doing is we're randomly trying different things in here to see what's going to generate a better reward and then of course we go ahead and choose our next state so we have our random choice playable actions and if you remember i mentioned on this let me just go ahead and let's do a free form when we were talking about the next state this right here just does a random selection instead of a random selection you might do something where whatever the best selection is which might be option three here and then so you can see that it might use a bell curve and then option two over here might have a bell curve like this oops and we start looking at these averages and these spreads or we can just add them all together and pick the one that kind of goes in all of those so those are some of the options we have in here we just go with the random choice that's usually where you start play with it and then we have our reward section down here and so we want to go ahead and find well in this case the temporal difference uh so you have your rewards new plus the self gamma and this is the formula we were looking at this is bellman's equation here so we have our current value our learning rate our discount rate involved in there the reward system coming in for that and we can add it all together this is of course our maximum expected future setup in here so this is all of our our bellman's equation that we're looking at here and then we come up in here and we update our q table that's all this is on this one that's right here we have self-cue current state next date and we add in our alpha because we don't want to we don't want to train all of it at once in case there's slight difference

is coming in there we want to slowly approach the answer and then we have our route equals the start location and next location equals start location so we're just incrementing we took a step forward and then finally remember i was telling you how we're going to do all this and just have some simple thing at the end or just generates a simple path we're going to go ahead and get the optimal route we want to find the best route in here and so we've created a definition for the optimal route down here scroll down for that and we get the optimal route we go ahead and put the information in including the q table self start location in location next location route q and it says while next location is not equal to in location so while we can still go our start location equals self location to state start location so we already have our best value for the start location uh the next state looks at the q table and says hey what's uh the next one with the best value and then the next location we go ahead and pull that in and we just append it that's what's going on down here and then our start location equals the next location and we just go through all the steps and we'll go ahead and run this and now that we have our cue table our cue agent loaded we're going to go ahead and take our q agent load them up with our alpha gamma that we set up above along with the location step action reward state to location and our goal is to plot a course between l9 and l1 and we're going to go through a hundred a thousand iterations on here and so when i run that it runs pretty quick why is this so fast if you've been running neural networks and you've been doing all these other models you sit here and wait a long time well we're a very small amount of data these are all integers these aren't float values there's not a the math is not heavy on the on the processing end and this is where q tables are so powerful if you have a small amount of information coming in you very quickly get an answer off of this even though we went through it a thousand times to train it and you'll see here we have l nine eight five two and one and that's based on our reward table we had set up on there and this is the shortest path going between these different setups in here and if you remember on our reward table you can see that if you start here you can go to here there's places you can't go that's how this reward table was set up so i can only go to certain places so kind of a little maze set up in there and you can play with it this is really fun set up to play with and you can see how you can take this whole code and you can like i was saying earlier you can embed it into another setup and model and predictions where you put things into buckets and you're trying to guess the best investment the best course of action long as you can take that course into action and reduce it down to a yes no or if you're using text you can use a one hot encoder which word is next there's all kinds of things you can do with a cue table depending on just how much information you're putting in there so that wraps up our demo in this demo we've found the shortest distance between two paths based on whatever rules or state rewards we have to get from point a to point b and what available actions there are the principal component analysis we're going to cover dimensionality reduction principle component analysis what is it important pca terminologies and you'll see it abbreviated normally as pca principal component analysis pca properties pca example and then we'll pull up some python code in our jupyter notebook and have some hands-on demo on the pca and how it's used dimensionality reduction dimensionality reduction refers to the technique that reduces the number of input variables in a data set and so you can see on the table on the right shows the orders made at an automobile parts retailer the retailer sells different automobile parts from different companies and you can see we have company b-packs iso max and they have the item the tire the axle an order id a price number and a quantity in order to predict the future cells we find out that using correlation analysis that we just need three attributes therefore we have reduced the number of attributes from five to three and clearly we don't really care about the part number i don't think the part number would have an effect on how many tires are bought and even the store who's buying them probably does not have an effect on that in this case that's what they've actually done is

remove those and we just have the item the tire the price and the quantity one of the things you should be taking away from this is in the scheme of things we are in the descriptive phase we're describing the data and we're pre-processing the data what can we do to clean it up why dimensionality reduction well number one less dimensions for a given data set means less computation or training time that can be really important if you're trying a number of different models and you're re-running them over and over again and even if you have seven gigabytes of data that can start taking days to go through all those different models so this is huge this is probably the hugest part as far as reducing our data set redundancy is removed after removing similar entries from the data set again pre-processing some of our models like a neural network if you put in two of the same data it might give them a higher weight than they would if it was just once we want to get rid of that redundancy it also increases the processing time if you have multiple data coming in space required to store the data is reduced so if we're committing this into a big data pool we might not send the company that bought it why would we want to store two whole extra columns when we added into that pool of data makes the data easy for plotting in 2d and 3d plots this is my favorite part very important you're in your shareholder meeting you want to be able to give them a really good clear and simplified version you want to reduce it down to something people can take in it helps to find out the most significant features and skip the rest which also comes in in post scribing uh leads to better human interpretation that kind of goes with number four it makes data easy for plotting you have a better interpretation when we're looking at it principle component analysis so what is it uh principal component analysis is a technique for reducing the dimensionality of data sets increasing interpretability but at the same time minimizing information loss so we take some very complex data set with lots of variables we run it through the pca we reduce the variables we end up with a reduced variable setup this is very confusing to look at because if you look at the end result we have the different colors all lined up so what we're going to take a look at is let's say we have a picture here uh let's say you are asked to take a picture of some toddlers and you are deciding which angle would be the best to take the picture from so if we come up here we look at this we say okay this is you know one angle we get the back of a lot of heads not many faces so we'll do it from here we might get the one person up front smiling a lot of the people in the class are missing so we have a huge amount off to the right of blank space maybe from up here again we have the back of someone's head and it turns out that the best angle to click the picture from might be this bottom left angle you look at it you say hey that makes sense it's a good configuration of all the people in the picture now when we're talking about data it's not you really can't do it by what you think is going to be the best we need to have some kind of mathematical formula so it's consistent and so it makes sense in the back end one of the projects i worked on many years ago has something similar to the iris if you've ever done the iris data sets probably one of the most common ones out there where they have the flower and they're measuring the stamen in the petals and they have width and they have length of the petal instead of putting through the width and the length of the petal we could just as easily do the width-to-length ratio we can divide the width by the length and you get a single number where you had two that's the kind of idea that's going on into this in pre-processing and looking at what we can do to bring the data down the very simplified example on my iris pedal example when we look at the similarity in pca we find the best picture or projection of the data points and so we look down at from one angle we've drawn a line down there we can see these data points based on in this case just two variables now keep in mind we're usually talking about 36 40 variables almost all of your business models usually have about 26 to 27 different variables they're looking at same thing with like a bank loan model we're talking 26 to 36 different variables they're looking at that are going in so we want to do is we want to find the best view in this case we're just looking at the x

y we look down at it and we have our second idea pc2 and again we're looking at the x i this x y this time from a different direction here for our e's we can consider that we get two principal components namely pc1 and pc2 comparing both the principal components we find the data points are sufficiently spaced in pc1 so if you look at what we got here we have pc1 you can see along the line how the data points are spaced versus the spacing in pc2 and that's what they're coming up with what is going to give us the best look for these data points when we combine them and we're looking at them from just a single angle whereas in pc2 they are less spaced which makes the observation and further calculations much more difficult therefore we accept the pc1 and not the pc2 as the data points are more spaced now obviously the back-end calculations are a little bit more complicated when we get into the math of how they decide what is more valuable this gives you an idea though that when we're talking about this we're talking about the perspective which would help in understanding how pca analysis works we want to go ahead and do is dive into the important terminologies under a pca and important terminologies are views the perspective through which data points are observed and so you'll hear that if someone's talking about a pca presentation and they're not taking the time to reduce it to something that the average person shareholders can understand you might hear them refer to it as the different views what view are we taking dimension number of columns in a data set are called the dimensions of that data set we talked about you'll hear features dimensions and that's talking about features there's usually when you're running a business you're talking 25 26 27 different features minimal and then you have the principal component new variables that are constructed as linear combinations or mixtures of the initial variables principal component is very important it's a combination if you remember my flower example it would be the width over the length of the petal as opposed to putting both width and length in you just put in the ratio instead which is a single number versus two separate numbers projections the perpendicular distance between the principal component and the data points and that goes to that line we had earlier it's that right angle line of where those point how those points fall onto the line important properties important properties number of principal components is always less than or equal to the number of attributes that just makes common sense you're not going to do 10 principal properties with only three features you're trying to reduce them so it's just kind of goofy but it is important to remember that people will throw weird code out there and just randomly do stuff with instead of really thinking it through principal components are orthogonal and this is what we're talking about that right angle from the line when we do pc1 we're looking at how those points fall on to that line same thing with pc2 we want to make sure that pc1 does not equal pc2 we don't want to have the same two principal points when we do two points the priority of principal components decreases as their numbers increase this is important to understand if you're going to create one principle component everything is summarized into that one component as we go to two components the priority how much it holds value decreases as we go down so if you have five different points each one of those points is going to have less value than just the one point which has everything summarized in it how pca works i said there was more in the back end we talk about the math this is what we're talking about is how does it actually work so now we have understanding that you're looking at a perspective now we want to see how that math side works pca performs the following operations in order to evaluate the principal components for a given data set first we start with the standardization then we have a covariance matrix computation and we use that to generate our i gene vectors and i gene values which is the feature vector and if you remember the i gene vector is like a translation for moving the data from x equals 1 to x equals 2 or whatever altering it and the ign value is the final value that we generate when we talk about standardization the main aim of this step is to standardize the range of the attributes so that each one of them lie within similar boundaries this

process involves removal of the mean from the variable values and scaling the data with respect to the standard deviation and you can see here we have z equals the variable values minus the mean over the standard deviation the covariance matrix computation covariance matrix is used to express the correlation between any two or more attributes in multi-dimensional data set the covariance matrix has the entries as the variance and the covariance of the tribute values the variance is denoted by var and the covariance is denoted by cov on the right we can see the covariance matrix for two attributes and their values when we do a hands-on and look at the code we'll do a display of this so you can see what we're talking about and what that looks like for now you can just notice that this is a matrix that we're generating with the variance and then the covariance of x to y on the right side we can see the covariance table for more than two attributes in a multi-dimensional data set this is what i was talking about we usually are looking at not just one feature two features we're usually looking at 25 30 features going on and so if we do i was set up like this we should see all those different features as the different variables covariance matrix tells us how the two or more variables are related positive covariance indicate that the value of one variable is directly proportional to the other variable negative covariance indicate that the value of one variable is inversely proportional to the other variable that is always important to note whenever we're doing any of these matrixes that we're going to be looking at that positive and negative whether it's inverted or not and then we have the iogene values and the i gene vectors iogene values and hygiene vectors are the mathematical value that are extracted from the covariance table they are responsible for the generation of a new set of variables from the old set of variables which further lead to the construction of the principal components i gene vectors do not change directions after linear transformation i gene values are the scalars or the magnitude of the i gene vectors and again this is just chain transforming that data so we're going to change the vector b to the b prime as denoted on the chart and so when we have like multiple variables how do we calculate that new variable and then we have feature vectors feature vectors is simply a matrix that has igen vectors of the components that we decide to keep as the columns here we decide whether we must keep or discard the less significant principal components that we have generated in the above steps this becomes really important as we start looking at the back end of this and we'll do this in the demo but one of the more important steps to understand and so we have the pca example consider matrix x within rows or observations and k columns or variables now for this matrix we would construct a variable space with as many dimensions as the variable but for our simplicity let's consider this three dimensions for now now each observation row of the matrix x is placed in the k-dimensional variable space such that the rows in the data table form a swarm of points in this space now we find the mean of all the observations and then place it along the data points on the plot the first principal component is a line that best accounts for the shape of the point swarm it represents the maximal variance direction in the data each observation may be projected onto this line in order to get a coordinate value along the pc one this value is known as a score usually only one principal component is insufficient to model the systematic variation for a data set thus a second principal axis is created the second principle component is oriented such that it reflects the second largest source of variation in the data while being orthogonal to pc1 pc2 also passes through the average point let's go ahead and pull this up and just see what that means inside our python scripting i'm going to use the anaconda navigator and i will be in python 3.6 for this example i believe there's even like a 3.9 out i tend to stay in 3.6 because a lot of the models i use especially with the neural networks are stable in 3 6. and then we open up our jupiter i'm in chrome and we go ahead and create a new python 3. and for ease of use our team in the back was nice enough to put this together for me and we'll go and start with the libraries the first thing i like to do whenever i'm looking at any new setup uh well you know what

let's do let's do the libraries first we're going to do our basic libraries which is matplot library the plt from the matplot library pandas our data frame pd numpy our numbers array np seaborn for graphing sns that goes with the plot that actually sits on matplot library so the seaborn sits on there and then we have our amber sign because we're in jupiter notebook matplot library in line the newer version actually doesn't require that but i put it in there either anyway just because i'm so used to it and then we want to go ahead and take a look at the data and in this case we're going to pull in certainly you can have lots of fun with different data but we're going to use the cancer data set and one of the reasons the cancer data set is it has like 36 35 different features so it's kind of fun to use that as our base for this and we'll go ahead and run this and look at our keys and the first thing we notice in our keys for the cancer data set is we have our data we have our target our frame target names description feature names and file name so what we're looking for in all this is let's take a look at the description let's go in here and pull up the description on here i'm not going to spend a huge amount of time on the description because this is we don't want to get into a medical domain we want to focus on our pca setup what's important is you start looking at what the different attributes are what they mean if you were in the medical field you'd want to note all these different things whether what they're measuring where it's coming from you can actually see the actual different measurements are taking no missing attributes we page all the way to the bottom and you're going to have your data in this case our target and if you dig deep enough to the target let's actually do this let's go ahead and print target names real quick here i always like to just take a look and see what's on the other end of this target names run that yeah so the target name is is it malignant or is it b9 in other words is this a dangerous growth or is it something we don't have to worry about that's the bottom line with the cancer in this case and then we can go ahead and load our data and you know let me go up a just a notch here for easy of reading it's hard to get that just right and that's all you have to do uh so let's go ahead and look at our data uh our we're gonna use our pandas and we're gonna go ahead and do our data frame it's going to equal cancer data columns equals cancer feature equals feature names so remember up here we already loaded the the names up of our of the features in there what is going to come out of this let me just see if we can get to that it's at the top of target names that's just this list of names here in the setup and we can go ahead and run this code and it'll print the head and you can see here we have the mean radius the mean texture mean perimeter i don't know about you this is a wonderful data set if you're playing with it because like many of the data that most of the data that comes in half the time we don't even know what we're looking at we're just handed a bunch of stuff as a data scientist going what the heck is this and so this is a good place to start because this has a number of different features in there we have no idea what these feature means or where they come from we want to just look at the data and figure that out and now we actually are getting into the pca side of it as we've noticed before is difficult to visualize high dimensional data we can use pca to find the first two principal components and visualize the data this new two-dimensional space with a single scatter plot before we do this we need to go ahead and scale our data now i haven't run this to see if you really have to scale the data on this but as just a general run time i almost do that as the first step of any modeling even if it's premodeling as we're doing here in neural networks that is so important with pca visualization it's already going to scale it when we do the means and deviation inside the pca but just in case it's always good to scale it and then we're going to take our pca with the scikit learn uses very similar process to other pre-processing functions that come with scikit-learn we instantiate a pca object find the principal components using the fit method then apply the rotation and dimensionality reduction by calling transform we can also specify how many components we want to keep when creating the pca object and so the code for this oops getting a little bit ahead let me go and run this code

uh so the code for this is from sklearn decomposition import pca pca equals pca in components equals 2 and that's really important to note that because we're only going to want to look at two components i would never go over four components especially if you're going to demo this with somebody else if you're showing this to the shareholders the whole idea is to reduce it to something people can see and then the pca fit we're going to is going to take the scaled data that we generated up here and then you can see we've created our pca model with in components equals two now whenever i use a new tool i like to go in there and actually see what i'm using so let's go to the scikit webpage for the pca and you can see in here here's our call statement it describes what all the different setups you have on there probably the biggest one to look at would be um well the biggest one is your components how many components do you want uh which you have to put in there pretty much and then you also might look at the svd solver it's on auto right now but you can override that and do different things with it it does a pretty good job as it is and if we go down all the way down to there we go to our methods if you notice we have fit and we have fit transform nowhere in here is predict because this is not used for prediction it's used to look at the data again we're in the describe setup we're fitting the data we're taking a look at it we've already looked at our minimum maximum we've already looked at what's in each quarter we've done a full description of the data this is part of describing the data that's the biggest thing i take away when i come zooming in here and of course i have examples of it down here if you forget and the biggest one of course is the number of components and then i mean the rest you can play with the actual solver whether you're doing a full or randomize there's different things it does pretty good on the auto and now we can transform this data to its first two principal components and so we have our xpca we're going to set that equal to pca transform scaled data so there we go there's our first transformation and let's just go ahead and print the scaled data shape and the xpca data shape and the reason we want to do this is just to show us what's going on here we've taken 30 features i think i said 36 or something like that but it's 30 and we've compressed it down to two features and we decided we wanted two features and that's where this comes from we still have 569 data sets i mean data rows data sets we still have 569 rows of data but instead of computing 30 features we're now only doing our model on two features so let's go ahead and uh plot these and take a look and see what's going on and we're just gonna use our plt figure we'll set the figure size on here here's our scatter plot xpca x underscore pca of one these are two different perceptions we're using uh and then you'll see right here c for color cancer equals target and so remember we have zero we have one and if i remember correctly zero was malignant one was b9 so everything in the zero column is going to be one color and the other color is going to be one and then we're going to use the plasma map just kind of telling you what color it is add some labels first principle component second principal component and we'll go ahead and run this and you can see here instead of having a chart one of those heat maps with 30 different columns in it we can look at this and say hey this one actually did a pretty good job of separating the data and a couple things when i'm looking at this that i notice is first we have a very clear area was clumped together where it's going to be b9 and we have a huge area it's still clumped together more spread out where it's going to be malignant or i think i had that backwards and then in the middle because we're dealing with something in this particular case cancer we would try to separate i would be exploring how to separate this middle group out in other words there's an area where everything overlaps and we're not going to have a clear result on it just because those are the people you want to go in there and have extra tests or treat it differently versus going in and saying just cutting into the can into the cancer so the body absorbs it and it dissipates versus uh actively going in there removing it testing it going through chemo and all the different things that's a big difference you know as far as what's going to happen here in that middle line where the overlap is going to be

huge that's domain specific uh going back to the data we can see here clearly by using these two components we can easily separate these two classes so the next step is what does that mean interpreting the components unfortunately with this great power of dimensionality reduction comes the cost of not being able to easily understand what these components represent i don't know what principle component one looks work represents or second principle the components correspond to combinations of original features the components themselves are stored as an attribute of the filtered pca object and so we talk look at that we can go ahead and do look at the pca components this is in our model we built we've trained it we can run that and you can see here's the actual components uh it's the two components have each have their own array and within the array you can see the what the scores are using and these actually give weight to what features are doing what so in this numpy matrix array each row represents a principal component and each column relates back to the original features what's really neat about this is we can now go in reverse and drop this onto a heat map and start seeing uh what this means and so let me go ahead and just put this down i'll play already got it down here uh we'll go ahead and put this in here we're going to use our df comp data frame and we do our pca components and i want you to notice how easy this is uh we're going to set our columns equal to cancer feature names that just makes it really easy and we're dumping it into a data frame what's neat about a data frame is when we get to seaborn it will pull that data frame apart and and set it up for us we want and so we're just going to do the see the seaborne heat map of our data frame composition and we'll use the plasma coloring and it creates a nice little color graph here you can see we have the mean radius and all the different features along the bottom on the right we have a scale so we can see we have the dark colors all the way to the really light colors which are what's really shining there this is like the primary stuff we want to look at so this heat map and the color bar basically represent the correlation between the various features and the principal component itself so you know very powerful map to look at and then you can go in here and we might notice that the mean radius look how how on the bottom of the map it is um on some of this uh so you have some interesting correlations here that change the variations on that and what means what this is more when you get to a post scribe you can also use this to try to guess as what these things mean and what you want to change to get a better result regularization in machine learning so our agenda on this one is fitting the data understanding linear regression bias and variance what is overfitting what is underfitting and those are like the biggest things right now in data science is overfitting and under fitting what does that mean and what is regularization and then we'll do a quick hands-on demo to take a look at this so fitting the data let's start with fitting the data and we talk about what is data fitting it's a process of plotting a series of data points and drawing the best fit line to understand the relationship between the variables and this is what we called data fitting and you can see here we have a couple of lines we've drawn on this graph we're going to go in a little deeper on there so we might have in this case just the two dimensions we have an efficiency of the car and we have the distance traveled in 1 000 kilometers and so what is data fitting well it's a linear relationship and a linear relationship very specifically linear means line the line used to represent the relationship is a straight line that passes through the data points and the variables have linear relationship linear regression so let's start with how linear regression works a linear regression finds a line that best fits the data point and gives a relationship between the two variables and so you can see here we have the efficiency of the car versus the distance traveled and you can see this nice straight line drawn through there and when you talk about multiple variables all you're doing is putting this instead of a line it now becomes a plane it gets a little more complicated with multiple variables but they all come down to this linear kind of drawing a line through your data and finding what fits the the data the best and so we can consider an

example uh let's say that we want to find the relationship between the temperature outside versus the sales of ice cream and so we start looking at that we're looking at the how many ice cream cones we're selling or how much money we sold in ice cream and we're looking at how warm it is outside which would hopefully draw a lot of people into the ice cream store and suppose we have two lines we're going to draw l1 and l2 and we're going to kind of guess which one we think is the best fit and which claim to describe the relationship between the variables and so first we find the square of the distance between the line l1 and each data point and add them all and find the mean distance and i want you to think about that when we square something if it's a negative or positive number it no longer matters because a minus 2 squared is 4 2 squared is 4. so we're removing what side of the line it's on and we're just looking for the error in this case the mean distance of each of the little dotted lines you see here this way of calculating the square of the distance adding them and then taking the mean is called mean square error or loss function and we talk about loss how far off are we that's what we're really talking about what did we miss when we have a positive distance and a negative distance and of course when we square it it is neither it just becomes a positive error and so we take the mean square error and a lot of times you'll see it referred to as mse if i look in a code and i'm going through my python code and i see mse i know that's a mean squared error and we take all the dotted lines and we calculate this error and we add them all together and then we average it or find the means and in this case they ran a demo on this and it was uh 1127.27 for our l1 line now we find the loss function for line l2 in a similar fashion and we get the mean square error to be 6397 and it's computed the same way so maybe you put this line just way outside the data range and this is the error you get by analyzing our results we find that the loss function or the mean square error is less for l1 than l2 hence l1 is the best fit line this process describes a lot of machine learning processes it was we're going to keep guessing and get as close as we can to find the right answer we have to have some way to invite algorithm has limited flexibility to learn from data variance defines the algorithm's sensitivity to specific sets of data let's start with bias and variance you can see here we have the two different setups bias you can think is very generalized where variance is very specific uh and so we talk about bias such models pay very little attention to the training data and over simplify the model therefore the validation error or prediction error and training error follow similar trends and uh with bias if you over simplify it so much you're going to miss your local uh if if you have like a really good fit you're going to miss it you're you're going to just kind of guess what the average is and that's what your answer is going to be with variance a model with a high variance pays a lot of attention to training data and does not generalize therefore the validation error or prediction error are far apart from each other such models always lead to a high error on training and test data as a bias does where variants such models usually perform very well on training data but have high error rates on test data and i want you to think about this when we're talking about a bias the error is going to be high both when you're training it and you're testing it why because we're just kind of getting an average we're not really fitting it close with variance we're fitting it so close that the test data does really good it's going to nail it every time if you're doing categorical testing that's a car that's a truck that's a bicycle but with variants suddenly a truck has to have certain features and it might have to be red because you had so many red pictures so if it has if it's an 18 wheeler it has to be red if it's blue then it has to be a bicycle that's the kind of variance we're talking about where it picks up on something and it cannot get the right answer unless it gets a very specific data and we see that so that as you're testing it your models and you programmed it you got to look for how i trained it what is coming out and if it's not if it's not looking good on either bias or if it's not looking good on the training or on the test data then your bias then you're biasing your data if it really looks good on the training data then that's going to be your variance you've over

fitted the data and those are very important things to know when you are building your models in regression of any kind or any kind of setup for predicting so in dark games if all the data found a particular pointer this can be considered as a biased throw and the player aims for the particular score for variance if all the darts fall in different pointers and no two darts fall on the same pointer then this can be considered as a varied throw and the player aims for various scores again the bias sums everything up in one point kind of averages it together where the variance really looks for the individual predictions coming out so let's go ahead and talk about overfitting when we talk about overfitting it's a scenario where the machine learning model tries to learn from the details along with the noise and the data tries to fit each data point on the curve you can see that if you plug in your coordinates you're just going to get the whatever it's fitted every point on the data stream there's no average there's no two points that might have that you know why might have two different answers because uh if the wind blows a certain way in the efficiency of your car maybe you have a headwind so your car might alter how efficient it is as it goes and so there's going to be this variance on here and this says no you can't have any variance with you know the this is it's going to be exactly this it can't be any you can't be the same speed or the same car and have a slightly different efficiency so as the model has very less flexibility it fails to predict new data points and thus the model rejects every new data point during the prediction so you'll get like a really high error on here and so reasons for overfitting data used for training is not cleaned and contains noise garbage values in it you can spend so much time cleaning your data and it's so important it's so important that if you have if you have some kind of something wrong with the data coming in it needs to be addressed whether it's a source of the data maybe they use in medical different measuring tools so you now have to adjust for data that came in from hospital a versus hospital b or even off of machine a and machine b that's testing something and those those numbers are coming in wrong the model has a high variance again wind is a good example i was talking about that with the car you may have a hundred tests but because the wind's blowing it's all over the place size of training data used is not enough so a small amount of data is going to also cause this problem you only have a few points and you try to plot everything the model is too complex this comes up a lot we put too many pieces together and how they interact can't even be tracked and so you have to go back back break it up and find out actually what correlates and what doesn't so what is underfitting a scenario where machine learning models can either learn the relationship between the data points nor predict or classify a new data point and you can see here we have our efficiency of our car and our line drawn and it's just going to be way off for both the training and the predicting data as the model doesn't fully learn the patterns it accepts every new data point during the prediction so instead of looking for a general pattern we just kind of accept everything data used for training is not cleaned and contains noise garbage and values again under fitting and overfitting same issue you've got to clean your data the model has a high bias we've seen this in all kinds of things from [Music] the most common is the driving cars to facial identification or whatever it is the model itself when they build it might have a bias towards one thing and this would be an underfitted model would have that bias because it's averaged it out so if you have five people from india and 10 people from africa and 20 people from the u.s you created a bias because it's looking at the 20 people and you only have a small amount of data to work with size of training data used is not enough that goes with the size i was just talking about so we have a model with a high bias we have size of training data used it's not enough the model is too simple again this is one straight line through all the data when it needs a slight shift to it for other reasons so what is a good fit a linear curve that best fits the data is neither overfitting or under fitting models but is just right and of course we have the nice examples here where we have overfitting lines going up and down every point is trying to be include gluted underfitting the line

really is off from where the data is and then a good fit is got to get rid of that minimize that um error coming through regularization is taking the guess work out you're looking at this graph and you're going oh which one is that really over fit or is that under fit that's pretty hard to tell so we talk about regularization regularization techniques are used to calibrate the linear regression models and to minimize the adjusted loss function and prevent overfitting or under fitting so what that means in this case we're going to go ahead and take a look at a couple different things we're going to look at regularization which we'll start with a linear model we'll look at the ridge regularization and the lasso regularization and these models are just like just like we did the mlp the multi-layered positron in the sk learn module you could bring in the ridge module and you can bring in the lasso module so when we talk about ridge regression it modifies the overfitted or underfitted models by adding the penalty equivalent to the sum of the squares of the magnitude of the coefficients and so we have a cost function equals loss equals lambda times the sum of w squared or the absolute value of w depending on how you're doing it now remember we talked about error whether we either square it or we absolute value it because that removes a plus or minus sign on there and there's reasons to do it either way but it is more common to square the value and then we have our in this case the lambda is going to be the penalty for the errors we've thrown in a greek character for you just to confuse everybody and w is the slope of the curve of the line so uh we're going to look at this and we're going to draw a line this is going to be like a linear regression model so if you had in sk learned you could import just a standard linear regression model it would plot this line across whatever data we're working on and we look at this of course we're just extrapolating this i know they use some specific data but you don't want to get into the actual domain and so for a linear regression line let's consider two points that are on the line and we'll go ahead and have a loss equals zero considering the two points on the line we'll go ahead and do lambda equals 1. we'll set our w is going to be 1.4 then the cost function equals 0 plus 1 times 1.4 squared which equals 1.96 so really don't get caught up too much in the math on this other than understanding um that this is something that's very easy for a computer to calculate and if you ever see the loss plus the plus the lambda times w the sum of w squared and then let's say we have a ridge regression line and it does this we go ahead and plot it and we do the calculations on the data and for the ridge regression let's assume a loss equals 0.3 squared plus 0.2 squared equals 0.13 so when they put all the calculations through of the two points we end up with the 0.0.62 so we've now had a linear regression model we now had a ridge regression model and the ridge regression model plots a little differently than the standard linear regression model and comparing the two models with all the data points we can see that the ridge regression line fits the model more accurately than the linear regression line and i find this true on a lot of data i work with i'll end up using either the ridge regression model or the lasso mars regression model for fitting um especially dealing with a lot of like uh stock markets daily setup they come out slightly better you get a slightly better fit and so we have our lasso we just talked about lasso coming in here and the cost function equals instead of doing a squared we're just going to do the absolute value and so if you remember this is where ridge regression changes where's my ridge regression model we're squaring the value here and if you look at this we're not squaring the value we're just finding the absolute value on here and so the loss of this of the squared individuals um and here is our lambda symbol again penalty for errors and w equals the slope of the curve and comparing the two models with all the data points we can see that the lasso regression line fits the model more accurately than the linear regression line and this is like i said i use these two models a lot uh the ridge and this is important this is this is kind of the meat of the matter how do you know which one to use some of it is you just do it a bunch of times and then you figure it out ridge regularization is useful when we have many variables with relatively smaller data

samples the model does not encourage convergence towards zero but is likely to make them closer to zero and prevent overfitting the lasser regularization model is preferred when we are fitting a linear model with fewer variables so in the la in the iris thing we had four or five variables as we measured the different leaf pieces uh you might be doing the measurements on the cancer project which has 36 different variables um so as we get down to the iris with four variables lasso lar will probably work pretty good where you might use the ridge regularization with more model with if you have something significantly larger and it encourages the coefficients of the variables to go towards zero because of the shape of the constraint which is an absolute value and with any of this we want to go ahead and do a demo and lasso and ridge regression so let's take a look and see what that looks like in our code and bring up our jupiter notebook we'll start with our imports pandas is pd import numpy is np import matplot library as plt um sklearn we're going to import our data sets it's kind of more generic we usually just import one data set instead of all of them but you know quick and dirty when you're putting some of these together uh we have our sklearn model selection we're going to import our train test split for splitting our data up and then we'll bring in our linear regression model and we'll go ahead and run these just to load them up and then load our data set we're just talking about that you could just have imported the load boston and boston data set in there instead of loading all the data sets and then once we've loaded our data set we want to go ahead and take a look at that data and see what we got here let me just go and pop that down there and go and run it and so we've gone ahead and taken our boston data we're going to look we put it into our pandas data frame the boston data set and then the boston columns we want to see what's going on with them we have our target we have the house price etc and so our x equals boston eye location now remember in pandas the new updates to pandas they want eye location if you're going to pull data we used to be able to leave this off but it does something different it creates a slice versus a direct setup so make sure using that eye location and the i the output so this is all just bringing our data together and we can see here if we do uh we print the boston panda's head we can see here all of our different aspects we're looking for and if you're following the x and the y the x is everything except for the last column where y is uh all the it's that's what this means all the rows except for the last column and then y is all the rows but just the last column so y is our house price and the x is the crimsian industry chas knox and all these other different statistics they've collected for house sales in boston there we go oops control so we'll go ahead and split our data x train and our x test y train y test equals the train test split which we imported and we have our boston you could have easily used the x and y on here as opposed to boston eye location and we'll create our test size we're going to take 25 of the data and put it in as a test uh and then we'll go ahead and run this need an extra drink there uh so we have our train and test and then of course the print the train data shape i love doing this kind of thing whenever i'm working with this data print out the shape make sure everything looks correct so that we have 127 by 13 and 127 by one 379 by 13 they should match and if they're if the the data sets are not quite matching then you know something's wrong and you're going to get all those errors i don't know how many times i've gone through here and it's dropped a row on one of them and not on the other or something weird has happened when i'm cleaning the data this is pretty straightforward and simple because the data comes in a nice pre-package is all clean for you so let's go ahead and apply apply the multiple linear regression model and we'll call this lreg lrag linear regression and we're going to go ahead and fit that linear regression model to x-train and y-train then we'll generate the prediction on the test set so here's our l-reg y predict with our x-test going into the prediction and let's calculate that mean square error mse i told you you'll see mse used a lot people use it in variables and things like that it's pretty common and we get our mean squared error equals this is just the basic formula we've already been talking about

what's the difference squared and then we look for the average of that we'll go ahead and just run this and you can see when we get through the end of this we have our mean square error on test we have our total and then we have each column coming down and at this point unless you really know the data you're working with it's not going to mean a whole lot so if it's in your domain you might be know what you're looking at when you see these kinds of numbers coming up uh but if it's not it's just a bunch of numbers and that's okay at least that's okay for this demo uh and then we're gonna go ahead and plot these so we can see what's going on and this is always kind of fun it's always nice to have a nice visual of what you're looking at and you can see here um we plot the coefficient scores on here and we the guys in the back did a great job putting some pretty colors together making it look nice and setting up the columns you can see here uh your nox has like just a huge coefficient when i look at a table like this i look for what has very little different coefficients they're not using a huge change and what has huge changes and that flags you for all kinds of things as you're working with the data but it depends so much on the domain you're working with these are great things though as just a quick look to see what's going on with your data and what you're looking for and of course once we look at this now our motive is to reduce the coefficient score so now we want to take these and and bring them down as much as we can and for that we're going to work with the ridge regression on here so let's start by going we're going to import our ridge model the red regression from the sk learn library or the scikit and we're going to go ahead and train the model so here's our ridge r equals alpha equals one and i mentioned that earlier um when i work with the ridge model you'll see alpha equals one if you set alpha equal to zero that's a standard linear regression model so you have alpha equals one two three four and you usually use one two or three four and a standard integer on there and we'll go ahead and fit the ridge model on there with our x train and our y train data generate a prediction for that for our x test and we'll calculate the mean square error just like we did before this should all look familiar and we'll go ahead and print that out and we'll look at the ridge coefficients for our data and see what that looks like now if i jump up and down between these two you'll get a headache ah you'll still see the knox value let's just look at the knocks because that was the biggest value it's a minus nine here and if we go back up here the nox value is a minus eighteen so right off the bat i'm seeing a huge change um in the biggest coefficient there uh so if we're going to do that nice setup we want to go ahead and just print it and see what that looks like here we go and we've set up our plot subplots and again the team put together some nice colors so it makes it look good we're doing an x bar based on the columns and our l regress coefficients color equals color x spine bottom and so forth so just put together a nice little graph and you're starting to see one this when you compare this if you put on the same graph as this one up here this is up here minus 18. this is at minus 9. and so this graph is half the size of the graph above the same thing with these values here they might look the same but they're actually all almost half the value on here and then finally you can do the same thing for the lasso regression this would all look uh very similar as far as what we worked on before and i'm just going to print that on here and run it and again let's go up to uh knox look where nox is it's all the way down to zero and if we look at our next biggest coefficient it's minus 0.8 and really here's our 22.73 let me go up here um 16.7 and we go up here and we look at the same number uh 16.69 and so we look at this uh if i was running this in doing a working on a project with this i would look at these numbers i start with the 16.69 come down here and compare it to 16.78 6 9 is better than 7 8. so from the very beginning we might start looking at this first model for overall predicting but there's other factors involved we might know that the nox value is central and the other ones aren't quite as good and so we might start looking at just certain setups like what is our what is this particular coefficient because it might have a certain meaning to us and so forth and so you look at all those different items in there

again but the bottom dollar is our first model did better than our other two models our mean square error on the test set continues to come down on this today we're going to look at feature selection in machine learning what's in it for you the need for feature selection what is feature selection feature selection methods feature selection statistics so the need for feature selection to train a model we collect huge quantities of data to help the machine learn better consider a table which contains information on old cars the model decides which cars must be crushed for spare parts and when we talk about huge quantities of data there they save everything from people's favorite cat pictures to and you can imagine there's so much data out there even in a company they'll save all these little pieces of information about people and companies and corporations you need some way to sort through because if you try to run your models on all of it you'll end up with these very clunky models and they might have issues which we'll talk about later but in this case we're talking about cars and crushing but not all this data will be useful to us some classes or part of the data may not contribute much to our model and can be dropped and you can see right here we have who was the owner of the car in our data a car will not be crushed based on its previous owner so yeah that's kind of a clear cut you can see that why would i care who owned the car before once it's in the junkyard and we're crushing the cars we're not going to really care about that so here we have dropped the owner column as it does not contribute to the model having too much unnecessary data can cause the model to be slow the model may also learn from this irrelevant data and be inaccurate so feature selection is a process of reducing the input variable to your model by using only relevant data and getting rid of the noise in the data consider the database given below a library wants to donate some old books to make place in their shells for new books we want to train a model to automate this task in this case the color of the book does not matter and keeping it can cause a model to learn to donate books based on color we can remove this as a feature using feature selection we can optimize our model in several ways and so the number one is to prevent learning from noise and overfitting that's actually the huge one because we we don't want it to give us the wrong prediction and that means also improved accuracy so instead of giving us a wrong prediction we also want it to be as close to the right answer as we can get and we want to reduce the training time it's an exponential growth in some of these models so the each feature you add in increases that much more training time we talk about feature selection methods uh we put together a nice little flow chart that shows the various methods used for feature selection and you have your basic feature selection and then there is supervised and unsupervised under supervised there's intrinsic wrapper method filter method so we talk about unsupervised feature selection refers to the method which does not need the output label class for feature selection and that was uh you can see here under super unsupervised we don't have i mean that's really a growing market right now unsupervised learning and so feature selection is the same thing supervised feature selection refers to the method which uses the output label class for the feature selection and if you remember we looked at three different we have intrinsic wrapper and filter method so we're going to start with the filter method on this now remember we know what the output is so we're going to be looking at that output to see how well it's doing versus the features in this method features are dropped based on their relation to the output or how they are correlating to the output you can see here we have a set of features selecting best feature learning algorithm and then performance and so we want to find out which feature correlates to the performance on the output consider the example of book classifier here we drop the color column based on simple deduction and that kind of sums it up in the in a nutshell as we want to filter out things that clearly do not go with what we're looking for we look at the wrapper method in the wrapper method we split our data into subsets and train a model using this based on the output of the model we add and subtract features and train the model again

and you can see here in the wrapper method we have a set of features we generate a subset we run it through the algorithm and we see how each one of those subset of features performs consider the book data set by using the wrapper method we would use a subset of different features to train the machine and adjust the subset according to the output and so you can see here let's say we take a name a number of times red and we run just those and we look at the output and if we looked at them with all four inputs and look at the output we'd see quite a different variation in there and we might say you know what condition of the book and color really doesn't affect what we're looking for and you can see here we've run it on condition of the book and color depending on the output of the model we will choose our final set of features these features will give us the best result for our model and it might come up that the name number of times red is probably pretty important the intrinsic method this method combines the qualities of both filter and wrapper method to create the best subset the model will train and check the accuracy of different subsets and select the best among them we kind of looked at a little overview of some of the stuff some of the common feature selection algorithms based on which method they belong to are given below and you'll see is primarily under supervised there's not like i said a lot of unsupervised methods and the ones that are usually used these methods and finds a way to create a supervised connection between the data and we talk about supervised methods we have our filter method which we talked about and it uses like the pearson's coefficient t squared anova coefficient those are all under the filter method and in the wrapper method recursive feature elimination so remember we're choosing a subset and we want to go through there and look at each one so you're just doing a lot of loops or recursive calculations to see which one works best and which ones don't have an impact on the output and there's a lot of genetic algorithms to go with this too on the wrapper method and how they evaluate it and with the intrinsic method there's the two main ones we're looking at is the lasso regularization the lasso algorithms are basically your standard regression model so it's finding out how these different methods fit together and which ones have the best add together to have the least amount of error the other one used in the intrinsic method is the decision tree it says hey if this one is this one produces this result this one produces this result yes no which way do we go based on the input and the output variables we can choose our feature selection model so you have your numeric input coming in you have a numeric output if you use the pearson's correlation coefficient or spearman's rank coefficient you can then select what features you're going to feed into that specific model and you maybe have a numerical input and a categorical input so we're going to be looking more at a nova correlation coefficient or kindle's rank coefficient and if you have a core categorical input and a numerical output we might be looking at anova correlation coefficient in kindle's rank coefficient so based on the input and the output variables we can choose our feature selection model and you can see we have categorical to categorical we might be looking at the chi-squared test contingency tables and mutual information let's go and take a look and see uh in the python code what we're talking about here and i'm going to go ahead and use for my ide the jupiter notebook in the and they always launch it out of anaconda on here and we'll go ahead and go up here and create a new python 3 module and we'll call it feature select and since we're in python we're going to be working mainly with your numpy your pandas your matplot library so we have our number array our data frame setup which goes with the number array the numpy the pandas data frame and then we want to go ahead and graph everything so we're going to import these three modules and then we put together some data we're going to read this in it's kobe bryant i guess he's a basketball player our guys in the back we have a number of them guys both we have a lot of men and women so it's probably a misnomer our team in the back they have a some of them have a liking for basketball and they know who kobe bryant is and they want to learn a little bit more about kobe

bryant and what's going in for what whatever is going on with his game in basketball so we're going to take a look at him and once we import the data we can see what columns are available original features count so we can see how many features there are the length of it and we'll actually have a list of them and then print just the data head the top five rows and so when we do this we can see from the csv file we have 25 original features our original features are your action type combine shot type game event id and so forth there's a lot of features in here that they're recorded on all of his shots this is we talk about like a massive amount of data i mean people are sitting there and they record all this stuff and they import this stuff for different reasons but depending on what we want to look at do we really want all those features maybe the question we're going to ask is what's the chance of him making any one specific shot in right from the beginning we can look at the some of these things and say team name uh team name probably i don't know maybe it does matter because the other team might be really good at defense uh game date maybe we don't really want to look at the game date team id definitely not of importance in any of this so we look at this we have 25 features and some of these features just really don't matter to us we also have location x location y latitude and longitude i'm guessing that's the same data we've actually imported the the very similar data maybe they're slightly zoned differently but as far as our program we don't want to repeat data some of the models when you repeat data into them and this is true for most models create a huge bias they weigh that data over other data so just at a glance these are the things we're looking at we want to find out well how do we get this these features down and get rid of this bias and all these extraneous features that we don't really want to spend time running our models on and programming on and as i pointed out there's a location x a location y latitude and longitude let's take a look at that and see what we're looking at here we'll go ahead and create a plot of these and we'll just plot we'll do a scatter plot of location x and location y and then we'll do a scatter plot of data lawn data lat which is probably longitude and latitude and the scatter plot let's go ahead and actually put a little title here location and scatter on there and we'll just go ahead and plot these and when you look at this uh coming in these two graphs are pretty identical except they're flipped and so when we look at the location from which they're shooting from they're probably the same and at this point we can say okay we can get rid of one of these sets of datas we don't need both x and y and latitude and longitude because it's the same data coming in and as we look at this particular data the latitude longitude we might also ask does it really make a difference which side of the chord you're on whether you're on the left side or the right side and so we might go ahead and explore instead of looking at this as x y we might look at it as a distance and an angle and we can easily compute that and you can see we can create our data distance equals location x plus the location y squared standard euclidean geometry or triangular geometry hypotenuse squared equals the each side squared and then once we've done that we can also compute the angle so the data angle is based on the arc tangent and so forth on here so this is all this is we're just going to compute the angle here and then set that up pi over 2 to get our angle and we'll go ahead and run that and you'll see some errors run come up and that's because when we took slices over here we took a slice of a slice there's ways to fix that but it's really not important for this example so if you do see that you want to start looking up here for instead of data location x of uh not location x0 this would be like i believe the term is ilo dot ilocation if this was this is in pandas so there's different things in there but for this it doesn't really matter these are just warnings that's all they are and then let's combine our remaining minutes and seconds column into one there's another one so if you remember up here we're trying to get rid of these columns do we really need let me see if i can find it on here there we go there's our minutes remaining um and then they had what was it it was uh minus remaining and seconds column so there's also a seconds column on here let me see if i can find

that one this is where it really gets kind of crazy because here's our seconds remaining so you can see that here's our minutes remaining this gets crazy when you're looking at hundreds of these features and you can see that if if i'm going to say write a model that's going to predict a lot of this and i want it to run in this case it's a basketball and how good his shots are as the data comes in let's say i want to have it run on your phone if i'm running it across hundreds of features it's gonna just hang up on your phone where if i can get it down to just a handful we'll actually be able to come in here and run it on a smaller device and not use up as much memory or processing power so we'll go ahead and take data remaining time here and data minutes remaining times 60 plus data seconds remaining so we're just going to combine those and we'll go ahead and reprint our data so we can see what we got um coming across we have our action type combined and this is we do this a lot uh we want to take a look at oops i got so so zoomed in let me see if i can zoom out just a little bit there we go boom all right so we come up here you can see that we now have our distance our angle remaining time which is now just a number uh that computes both the minutes and seconds together and we still have we've we've been adding columns i thought you said we're supposed to subtract columns right um we're going to delete the obsolete columns when we get to them so we're just filtering out and this is a filter method we're just filtering through the things that we really don't need and next let's go ahead and explore team id and team name let me just go ahead and run that and if you look at this we have los angeles lakers and then they have the team id here and they're unique there's not that's not really anything that's going to come up because that's this particular athletes works for that team so it's the same on every line so there's another thing we can filter out on there team id and t name is just useless the whole column contains only one value each and it's pretty much useless let's go ahead and take a look at match up an opponent that's an interesting one and we see here that we have the lal versus por and the opponent is por and ind again here's a lot of duplicate information so this basically contains the same information on here again we're filtering all this stuff out and this is because we're only looking at one athlete and this might change if you're looking at multiple athletes that kind of thing now these are easy to see we might have something that looks more like this we might have something where we're looking at the distance which we computed and the shot distance are they the same thing and what we can do is we can plot that and plot them against each other on here and we see it just draws a nice straight line and so again we're looking at the same information so again we're repeating stuff and we really don't want to be running our model on repeat information on here so again it contains the same information so now let's look at the shot zone area shot zone basic shot zone range so now we're looking at the zones and what does that mean and we'll go ahead and and do this also in a scatter plot in this case we're going to just create three of these side by side so we're going to create our plot figure side 20 by 10 and then we're going to define our scatter plot by category feature and we're going to do each one set up on here give it a slightly different color and so our shot zone area is going to be plot subplot 131 scatter 131 is how that's read by the way meaning that it's number one um we have three across and this is the first one down so one one one our scatter plot by category is going to be the shot zone area we're going to plot that and then we're going to do the shot zone basic and then the shot zone range and you just push them through our definition so each of those areas go through and you'll see one three one one three two one three three again it's a one by three setup and then it's just a place on each one and so we look at this we can see that these shots uh they map out the same so it's very again redundant information that should be intuitive when we're looking at this in these color graphs it kind of helps you start looking at something you it's very intuitive like this is you start to realize that some of the stuff you'll be looking for in data you might not understand and you'll see these circular patterns where they match or they mostly match and you start to realize

when you're looking at these that they're repetitive data and then you want to explore them more closely depending on what domain you're working in so we look at these we look at them and they look just like the regions of the court but we already have stored this information in angle and distance columns so we've seen this image before we go back up here and here's our the similar image and repeating that image is down here and so let's go ahead and drop some of this in this stuff so now let's drop all the useless columns and we can drop the shot id team id team name shot zone area shot zone range shot zone basic the match up the longitude and latitude because we're putting that into distance seconds remaining minutes remaining because we combine that into one column shot distance because we have just distance on there location x location y the game event id game id all this stuff is just being dropped on here and we'll just go ahead and loop through our drops and this is a nice way of doing this because as you're playing with this this kind of data putting your list into one setup helps because then you're just running it through an iteration and you can come back and change it you might be playing with different models and do this with models you might be looking at all kinds of different things that you can drop and add in as you test these out and again we're working in the filter method so this is a lot of human interaction with the data and it takes a lot of critical thinking to look at this stuff and say what matches and what doesn't and so we look at the remaining features let me go ahead and just run this the original to the new count we had 25 features now we have 11 features you can see that right there we just circle that there's our 25 and there's uh old new now we're down to 11. so we've cut it down to less than half and you can just see the actual different information on here and the remaining time at this point we filtered it through and then we'd move into the next process which would be to run our model on this and maybe we would drop some of the features and see if it runs better or worse and what happens that's kind of would be the next step on there versus the filter setup and that would be one of the other setups depending on which algorithm you use so that wraps up our demo on filter the feature selection and going through and seeing how these different features are being repeated in this particular in a basketball setup hi everyone today we have a really interesting topic for you in this video we'll be analyzing the upcoming united states presidential election using twitter sentiment analysis in python in the course of this session we will understand in brief about the upcoming election the candidates fighting the 2020 u.s election and throw light on the result of the 2016 u.s election we look at the election forecast by some of the popular news agencies as well as pole analysis websites such as 538 before we begin i want to make it clear that this video is solely for learning purposes we had simply learned to not lean towards any party in the united states or are biased towards any candidate the analysis is based on what people have tweeted and we will understand the mood and sentiment of the public the main objective of this video is to help you understand how to extract tweets from twitter handles using python libraries store it in a csv file and perform textual analysis the results obtained are purely based on the data that we have collected let's begin the battle for the 2020 u.s presidential election has begun with the 59th united states presidential election scheduled to be held on the 3rd of november 2020 this will decide who will be in the white house for the next four years all 435 seats in the united states house of representatives 35 of the 100 seats in the united states senate and office of the president of the united states will be contested both the republican party and the democratic party have nominated their respective president and vice presidential candidates the incumbent president donald trump is the republican nominee while the 47th vice president of the united states joe biden is the democratic party's nominee the current vice president mike pence is the republican party's vice presidential nominee and is running for a second term in office senator kamala harris of california was chosen by joe biden as his running mate on 11th of august 2020. now let's see the results of the last u.s presidential election in 2016. the republican ticket of

businessman donald trump and indiana governor mike pence had defeated the democratic ticket of former secretary of state hillary clinton and u.s senator from virginia tim kaine donald trump received a total of 304 electoral votes while hillary clinton got 227 electoral votes a significant number of central states in the united states were swept by the republican party it was surprising to see that donald trump getting nearly 46.4 popular vote as compared to hillary clinton who got 48.5 votes donald trump's victory came after crucial wins in the battleground streets of florida ohio iowa north carolina and pennsylvania will donald trump be re-elected to the white house or are we going to see a new president and vice president of the united states is the big question let's see what our pollsters have to say and what has been their forecast for the upcoming election according to 538 donald trump has not gained ground since the final debate on 22nd of october at this moment joe biden is favored to win the election and he leads in both national and state pools the economist is analyzing polling economic and demographic data to predict the u.s elections their model thinks joe biden is very likely to beat donald trump in the electoral college as per their forecast joe biden has 96 percent chance of winning the electoral college and this 99 chance of biden winning the most votes the democrats have 75 chance of controlling the senate while the republicans have around 25 percent the democrats are highly likely to keep their majority in the house of representatives now according to the guardian joe biden is leading donald trump in the national polls for the presidential election but to keep in mind this does not guarantee the democratic candidate joe biden's victory hillary clinton also had a clear lead over donald trump in the polls for almost the entire 2016 campaign she ended up losing the electoral college now the bbc forecast tells that biden is leading national presidential polls with 51 percent he has hovered around 50 in recent months and has had a 10 point lead on occasions so clearly joe biden is set to be the next u.s president let's wait and watch what happens on the 3rd of november the voters of america will decide on 3rd of november whether donald trump remains in the white house for another 4 years let's perform a twitter sentiment analysis to understand the sentiments of people tweeting about donald trump and joe biden by tagging them in their tweets we have our code implemented already i'll run through each line of code and explain our analysis in detail the objective is to check the polarity of each tweet to figure out if it's positive negative or neutral we'll compare the tweets of donald trump and joe biden to see who people are favoring the most finally we learn to create a word cloud i have already extracted the tweets and stored them in two separate csv files it takes some time to fetch 10 000 tweets hence it won't be possible for me to show it to you live if you want to know how we extracted the tweets for donald trump and joe biden then please put your email ids in the comments section of the video we'll share the code file and the source file with you now you would need to install the 2p library fetch the twitter keys and access tokens and then authenticate the twitter api i've extracted 5000 tweets each for donald trump and joe biden converted into a data frame and stored it in a csv file so let me open and show you both the files so here is my data set for donald trump on the top you can see trump underscore data 5000 which means there are 5000 tweets here the first column has the number of tweets it starts from zero and goes all the way to 4999 the next column is the text column which has the content of the tweet you can see they have tagged at the rate real donald trump there are links as well there are special characters like emoticons and the third column is the user which means it's the username who had put that tweet similarly i have another data for biden you can see these are for joe biden and here you can see the content they have tagged kamala harris also similarly we have the username now let me take you straight to my jupyter notebook where i have implemented the code okay so first and foremost let's import all the required libraries so we have the numpy library for numerical computation then we have the pandas library for data manipulation next we are importing two libraries matplotlib and c bond for data visualization then

we have text blob and word cloud to get the sentiment and to build the word cloud and finally we are importing the plotly library for creating graphs let me run this cell in the current cell we'll import the data sets using read underscore csv function present in the pandas library so i have passed in my location where the data file is so i placed it on my desktop and the index underscore col parameter or column parameter refers to the columns to be used as the row labels of the data frame here 0 indicates that the first column will be used as row labels let me run it okay now we have imported both the data files let me go ahead and print the shape of one of the data frames so i'll write print let's say we'll check the shape of biden data frame so i'll write print biden dot shape i'll hit shift enter you can see there are 5000 rows and there are two columns one is the text column and the next column is the user column now let me go ahead and print the head of the data sets so the head function will return the first five rows i'll write trump dot head you can see these are the first five rows from the trump data frame and similarly if you want to check for biden i can write biden dot head you can see it here now you can also use the info method so let me show you that you can write drum dot info and this is the result so it tells there are 5000 entries from 0 to 4999 there are two columns these are the number of rows and similarly you have the data types okay moving ahead in the next cell i am displaying the 200th tweet present in the trump data frame so i have my data frame name mentioned then i have my text column and inside one more square bracket i have put 200 which means i want to see what's there in the 200th row if i hit enter so you can see this was the tweet fake fake president next let me show you the tweet number 1000 for biden so i'll write biden and inside square brackets i'll give the text column and the number is 1000 if i hit shift enter so this is the tweet now the text blob function is used in natural language processing the sentiment property returns a named tuple in the form of sentiment in brackets polarity and subjectivity let's check the sentiment of the 200th tweet made for donald trump if i hit shift enter you can see this was the polarity now minus 0.399 means the polarity was negative and you can see the subjectivity is around 0.933 similarly let's check for biden's tweet number 1000 so i'll write text blob which is my function name and inside brackets i'll give biden then pass in my column name that is text and i'll give the row number which is 1000 and then i'll give the parameter or the attribute sentiment if i hit shift enter you can see the polarity and the subjectivity now moving ahead here i have defined a function that will add another column called polarity to our two data frames trump and biden let me run this okay in the next cell i have used the apply function that will return the polarity based on the tweets you can see we have the apply function this might take some time to run it okay we are done now let me now display the top five rows of the data frame so what i'll do is i'll click on this cell go to insert and i'll hit insert cell below let's do one more okay now here i'll display the head of the data frame once again so that you can see the polarity column added you can see it here so for donald trump we have the polarity you can see the tweet and the polarity now 0.0 which means these are neutral minus is negative and anything that's in positive value indicates it's a positive tweet or it has positive polarity similarly let's see for biden i'll write biden dot head okay so here you can see the polarity for biden now this point five means it's a positive tweet okay now i want to create another column called expression where we will classify the tweets as positive negative or neutral based on the polarity value so i have used the np dot where function and the condition we have here is if polarity is greater than zero it's a positive tweet else if it is less than zero it's a negative to it and if the polarity is equal to zero it means it's a neutral tweet let me run this okay so here you can see we have the column called expression and it has classified the polarity based on negative neutral as well as positive similarly let's do the same for biden data frame as well okay before moving ahead if you want to get access to the source code and the data sets we have used in this demo then please put your email address in the chat section we'll send it to you via mail also please subscribe to

simply launch channel and hit the bell icon to never miss an update from simply loan in the next cell of code we have another function that will basically help us creating a bar plot based on the expression column from both the data frames so this is my user defined function name and it has two parameters reviews and title you can see we have grouped the expression column and found out the count using the count function then we have created or converted it into a list similarly below you can see here we have defined the color for the bars using rgb values now let me run it okay so we have successfully created our user defined function now let me go ahead and call the function and pass in my two parameters reviews and title so here we have the function name and under reviews we have the data frame name trump and the title we have given is analysis for trump let me run this okay so you can see we have a horizontal bar plot on the top you can see the title of the plot which is analysis for trump and as per the tweets it has classified as positive neutral or negative if i hover over you can see there were 2 831 neutral tweets 1211 positive tweets and similarly there were 958 negative tweets for donald trump now let me call that function once again so i'll write exp underscore graph which is my function name then i'll pass in my parameters this time i'll take biden and the title i'll put is analysis for biden if i run it okay you can see the plot for biden now if i hover over there were 1290 positive tweets for biden and 747 negative tweets okay let's scroll down now let me check the number of rows that have polarity equal to 0 the number of rows that have polarity less than 0 and greater than 0 for both the data frames using the dot shape attribute so first we have for trump now equal to equal to 0 means we are checking the total number of neutral tweets you can see the result now let's check for biden we have the result below let me run it okay now we'll drop those columns that have a polarity equal to zero from both the data frames and display the shape you can see we have the condition mentioned and we have also used the drop function so all the neutral tweets will be deleted and i'm printing the shape of the data frame there you go so out of the 5000 tweet it has deleted all the neutral tweets or dropped all the neutral treats and this is the final result in the current cell we have defined another function called balanced underscore data now this will help us get an equal number of rows and columns and delete a specified number of rows let me run it all right now i'll call that function to delete 169 rows from the trump's data frame and 37 rows from biden's data frame this will help us subset the data frames you can see we have now 2000 tweets from each of the data frames up next we'll create some visualizations to analyze the textual data so first i'm using the c bond library and creating a distribution plot for donald trump if i hit shift enter you can see this is the distribution plot this there are bars at the back and there's a trend line okay similarly let me now create a box plot so this is a box plot here you can see there are some outliers now let me add a few cells below so that i can show that plot for biden so i'll write sns dot i'll use the function dist plot and under this plot i'll give biden underscore subset and i'll pass in my column that is polarity let me run it so this is the distribution plot for joe biden and now let me also create the box plot so i'll use the c bond library and the box plot function i'll pass in my data frame which is biden underscore subset and i'll give my column name that is polarity i'll run it so here you can see the box plot for joe biden let me tell you how you can create a cross table using the group by function so before coming to this cell of code let me insert cell above okay now using group by function you can create a cross table so i'll use the trump underscore subset data frame and then i'll give my group by function inside group by function i'll give my expression column and i'll count the number of expressions if i run this you can see this is a cross table similarly you can do it for joe biden as well i'll just replace the names i'll write biden and the rest is the same you can see this is the cross table for biden underscore subset now moving ahead let's find the polarity percentage for negative and positive tweets to do this i have defined a function called poll underscore percent and we have two parameters declared subset and total let me run this okay now here i am calling the above

defined function and printing the polarity percentage for biden and trump and i have mentioned my two parameters subset and the total number so these are the values moving ahead now let's create a bar graph that will show the negative and positive polarity percentage for donald trump and joe biden let me hit shift enter you can see we have a really good bar graph now if i hover the mouse cursor you can see donald trump had 27.825 percent of positive tweets and 22.17 negative similarly joe biden had 31.6 positive tweets and 18.4 negative tweets now we want to find the public sentiment based on trump and biden tweets let me run this two cells first okay now for donald trump the total positive polarity is a combination of trump's total positive percentage and biden's negative percentage similarly for joe biden the total positive polarity is a combination of biden's total positive percentage and trump's negative percentage let me run this you can see the result here next let's plot a horizontal bar graph to visualize the public sentiment we have our x and y labels defined we also have set our bar colors with the help of rgb values and we have the title as well if i go up you can see this is the public sentiment so 26.88 is the percentage that is in favor of joe biden and for donald trump you have 23.11 now coming down now in the next cell of code we have a user defined function that will basically plot a table which has the most positive tweets for donald trump and joe biden the condition we are checking is that the polarity should be equal to one so if it's close to one or equal to one which means the tweet has positive polarity let me run it okay now let me go ahead and call the function now let me scroll over to the top you can see these are the top five positive tweets for donald trump and here you can see the top five positive tweets for joe biden likewise let's check the polarity for most negative tweets here the condition will be the polarity should be equal to minus 1 which means it's highly negative let me run it okay now let's go ahead and call the function negative tweets so i'll write n e g underscore tweets and then inside bracket i'll give my parameters which is trump underscore subset and then i'll give my title as negative tweets for donald trump okay now let's copy this we'll also check for joe biden so i'll just replace this with biden and let's say negative tweet for joe biden let's run it okay if i go on top here you can see the top negative tweets for donald trump and similarly we have the top five negative tweets for joe biden all right finally let's see how to create a word cloud now word cloud is a method to show which words are most frequently occurring among the given text i have defined a function called word cloud you can see here and it has two parameters data and title inside the built-in function or the word cloud function we have mentioned the max font size the maximum words scale value there's also background color which is white okay let's run it now this will create a word cloud so let me go ahead and call the word cloud function and pass in the arguments so i'll write word cloud which is my user defined function and i'll give my parameters trump underscore subset comma i'll give my title as word cloud for donald trump let's see donald trump tweet replies okay similarly let's do it for joe biden write word cloud biden underscore subset and then i'll give the title as word cloud for joe biden to it replies okay let me run it there you go you can see we have a nice word cloud so on the top you can see the title which says word cloud for donald trump tweet replies so here you can see some of the words like real donald trump then there's precedent love right all these similarly if i scroll down you can see the word cloud for joe biden tweet replies you can see there's the handle joe biden falls there are some other words like do taxes believe even will it's fact and you can see kamala had his name here okay now that brings us to the end of this demo on twitter sentiment analysis using text data you learned how to classify tweets and find the polarity we also looked at how to create interesting visualizations with this data and understood the mood of the public finally you saw how to create a word cloud that had the most frequently occurring words in the tweets today we'll talk about interview questions for machine learning now this video will probably help you when you're attending interviews for machine learning positions and attempt here is to probably consolidate 30 most

commonly asked questions and to help you in answering these questions we tried our best to give you the best possible answers but of course what is more important here is rather than the theoretical knowledge you need to kind of add to the answers or supplement your answers with your own experience so the responses that we put here are a bit more generic in nature so that if there are some concepts that you are not clear this video will help you in kind of getting those concepts cleared up as well but what is more important is that you need to supplement these responses with your own practical experience okay so with that let's get started so one of the first questions that you may face is what are the different types of machine learning now what is the best way to respond to this there are three types of machine learning if you read any material you will always be told there are three types of machine learning but what is important is you would probably be better off emphasizing that there are actually two main types of machine learning which is supervised and unsupervised and then there is a third type which is reinforcement learn so supervised learning is where you have some historical data and then you feed that data to your model to learn now you need to be aware of a keyword that they will be looking for which is labeled data right so if you just say past data or historical data the impact may not be so much you need to emphasize on labeled data so what is label data basically let's say if you are trying to do train your model for classification you need to be aware of for your existing data which class each of the observations belong to right so that is what is labeling so it is nothing but a fancy name you must be already aware but just make it a point to throw in that keyword labeled so that will have the right impact okay so that is what is supervised learning when you have existing labeled data which you then use to train your model that is known as supervised learning and unsupervised learning is when you don't have this label data so you have data it is not labeled so the system has to figure out a way to do some analysis on this okay so that is unsupervised learning and you can then add a few things like what are the ways of performing uh supervised learning and unsupervised learning and what are some of the techniques so supervised learning we we perform or we do regression and classification and unsupervised learning uh we do clustering and clustering can be of different types similarly regression can be of different types but you don't have to probably elaborate so much if they are asking for uh just the different types you can just mention these and just at a very high level but if they want you to elaborate give examples then of course i think there is a different question for that we will see that later then the third so we have supervised then we have unsupervised and then reinforcement you need to provide a little bit of information around that as well because it is sometimes a little difficult to come up with a good definition for reinforcement learning so you may have to little bit elaborate on how reinforcement learning works right so reinforcement learning works in in such a way that it basically has two parts to it one is the agent and the environment and the agent basically is working inside of this environment and it is given a target that it has to achieve and every time it is moving in the direction of the target so the agent basically has to take some action and every time it takes an action which is moving uh the agent towards the target right towards a goal a target is nothing but a goal okay then it is rewarded and every time it is going in a direction where it is away from the goal then it is punished so that is the way you can explain and this is used primarily or very very impactful for teaching the system to learn games and so on examples of this are basically used in alphago you can throw that as an example where alphacore used reinforcement learning to actually learn to play the game of go and finally it defeated the co-world champion all right this much of information that would be good enough okay then there could be a question on overfitting so the question could be what is overfitting and how can you avoid it so what is overfitting so let's first try to understand the concept because sometimes overfitting may be a little difficult to understand overfitting is a situation where the model has kind of memorized the data so this is an equivalent

of memorizing the data so we can draw an analogy so that it becomes easy to explain this now let's say you're teaching a child about recognizing some fruits or something like that okay and you're teaching this child about recognizing let's say three fruits apples oranges and pineapples okay so this is a small child and for the first time you're teaching the child to recognize fruits then so what will happen so this is very much like that is your training data set so what you will do is you will take a basket of fruits which consists of apples oranges and pineapples okay and you take this basket to this child and there may be let's say hundreds of these fruits so you take this basket to this child and keep showing each of this food and then first time obviously the child will not know what it is so you show an apple and you say hey this is apple then you show maybe an orange and say this is orange and so on and so forth and then again you keep repeating that right so till that basket is over this is basically how training work in machine learning also that's how training works so till the basket is completed maybe 100 fruits you keep showing this child and then the process what has happened the child has pretty much memorized these so even before you finish that basket right by the time you are halfway through the child has learnt about recognizing the apple orange and pineapple now what will happen after halfway through initially you remember it made mistakes in recognizing but halfway through now it has learned so every time you show a fruit it will exactly hundred percent accurately it will identify it will say the child will say this is an apple this is an orange and if you show a pineapple it will say this is a pineapple right so that means it has kind of memorized this data now let's say you bring another basket of fruits and it will have a mix of maybe apples which were already there in the previous set but it will also have in addition to apple it will probably have a banana or maybe another fruit like a jackfruit right so this is an equivalent of your test data set which the child has not seen before some parts of it it probably has seen like the apples it has seen but this banana and jackfruit it has not seen so then what will happen in the first round which is an equivalent of your training data set towards the end it has 100 it was telling you what the fruits are right apple was accurately recognized orange was accurately recognized and pineapples were accurately recognized right so that is like a hundred percent accuracy but now when you get another a fresh set which we're not part of the original one what will happen all the apples maybe it will be able to recognize correctly but all the others like the jackfruit or the banana will not be recognized by the child right so this is an analogy this is an equivalent of overfitting so what has happened during the training process it is able to recognize or reach 100 accuracy maybe very high accuracy okay and we call that as very low loss right so that is the technical term so the loss is pretty much zero and accuracy is pretty much hundred percent whereas when you use testing there will be a huge error which means the loss will be pretty high and therefore the accuracy will be also low okay this is known as overfitting this is basically a process where training is done training processes it goes very well almost reaching 100 accuracy but while testing it really drops down now how can you avoid it so that is a extension of this question there are multiple ways of avoiding overfitting there are techniques like what you call regularization that is the most common technique that is used for avoiding overfitting and within regularization there can be a few other subtypes like dropout in case of neural networks and a few other examples but i think if you give example or if you give regularization as the technique probably that should be sufficient so so there will be some questions where the interviewer will try to test your fundamentals and your knowledge and depth of knowledge and so on and so forth and then there will be some questions which are more like trick questions that will be more to stump you okay then the next question is around the methodology so when we are performing machine learning training we split the data into training and test right so this question is around that so the question is what is training set and test set in machine learning model and how is the split done so the question can be like that so

in machine learning when we are trying to train the model so we have a three-step process we train the model and then we test the model and then once we are satisfied with the test only then we deploy the model so what happens in the train and test is that you remember the labeled data so let's say you have 1000 records with labeling information now one way of doing it is you use all the thousand records for training and then maybe right which means that you have exposed all these thousand records during the training process and then you take a small set of the same data and then you say okay i will test it with this okay and then you probably what will happen you may get some good results all right but there is a flaw there what is the flaw this is very similar to human beings it is like you are showing this model the entire data as a part of training okay so obviously it has become familiar with the entire data so when you are taking a part of that again and you are saying that i want to test it obviously you will get good results so that is not a very accurate way of testing so that is the reason what we do is we have the label data of this thousand records or whatever we set aside before starting the training process we set aside a portion of the data and we call that test set and the remaining we call as training set and we use only this for training our model now the training process remember is not just about passing one round of this data set so let's say now your training set has 800 records it is not just one time you pass this 800 records what you normally do is you actually as a part of the training you may pass this data through the model multiple times so this thousand records may go through the model maybe 10 15 20 times till the training is perfect till the accuracy is high till the errors are minimized okay now so which is fine which means that your that is what is known as the model has seen your data and gets familiar with your data and now when you bring your test data what will happen is this is like some new data because that is where the real test is now you have trained the model and now you are testing the model with some data which is kind of new that is like a situation like a realistic situation because when the model is deployed that is what will happen it will receive some new data not the data that it has already seen right so this is a realistic test so you put some new data so this data which you have set aside is for the model it is new and if it is able to accurately predict the values that means your training has worked okay the model got trained properly but let's say while you're testing this with this test data you're getting lot of errors that means you need to probably either change your model or retrain with more data and things like that now coming back to the question of how do you split this what should be the ratio there is no fixed number again this is like individual preferences some people split it into 50 50 50 test and 50 training some people prefer to have a larger amount for training and a smaller amount for test so they can go by either 60 40 or 70 30 or some people even go with some odd numbers like 65 35 or 63.33 and 33 which is like one third and two thirds so there is no fixed rule that it has to be something that the ratio has to be this you can go by your individual preferences all right then you may have questions around data handling data manipulation or what you call data management or preparation so these are all some questions around that area there is again no one answer one single good answer to this it really varies from situation to situation and depending on what exactly is the problem what kind of data it is how critical it is what kind of data is missing and what is the type of corruption so the whole lot of things this is a very generic question and therefore you need to be little careful about responding to this as well so probably have to illustrate this again if you have experience in doing this kind of work in handling data you can illustrate with example saying that i was on one project where i received this kind of data these were the columns where data was not filled or these were the this many rows where the data was missing that would be in fact the perfect way to respond to this question but if you don't have that obviously you have to provide some good answer i think it really depends on what exactly the situation is and there are multiple ways of handling the missing data or corrupt data

now let's take a few examples now let's say you have data where some values in some of the columns are missing and you have pretty much half of your data having these missing values in terms of number of rows okay that could be one situation another situation could be that you have records or data missing but when you do some initial calculation how many records are correct or how many rows or observations as we call it has this missing data let's assume it is very minimal like 10 okay now between these two cases how do i so let's assume that this is not a mission critical situation and in order to fix this 10 percent of the data the effort that is required is much higher and obviously effort means also time and money right so it is not so mission critical and it is okay to let's say get rid of these records so obviously one of the easiest ways of handling the data part or missing data is remove those records or remove those observations from your analysis so that is the easiest way to do but then the downside is as i said in as in the first case if let's say 50 percent of your data is like that because some column or the other is missing so it is not like every in every place in every row the same column is missing but you have in maybe ten percent of the records column one is missing and another ten percent column two is missing another ten percent column three is missing so on and so forth so it adds up to maybe half of your data set so you cannot completely remove half of your data set then the whole purpose is lost okay so then how do you handle then you need to come up with ways of filling up this data with some meaningful value right that is one way of handling so when we say meaningful value what is that meaningful let's say for a particular column you might want to take a mean value for that column and fill wherever the data is missing fill up with that mean value so that when you're doing the calculations your analysis is not completely way off so you have values which are not missing first of all so your system will work number two these values are not so completely out of whack that your whole analysis goes for a task right there may be situations where if the missing values instead of putting mean may be a good idea to fill it up with the minimum value or with a zero so or with the maximum value again as i said there are so many possibilities so there is no like one correct answer for this you need to basically talk around this and illustrate with your experience as i said that would be the best otherwise this is how you need to handle this question okay so then the next question can be how can you choose a classifier based on a training set data size so again this is one of those questions uh where you probably do not have like a one-size-fits-all answer first of all you may not let's say decide your classifier based on the training set size maybe not the best way to decide the type of the classifier and even if you have to there are probably some thumb rules which we can use but then again every time so in my opinion the best way to respond to this question is you need to try out few classifiers irrespective of the size of the data and you need to then decide on your particular situation which of these classifiers are the right ones this is a very generic issue so you will never be able to just by if somebody defines a problem to you and somebody even if they show the data to you or tell you what is the data or even the size of the data i don't think there is a way to really say that yes this is the classifier that will work here no that's not the right way so you need to still you know test it out get the data try out a couple of classifiers and then only you will be in a position to decide which classifier to use you try out multiple classifiers see which one gives the best accuracy and only then you can decide then you can have a question around confusion matrix so the question can be explained confusion matrix right so confusion matrix i think the best way to explain it is by taking an example and drawing like a small diagram otherwise it can really become tricky so my suggestion is to take a piece of pen and paper and explain it by drawing a small matrix and confusion matrix is about to find out this is used especially in classification learning process and when you get the results and our model predicts the results you compare it with the actual value and try to find out what is the accuracy okay so in this case let's say this is an example of a confusion matrix and it is a binary matrix so you

have the actual values which is the labeled data right and which is so you have how many yes and how many no so you have that equation and you have the predicted values how many yes and how many know right so the total actual values the total yes is 12 plus 130 and they are shown here and the actual value those are 9 plus 3 12 okay so that is what this information here is so this is about the actual and this is about the predicted similarly the predicted values there are yes are 12 plus 3 15 yeses and no are one plus nine ten nodes okay so this is the way to look at this confusion matrix okay and out of this what is the meaning convey so there are two or three things that needs to be explained outright the first thing is for a model to be accurate the values across the diagonal should be high like in this case right that is one number two the total sum of these values is equal to the total observations in the test data set so in this case for example you have 12 plus 3 15 plus 10 25 so that means we have 25 observations in our test data set okay so these are the two things you need to first explain that the total sum in this matrix the numbers is equal to the size of the test data set and the diagonal values indicate the accuracy so by just by looking at it you can probably have a idea about is this uh an accurate model is the model being accurate if they're all spread out equally in all these four boxes that means probably the accuracy is not very good okay now how do you calculate the accuracy itself right how do you calculate the accuracy itself so it is a very simple mathematical calculation you take some of the diagonals right so in this case it is 9 plus 12 21 and divided by the total so in this case what will it be let me take a pen so your your diagonal values is equal to if i say d is equal to 12 plus 9 so that is 21 right and the total data set is equal to right we just calculated it is 25 so what is your accuracy it is 21 by your accuracy is equal to 21 by 25 and this turns out to be about 85 percent right so this is 85 so that is our accuracy okay so this is the way you need to explain draw diagram give an example and maybe it may be a good idea to be prepared with an example so that it becomes easy for you don't have to calculate those numbers on the fly right so a couple of uh hints are that you take some numbers which are with which add up to 100 that is always a good idea so you don't have to really do this complex calculations so the total value will be 100 and then diagonal values you divide once you find the diagonal values that is equal to your percentage okay all right so the next question can be a related question about false positive and false negative so what is false positive and what is false negative now once again the best way to explain this is using a piece of paper and pen otherwise it will be pretty difficult to explain this so we use the same example of the confusion matrix and we can explain that so a confusion matrix looks somewhat like this and when we just take yeah it looks somewhat like this and we continue with the previous example where this is the actual value this is the predicted value and in the actual value we have 12 plus 1 13 yeses and 3 plus 9 12 nos and the predicted values there are 12 plus 3 15 yeses and 1 plus 9 10 nos okay now this particular case which is the false positive what is a false positive first of all the second word which is positive okay is referring to the predicted value so that means the system has predicted it as a positive but the real value so this is what the false comes from but the real value is not positive okay that is the way you should understand this term false positive or even false negative so false positive so positive is what your system has predicted so where is that system predicted this is the one positive is what yes so you basically consider this row okay now if you consider this row so this is this is all positive values this entire row is positive values okay now the false positive is the one which where the value actual value is negative predicted value is positive but the actual value is negative so this is a false positive right and here is a true positive so the predicted value is positive and the actual value is also positive okay i hope this is making sense now let's take a look at what is false negative false negative so negative is the second term that means that is the predicted value that we need to look for so which are the predicted negative values this row corresponds to predicted negative values all right so this row

corresponds to predicted negative values and what they are asking for false so this is the row for predicted negative values and the actual value is this one right this is predicted negative and the actual value is also negative therefore this is a true negative so the false negative is this one predicted is negative but actual is positive right so this is the false negative so this is the way to explain and this is the way to look at false positive and false negative same way there can be true positive and true negative as well so again positive the second term you will need to use to identify the predicted row right so if we say true positive positive we need to take for the predicted part so predicted positive is here okay and then the first term is for the actual so true positive so true in case of actual is yes right so true positive is this one okay and then in case of actual the negative now we are talking about let's say true negative true negative negative is this one and the true comes from here so this is true negative right 9 is true negative the actual value is also negative and the predicted value is also negative okay so that is the way you need to explain this the terms false positive false negative and true positive true negative then you might have a question like what are the steps involved in the machine learning process or what are the three steps in the process of developing a machine learning model right so it is around the methodology that is applied so basically the way you can probably answer in your own words but the way the model development of the machine learning model happens is like this so first of all you try to understand the problem and try to figure out whether it is a classification problem or a regression problem based on that you select a few algorithms and then you start the process of training these models okay so you can either do that or you can after due diligence you can probably decide that there is one particular algorithm which is most suitable usually it happens through trial and error process but at some point you will decide that okay this is the model we are going to use okay so in that case we have the model algorithm and the model decided and then you need to do the process of training the model and testing the model and this is where if it is supervised learning you split your data the label data into training data set and test data set and you use the training data set to train your model and then you use the test data set to check the accuracy whether it is working fine or not so you test the model before you actually put it into production right so once you test the model you're satisfied it's working fine then you go to the next level which is putting it for production and then in production obviously new data will come and the inference happens so the model is readily available and only thing that happens is new data comes and the model predicts the values whether it is regression or classification now so this can be an iterative process so it is not a straightforward process where you do the training through the testing and then you move it to production now so during the training and test process there may be a situation where because of either overfitting or things like that the test doesn't go through which means that you need to put that back into the training process so that can be an iterative process not only that even if the training and test goes through properly and you deploy the model in production there can be a situation that the data that actually comes the real data that comes with that this model is failing so in which case you may have to once again go back to the drawing board or initially it will be working fine but over a period of time maybe due to the change in the nature of the data once again the accuracy will deteriorate so that is again a recursive process so once in a while you need to keep checking whether the model is working fine or not and if required you need to tweak it and modify it and so on and so forth so net net this is a continuous process of tweaking the model and testing it and making sure it is up to date then you might have question around deep learning so because deep learning is now associated with ai artificial intelligence and so on so can be as simple as what is deep learning so i think the best way to respond to this could be deep learning is a part of machine learning and then obviously the question would be then what is the difference right so deep learning you need to mention there are two key parts that

interviewer will be looking for when you are defining deep learning so first is of course deep learning is a subset of machine learning so machine learning is still the bigger let's say uh scope and deep learning is one part of it so then what exactly is the difference deep learning is primarily when we are implementing these our algorithms or when we are using neural networks for doing our training and classification and regression and all that right so when we use neural network then it is considered as deep learning and the term deep comes from the fact that you can have several layers of neural networks and these are called deep neural networks and therefore the term deep deep learning the other difference between machine learning and deep learning which the interviewer may be wanting to hear is that in case of machine learning the feature engineering is done manually what do we mean by feature engineering basically when we are trying to train our model we have our training data right so we have our training label data and this data has several let's say if it is a regular table it has several columns now each of these columns actually has information about a feature right so if we are trying to predict the height weight and so on and so forth so these are all features of human beings let's say we have census data and we have all this so those are the features now there may be probably 50 or 100 in some cases there may be 100 such features now all of them do not contribute to our model right so we as a data scientist we have to decide whether we should take all of them all the features or we should throw away some of them because again if we take all of them number one of course your accuracy will probably get affected but also there is a computational part so if you have so many pictures and then you have so much data it becomes very tricky so in case of machine learning we manually take care of identifying the features that do not contribute to the learning process and thereby we eliminate those features and so on right so this is known as feature engineering and in machine learning we do that manually whereas in deep learning where we use neural networks the model will automatically determine which features to use and which to not use and therefore feature engineering is also done automatically so this is a explanation these are two key things probably will add value to your response all right so the next question is what is the difference between or what are the differences between machine learning and deep learning so here this is a quick comparison table between machine learning and deep learning and in machine learning learning enables missions to take decisions on their own based on past data so here we are talking primarily of supervised learning and it needs only a small amount of data for training and then works well on low end systems so you don't need large machines and most features need to be identified in advance and manually coded so basically the feature engineering part is done manually and the problem is divided into parts and solved individually and then combined so that is about the machine learning part in deep learning deep learning basically enables machines to take decisions with the help of artificial neural network so here in deep learning we use neural lines so that is the key differentiator between machine learning and deep learning and usually deep learning involves a large amount of data and therefore the training also requires usually the training process requires high-end machines because it needs a lot of computing power and the machine learning features are the or the feature engineering is done automatically so the neural networks takes care of doing the feature engineering as well and in case of deep learning therefore it is said that the problem is handled end-to-end so this is a quick comparison between machine learning and deep learning in case you have that kind of a question then you might get a question around the uses of machine learning or some real life applications of machine learning in modern business the question may be worded in different ways but the meaning is how exactly is machine learning used or actually supervised machine learning it could be a very specific question around supervised decision learning so this is like give examples of supervised machine learning use of supervised machine learning in modern business so that could be the

next question so there are quite a few examples or quite a few use cases if you will for supervised machine learning the very common one is email spam detection so you want to train your application or your system to detect between spam and non-spam so this is a very common business application of supervised machine learning so how does this work the way it works is that you obviously have historical data about of your emails and they are categorized as spam and not spam so that is what is the labeled information and then you feed this information or the all these emails as an input to your model right and the model will then get trained to detect which of the emails are to detect which is spam and which is not spam so that is the training process and this is supervised machine learning because you have label data you already have emails which are tagged as spam or not spam and then you use that to train your model right so this is one example now there are a few industry specific applications for supervised machine learning one of the very common ones is in healthcare diagnostics in healthcare diagnostics you have these images and you want to train models to detect whether from a particular image whether it can find out if the person is sick or not whether a person has cancer or not right so this is a very good example of supervised machine learning here the way it works is that existing images it could be x-ray images it be mri or any of these images are available and they are tagged saying that okay this x-ray image is deflective or the person as an illness or it could be cancer whichever illness right so it is tagged as defective or clear or good image and defective something like that so we come up with a binary or it could be multi-class as well saying that this is defective to 10 percent this is 25 and so on but let's keep it simple you can give an example of just a binary classification that would be good enough so you can say that in healthcare diagnostics using image we need to detect whether a person is ill or whether a person is having cancer or not so here the way it works is you feed labeled images and you allow the model to learn from that so that when new image is fed it will be able to predict whether this person is having that illness or not having cancer or not right so i think this would be a very good example for supervised machine learning in modern business all right then we can have a question like so we've been talking about supervised and unsupervised then so there can be a question around semi-supervised machine learning so what is semi-supervised machine learning now semi-supervised learning as the name suggests it falls between supervised learning and unsupervised learning but for all practical purposes it is considered as a part of supervised learning and the reason this has come into existence is that in supervised learning you need labeled data so all your data for training your model has to be labeled now this is a big problem in many industries or in many under many situations getting the label data is not that easy because there's a lot of effort in labeling this data let's take an example of diagnostic images we can just let's say take x-ray images now there are actually millions of x-ray images available all over the world but the problem is they are not labeled so their images are there but whether it is effective or whether it is good and information is not available along with it right in a form that it can be used by a machine which means that somebody has to take a look at these images and usually it should be like a doctor and then say that okay yes this image is clean and this image is cancerous and so on and so forth now that is a huge effort by itself so this is where semi-supervised learning comes into play so what happens is there is a large amount of data maybe a part of it is labeled then we try some techniques to label the remaining part of the data so that we get completely labeled data and then we train our model so i know this is a little long winding explanation but unfortunately there is no quick and easy definition for semi-supervised machine learning this is the only way probably to explain this concept we may have another question as what are unsupervised machine learning techniques or what are some of the techniques used for performing unsupervised machine learning so it can be worded in different ways so how do we answer this question so unsupervised learning

you can say that there are two types clustering and association and clustering is a technique where similar objects are put together and there are different ways of finding similar objects so their characteristics can be measured and if they have in most of the characteristics if they are similar then they can be put together this is clustering then association you can i think the best way to explain association is with an example in case of association you try to find out how the items are linked to each other so for example if somebody bought maybe a laptop the person has also purchased a mouse so this is more in an e-commerce scenario for example so you can give this as an example so people who are buying and laptops are also buying the mouse so that means there is an association between laptops and mouse or maybe people who are buying bread are also buying butter so that is a association that can be created so this is unsupervised learning one of the techniques okay all right then we have very fundamental question what is the difference between supervised and unsupervised machine learning so machine learning these are the two main types of machine learning supervised and unsized and in case of supervised and again here probably the key word that the person may be wanting to hear is labeled data now very often people say we have historical data and if we run it it is supervised and if we don't have historical data yes but you may have historical data but if it is not labeled then you cannot use it for supervised learning so it is it's very key to understand that we put in that keyword label okay so when we have labeled data for training our model then we can use supervised learning and if we do not have label data then we use unsupervised learning and there are different algorithms available to perform both of these types of trainings so there can be another question a little bit more theoretical and conceptual in nature this is about inductive machine learning and deductive machine learning so the question can be what is the difference between inductive machine learning and deductive machine learning or somewhat in that manner so the exact phrase or exact question can vary they can ask for examples and things like that but that could be the question so let's first understand what is inductive and deductive training inductive training is induced by somebody and you can illustrate that with a small example i think that always helps so whenever you're doing some explanation try as much as possible as i said to give examples from your work experience or give some analogies and that will also help a lot in explaining as well and for the interviewer also to understand so here we'll take an example or rather we will use an analogy so inductive training is when we induce some knowledge or the learning process into a person without the person actually experiencing it okay what can be an example so we can probably tell the person or show a person a video that fire can burn the thing burn his finger or fire can cause damage so what is happening here this person has never probably seen a fire or never seen anything getting damaged by fire but just because he has seen this video he knows that okay fire is dangerous and if a fire can cause damage right so this is inductive learning compared to that what is deductive learning so here you draw a conclusion or the person draws conclusion out of experience so we will stick to the analogy so compared to the showing a video let's assume a person is allowed to play with fire right and then he figures out that if he puts his finger it's burning or if throw something into the fire it burns so he is learning through experience so this is known as deductive learning okay so you can have applications or models that can be trained using inductive learning or deductive learning all right i think uh probably that explanation will be sufficient the next question is are knn and k means clustering similar to one another or are they same right because the letter k is kind of common between them okay so let us take a little while k being common between them k n is completely different k means clustering is completely different k n n is a classification process and therefore it comes under supervised learning whereas k means clustering is actually unsupervised okay when you have k and n when you want to implement k n which is basically k nearest neighbors the value of k is a

number so you can say k is equal to 3 you want to implement k n n with k is equal to 3 so which means that it performs the classification in such a way that how does it perform the classification so it will take three nearest objects and that's why it's called nearest neighbor so basically based on the distance it will try to find out its nearest objects that are let's say three of the nearest objects and then it will check whether the class they belong to which class right so if all three belong to one particular class obviously this new object is also classified as that particular class but it is possible that they may be from two or three different classes okay so let's say they are from two classes and then if they are from two classes now usually you take a odd number you assign a odd number to so if there are three of them and two of them belong to one class and then one belongs to another class so this new object is assigned to the class to which the two of them belong now the value of k is sometimes tricky whether should you use three should you use five should you use seven that can be tricky because the ultimate classification can also vary so it's possible that if you're taking k as three the object is probably in one particular class but if you take k is equal to five maybe the object will belong to a different class because when you are taking three of them probably two of them belong to a class one and one belong to class two whereas when you take five of them it is possible that only two of them belong to class one and three of them belong to class two so which means that this object will belong to class two right so you see that so it is the class allocation can vary depending on the value of k now k means on the other hand is a clustering process and it is unsupervised where what it does is the system will basically identify how the objects are how close the objects are with respect to some of their features okay and but the similarity of course is the the letter k and in case of k means also we specify its value and it could be three or five or seven there is no technical limit as such but it can be any number of clusters that you can create okay so based on the value that you provide the system will create that many clusters of similar objects so there is a similarity to that extent that k is a number in both the cases but actually these two are completely different processes we have what is known as naive bayes classifier and people often get confused thinking that naive bayes is the name of the person who found this classifier in order to develop this classifier which is not 100 true base is the name of the person bayes is the name of the person but naive is not the name of the person right so naive is basically an english word and that has been added here because of the nature of this particular classifier naive bayes classifier is a probability based classifier and it makes some assumptions that presence of one feature of a class is not related to the presence of any other feature of maybe other classes right so which is not a very strong or not a very what do you say accurate assumption because these features can be related and so on but even if we go with this assumption this whole algorithm works very well even with this assumption and that is the good side of it but the term comes from that so that is the explanation that you can then there can be question around reinforcement learning it can be paraphrased in multiple ways one could be can you explain how a system can play a game of chess using reinforcement learning or it can be any game so the best way to explain this is again to talk a little bit about what reinforcement learning is about and then elaborate on that to explain the process so first of all reinforcement learning has an environment and an agent and the agent is basically performing some actions in order to achieve a certain goal and this goals can be anything either if it is related to game then the goal could be that you have to score very high score a high value high number or it could be that your number of lives should be as high as possible don't lose life so this could be some of them more advanced examples could be for driving the automotive industry self-driving cars they actually also make use of reinforcement learning to teach the car how to navigate through the roads and so on and so forth that is also another example now how does it work so if the system is basically there is an agent and environment and every time the agent takes a step or

performs a task which is taking it towards the goal the final goal let's say to maximize the score or to minimize the number of lives and so on or minimize the debts for example it is rewarded and every time it takes a step which goes against that code right contrary or the reverse direction it is penalized okay so it is like a carrot and a stick system now how do you use this to create a game of chess or to create a system to play a game of chess now the way this works is and this could probably go back to this alphago example where alphago defeated a human champion so the way it works is in reinforcement learning the system is allowed for example in this case we are talking about chess so we allow the system to first of all watch playing a game of chess so it could be with the human being or it could be the system itself there are computer games of chess right so either this new learning system has to watch that game or watch a human being play the game because this is reinforcement learning is pretty much all visual so when you're teaching the system to play a game the system will not actually go behind the scenes to understand the logic of your software of this game or anything like that it is just visually watching the screen and then it learns okay so reinforcement learning to a large extent works on that so you need to create a mechanism whereby your model will be able to watch somebody playing the game and then you allow the system also to start playing the game so it pretty much starts from scratch okay and as it moves forward it it's at right at the beginning and the system really knows nothing about the game of chess okay so initially it is a clean slate it just starts by observing how you're playing so it will make some random moves and keep losing badly but then what happens is over a period of time so you need to now allow the system or you need to play with the system not just one two three four or five times but hundreds of times thousands of times maybe even hundreds of thousands of times and that's exactly how alphago has done it played millions of games between itself and the system right so for the game of chess also you need to do something like that you need to allow the system to purchase and then learn on its own over a period of repetition so i think you can probably explain it to this much to this extent and it should be sufficient now this is another question which is again somewhat similar but here the size is not coming into picture so the question is how will you know which machine learning algorithm to choose for your classification problem now this is not only classification problem it could be a regression problem i would like to generalize this question so if somebody asks you how will you choose how will you know which algorithm to use the simple answer is there is no way you can decide exactly saying that this is the algorithm i am going to use in a variety of situations there are some guidelines like for example you will obviously depending on the problem you can say whether it is a classification problem or a regression problem and then in that sense you are kind of restricting yourself to if it is a classification problem there are you can only apply a classification algorithm right to that extent you can probably let's say limit the number of algorithms but now within the classification algorithms you have decision trees you have svm you have logistic regression is it possible to outright say yes so for this particular problem since you have explained this now this is the exact algorithm that you can use that is not possible okay so we have to try out a bunch of algorithms see which one gives us the best performance best accuracy and then decide to go with that particular algorithm so in machine learning a lot of it happens through trial and error there is no real possibility that anybody can just by looking at the problem or understanding the problem tell you that okay in this particular situation this is exactly the algorithm that you should use then the questions may be around application of machine learning and this question is specifically around how amazon is able to recommend other things to buy so this is around recommendation engine how does it work how does the recommendation engine work so this is basically the question is all about so the recommendation engine again works based on various inputs that are provided obviously something like uh you know amazon a website or

e-commerce site like amazon collects a lot of data around the customer behavior who is purchasing what and if somebody is buying a particular thing they are also buying something else so this kind of association right so this is the unsupervised learning we talked about they use this to associate and link or relate items and that is one part of it so they kind of build association between items saying that somebody buying this is also buying this that is one part of it then they also profile the users right based on their age their gender their geographic location they will do some profiling and then when somebody is logging in and when somebody is shopping kind of the mapping of these two things are done they try to identify obviously if you have logged in then they know who you are and your information is available like for example your age may be your agenda and where you're located what you purchased earlier right so all this is taken and the recommendation engine basically uses all this information and comes up with recommendations for a particular user so that is how the recommendation engine work all right then the question can be uh something very basic like when will you go for classification versus regression right when do you do classification instead of regression or when you use classification instead of regression now yes so so this is basically going back to the understanding of the basics of classification and regression so classification is used when you have to identify or categorize things into discrete classes so the best way to respond to this question is to take up some examples and use it otherwise it can become a little tricky the question may sound very simple but explaining it can sometimes be very tricky in case of regression we use of course there will be some keywords that they will be looking for so just you need to make sure you use those keywords one is the discrete values and other is the continuous values so for regression if you are trying to find some continuous values you use regression whereas if you are trying to find some discrete values you use classification and then you need to illustrate what are some of the examples so classification is like let's say there are images and you need to put them into classes like cat dog elephant tiger something like that so that is a classification a problem or it can be that is a multi-class classification problem it could be binary classification problem like for example whether a customer will buy or he will not buy that is a classification binary classification it can be in the weather forecast area now weather forecast is again a combination of regression and classification because on the one hand you want to predict whether it's going to rain or not that's a classification problem that's a binary classification right whether it's going to rain or not rain however you also have to predict what is going to be the temperature tomorrow right now temperature is a continuous value you can't answer the temperature in a yes or no kind of a response right so what will be the temperature tomorrow so you need to give a number which can be like 20 degrees 30 degrees or whatever right so that is where you use regression one more example is stock price prediction so that is where again you will use regression so these are the various examples so you need to illustrate with examples and make sure you include those keywords like discrete and continuous so the next question is more about a little bit of a design related question to understand your concepts and things like that so it is how will you design a spam filter so how do you basically design or develop a spam filter so i think the main thing here is he is looking at probably understanding your concepts in terms of what is the algorithm you will use or what is your understanding about difference between classification and regression and things like that and the process of course the methodology and the process so the best way to go about responding to this is we say that okay this is a classification problem because we want to find out whether an email is a spam or not spam so that we can apply the filter accordingly so first thing is to identify what type of a problem it is so we have identified that it is a classification then the second step may be to find out what kind of algorithm to use now since this is a binary classification problem logistic regression is a very common very common algorithm but however right as i said earlier also we

can never say that okay for this particular problem this is exactly the algorithm that we can use so we can also probably try decision trees or even support vector missions for example svm so we will kind of list down a few of these algorithms and we will say okay we want we would like to try out these algorithms and then we go about taking your historical data which is the labeled data which are marked so you will have a bunch of emails and then you split that into training and test data sets you use your training data set to train your model that or your algorithm that you have used or rather the model actually so and you actually will have three models let's say you are trying to test out three algorithms so you will obviously have three models so you need to try all three models and test them out as well see which one gives the best accuracy and then you decide that you will go with that model okay so training and tests will be done and then you zero in on one particular model and then you say okay this is the model will you use we will use and then go ahead and implement that or put that in production so that is the way you design a spam file the next question is about random forest what is random forest so this is a very straightforward question however the response you need to be again a little careful while we all know what is random forest explaining this can sometimes be tricky so one thing is random forest is kind of in one way it is an extension of decision trees because it is basically nothing but you have multiple decision trees and trees will basically you will use for doing if it is classification mostly it is classification you will use the trees for classification and then you use voting for finding the the final class so that is the underlyings but how will you explain this how will you respond to this so first thing obviously we will say that random forest is one of the algorithms and the more important thing that you need to probably the interviewer is waiting to hear is ensemble learner right so this is one type of ensemble learner what is ensemble learner ensemble learner is like a combination of algorithms so it is a learner which consists of more than one algorithm or more than one maybe models okay so in case of random forest the algorithm is the same but instead of using one instance of it we use multiple instances of it and we use so in a way that is a random forest is an ensemble learner there are other types of ensemble learners where we have like we use different algorithms itself so you have one maybe logistic regression and a decision tree combined together and so on and so forth or there are other ways like for example splitting the data in a certain way and so on so that's all about ensembl we will not go into that but random foreign i think the interviewer will be happy to hear this word ensemble learners and so then you go and explain how the random forest works so if the random forest is used for classification then we use what is known as a voting mechanism so basically how does it work let's say your random forest consists of 100 trees okay and each observation you pass through this forest and each observation let's say it is a classification problem binary classification 0 or 1 and you have 100 trees now if 90 trees say that it is a 0 and 10 of the trees say it is a 1 you take the majority you may take a vote and since 90 of them are saying 0 you classify this as 0 then you take the next observation and so on so that is the way uh random forest works for classification if it is a regression problem but somewhat similar but only thing is instead of vote what we will do is sorry in regression remember what happens you actually calculate a value right so for example you're using regression to predict the temperature and you have 100 trees and each tree obviously will probably predict a different value of the temperature they may be close to each other but they may not be exactly the same value so these 100 trees so how do you now find the actual value the output for the entire forest right so you have outputs of individual trees which are a part of this forest but then you need to find the final output of the forest itself so how do you do that so in case of regression you take like an average or the mean of all the 100 trees right so this is also a way of reducing the error so maybe if you have only one tree and if that one tree makes a error it is basically 100 percent wrong or 100 right right but if you have on the other hand if you have a bunch of trees you are

basically mitigating that reducing that error okay so that is the way random forest works so the next question is considering the long list of machine learning algorithms how will you decide on which one to use so once again here there is no way to outright say that this is the algorithm that we will use for a given data set this is a very good question but then the response has to be like again there will not be a one size fits all so we need to first of all you can probably shorten the list in terms of by saying okay whether it is a classification problem or it is a regression problem to that extent you can probably shorten the list because you don't have to use all of them if it is a classification problem you only can pick from the classification algorithms right so for example if it's a classification you cannot use linear regression algorithm there or if it is a regression problem you cannot use svm or maybe now you can use svm but maybe a logistic regression right so to that extent you can probably shorten the list but still you will not be able to 100 decide on saying that this is the exact algorithm that i am going to use so the way to go about is you choose a few algorithms based on what the problem is you try out your data you train some models of these algorithms check which one gives you the lowest error or the highest accuracy and based on that you choose that particular algorithm okay all right then they can be questions around bias and variance so the question can be what is bias and variance in machine learning so you just need to give out a definition for each of these for example a bias in machine learning it occurs when the predicted values are far away from the actual values so that is a bias okay and whereas they are all all the values are probably they are far off but they are very near to each other though the predicted values are close to each other right while they are far off from the actual value but they are close to each other you see the difference so that is bias and then the other part is your variance now variance is when the predicted values are all over the place right so the variance is high that means it may be close to the target but it is kind of very scattered so the points the predicted values are not close to each other right in case of buyers the predicted values are close to each other but they are not close to the target but here they may be close to the target but they may not be close to each other so they are a little bit more scattered so that is what in case of a variance okay then the next question is about again related to bias and variance what is the trade-off between bias and variance yes i think this is a interesting question because these two are heading in different directions so for example if you try to minimize the bias variance will keep going high and if you try to minimize the variance bias will keep going high and there is no way you can minimize both of them so you need to have a trade-off saying that okay this is the level at which i will have my bias and this is the level at which i will have variance so the trade-off is that pretty much attack you you decide what is the level you will tolerate for your bias and what is the level you will tolerate for variance and a combination of these two in such a way that your final results are not way off and having a trade-off will ensure that the results are consistent right so that is basically the output is consistent and which means that they are close to each other and they are also accurate that means they are as close to the target as possible right so if either of these is high then one of them will go off the track define precision and recall now again here i think it would be best to draw a diagram and take the confusion matrix and it is very simple the definition is like a formula your precision is true positive by true positive plus false positive and your recall is true positive by true positive plus false negative okay so that's you can just show it in a mathematical way that's pretty much you know that can be shown that's the easiest way to define so the next question can be about decision tree what is decision tree pruning and why is it so basically decision trees are really simple to implement and understand but one of the drawbacks of decision trees is that it can become highly complicated as it grows right and the rules and the conditions can become very complicated and this can also lead to overfitting which is basically that during training you will get 100 accuracy but when you are doing testing you will get a lot of

errors so that is the reason pruning needs to be done so the purpose or the reason for doing decision tree pruning is to reduce overfitting or to cut down on overfitting and what is decision tree pruning it is basically that you reduce the number of branches because as you may be aware a tree consists of the root node and then there are several internal nodes and then you have the leaf nodes now if there are too many of these internal nodes that is when you face the problem of overfitting and pruning is the process of reducing those internal nodes all right so the next question can be what is logistic regression uh so basically logistic regression is um one of the techniques used for performing classification especially binary classification now there is something special about logistic regression and there are a couple of things you need to be careful about first of all the name is a little confusing it is called logistic regression but it is used for classification so this can be sometimes confusing so you need to probably clarify that to the interviewer if it's really you know if it is required and they can also ask this like a trick question right so that is one part second thing is the term logistic has nothing to do with the usual logistics that we talk about but it is derived from log so that the mathematical derivation involves log and therefore the name logistic regression so what is logistic regression and how is it used so logistic regression is used for binary classification and the output of a logistic regression is either a 0 or a 1 and it varies so it's basically it calculates a probability between 0 and 1 and we can set a threshold that can vary typically it is 0.5 so any value above 0.5 is considered as 1 and if the probability is below 0.5 it is considered as zero so that is the way we calculate the probability or the system calculates the probability and based on the threshold it sets a value of zero or one which is like a binary classification zero or one okay then we have a question around k nearest neighbor algorithm so explain k nearest neighbor algorithm so first of all what is a k nearest neighbor algorithm this is a classification algorithm so that is the first thing we need to mention and we also need to mention that the k is a number it is an integer and this is variable and we can define what the value of k should be it can be 2 3 5 7 and usually it is an odd number so that is something we need to mention technically it can be even number also but then typically it would be odd number and we will see why that is okay so based on that we need to classify objects okay we need to classify objects so again it will be very helpful to draw a diagram you know if you're explaining i think that will be the best way so draw some diagram like this and let's say we have three clusters or three classes existing and now you want to find for a new item that has come you want to find out which class this belongs to right so you go about as the name suggests it you go about finding the nearest neighbors right the points which are closest to this and how many of them you will find that is what is defined by k now let's say our initial value of k was five so you will find the k the five nearest data points so in this case as it is illustrated these are the five nearest data points but then all five do not belong to the same class or cluster so there are one belonging to this cluster one the second one belonging to this cluster two three of them belonging to this third cluster so how do you decide that's exactly the reason we should as much as possible try to assign an odd number so that it becomes easier to assign this so in this case you see that the majority actually if there are multiple classes then you go with the majority so since three of these items belong to this class we assign which is basically the in in this case the green or the tennis or the third cluster as i was talking about right so we assign it to this third class so in this case it is uh that's how it is decided okay so k nearest neighbor so first thing is to identify the number of neighbors that are mentioned as k so in this case it is k is equal to 5 so we find the five nearest points and then find out out of these five which class has the maximum number in that and and then the new data point is assigned to that class okay so that's pretty much how k nearest neighbors work welcome to this video on machine learning roadmap 2021 ai and machine learning have been the top burst words in 2020. machine learning can be an interesting career move for programmers developers and

other professionals looking for a shift in their career over the last few years machine learning and artificial intelligence together have become a crucial component in many of the world's most popular business applications as we approach 2021 it's a good time for us to take a look at the machine learning roadmap for 2021. in this video we will discuss the basics of machine learning and the company is actively hiring for machine learning roles you will understand the skills required to become a machine learning expert in 2021 the salary of a machine learning engineer and how simply learn can help you get certified in machine learning now let's look at what machine learning is machine learning is an application of artificial intelligence that allows systems to learn from vast volumes of data it enables you to process analyze and train machines using data and create models to solve business problems machine learning has the potential to learn from structured semi-structured and unstructured data there are various supervised unsupervised and reinforcement learning algorithms used to build machine learning models some of the most popular machine learning algorithms are linear regression logistic regression support vector machines k nearest neighbors k means clustering knife base and others now some of the top companies hiring for machine learning roles are microsoft spotify google cell ericsson oracle and walmart there are numerous product based service based as well as startup companies that are hiring for machine learning positions now learning machine learning can position an individual for a variety of exciting careers in a growing number of industries with machine learning becoming more widely accepted and adapted technology let's look at the important skills you need to know to become a machine learning expert in 2021 first we have programming machine learning mostly depends on algorithms which means one should possess sound knowledge of different programming languages such as python and r you should have knowledge of basic programming concepts and understand data structures this will help you write better and efficient codes you should also know about searching sorting and optimization algorithms the second skill you need to know is mathematics and statistics statistics is the backbone of data analytics you should have knowledge of various measures such as mean median variance etc distributions such as uniform normal binomial poison etc and analysis methods such as anova manova t-test chi-square test hypothesis testing and others now this is required for building and validating models from observed data a basic understanding of high school mathematics will help you understand how machine learning algorithms work the topics include probability linear algebra and calculus the third skill you need to know is database and sql very often machine learning tasks are carried out using data stored in the form of tables that are present in database servers so it's important to know about databases and sql good understanding of relational databases and no databases such as mysql postgresql oracle database microsoft sql server is vital to store manipulate retrieve and handle structured data now we have data wrangling now there's a lot of pre-processing of data that is needed before you start building your machine learning model collecting the data cleaning it identifying the problems asking the right questions and formatting the data are crucial steps in machine learning you should have knowledge of libraries and packages such as numpy pandas plier deep flyer and idea to perform data wrangling next we have data visualization while working on any machine learning problem it's important to understand the data well data visualization and exploratory data analysis skills are very important to show the hidden trends and insights in a visual form using various charts and graphs this will help you find out unseen patterns in the data you need to have experience with matplotlib c-bond plotly and ggplot libraries it would be an advantage to know bi tools such as tableau and power bi finally the most important skill is machine learning itself you need to learn different machine learning techniques and algorithms that are widely used to solve business problems then implement the algorithms using python and r libraries such as scikit-learn tensorflow garrett and mlr having knowledge of

natural language processing computer vision and time series analysis would be a really good advantage to gain more experience in solving real world problems you need to work on small projects using variety of data sets data science and machine learning challenges such as those on github and kegel are a great way to get exposed to different kinds of problems and their nuances now talking about the salary of a machine learning engineer according to glassdoor in the united states a machine learning engineer can earn around 114 thousand dollars per annum while in india you can earn nearly 7 lakh 73 000 rupees per annum this salary may vary based on your experience the industry you are applying for and the company policy now moving on to the final section let me tell you how simply learn can help you start your career in machine learning so let me take you to our website first okay so i am on the chrome browser let me search for simplylearn.com and here under what you want to learn let me type machine learning it will show me the relevant courses that simply learn offers in the machine learning category so you can see here there are multiple courses first let me open the first link and let's open the second link as well so so this is the post graduate program in ai in machine learning which is in collaboration with purdue university and ibm if i scroll down you can see the key features of this course so we'll get purdue alumni association membership industry recognized ibm certificates enrollment to simply launch job assist there's 25 plus hands-on projects on gpu enabled labs you have 450 plus hours of applied learning capstone projects in three domains and much more and here you can see on the right this is the certificate that you will get after completing this program and you will also get certificates recognized by ibm and another key feature of this course is you can enroll to simply learn job assist program so you will get im jobs crew membership for six months resume assistance and career monitoring there's interview preparation and career affairs if i scroll further here you can see the learning path and you will learn about python for data science this machine learning deep learning with tensorflow and carers that's advanced deep learning and computer vision you will also learn about natural language processing that is nlp and speech recognition we have reinforcement learning and you also have the opportunity to select a few electives so we have ibm watson for chat bots machine learning with r there's git and github training and two others here you can see the skills that will be covered so you will learn about statistics which is a core component of machine learning we learn about python supervised learning there's dance computer vision tensorflow there's reinforcement learning speech recognition you will also learn about numpy pandas and other libraries and here you can see these are some of the tools that will be covered in this course if i scroll further now this is the important section you can see the industry projects that you will get to work on so this is on twitter there's one on zumato one on uber and mercedes benz as well now they are our course advisors so please go ahead and enroll to this postgraduate program in ai machine learning if you want to kick start your career in machine learning now the next course we have is machine learning certification course if i scroll down you can see the details of this course so you will gain expertise with 25 plus hands-on exercises you will get to work on four real-life industry based projects with integrated labs there will be dedicated mentoring sessions from industry experts 44 hours of instructor-led training with certification here on the right you can see the skills that will be covered so we learn about time series modeling linear and logistic regression you will also learn about support vector machines games clustering knife base decision trees random forest and you learn the concepts of bagging and boosting and deep learning fundamentals as well here you can see the course content and if i scroll further now these are the projects that you'll get to work on and finally you'll receive this certificate once you complete the course so that brings us to the end of this video on machine learning full course i hope it was useful and interesting if you enjoyed the video then please subscribe to the simply learn channel thank you for watching and keep learning you